SIMATS ENGINEERING

Department of Computer Science and Engineering

CSA37 – Software Testing

ASSIGNMENT 3 – CO3

Analyze, Compare and Justify Unit, Integration, System, Regression and Acceptance Testing Strategies for a Railway Ticket Reservation System

CO3: Analyze unit, integration, system, regression and acceptance testing for validating software systems.

Student Name	TANUJA P	**Register Number**	192321052
Department / Section	IT	**Academic Year / Semester**	2025-2026 / 7th sem
Faculty Name	Dr Gnanajeyaraman	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission		**Date of Evaluation**	

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Analyze Unit Testing and Unit Test Planning	4		
2	Analyze Unit Test Design	4		
3	Build and Analyze a Test Harness	4		
4	Analyze Integration Strategies	5		
5	Plan Integration Testing	5		
6	Analyze System Testing	5		
7	Analyze Regression Testing	5		
8	Analyze Alpha, Beta and Acceptance Testing	4		
9	Integrate Multi-Level Testing	6		
10	Re-analyze and Adapt the Test Strategy	8		
TOTAL		50		Percentage:

OVERALL REMARKS

TANUJA P		
Student Signature	**Faculty Signature**	**Program Director**

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = present ✗ = missing / not satisfied P = partly present. Based on the "Evidence Expected" of each task.

Task	Evidence expected	Status	Note
1	A unit test plan, a faults-found/coverage/effort table for the three approaches, and a justified approach with its exit criterion.	✓	Table 1, Table 2; Figure 1
2	Three unit test suites for refund_amount, a faults-found and maintenance table, and a justified choice of design style.	✓	Table 6, Table 7; Annexure F
3	Three working harnesses, a defects-found and effort table, and a justified choice with the limits of each harness stated.	✓	Table 10; Figure 3, Figure 4
4	A dependency diagram, stubs and drivers table, a faults-located table for the three strategies, and a justified integration strategy.	✓	Figure 5; Table 12, Table 13
5	Three integration test plans, a checklist score table, interface test cases for B→P, and a justified plan with entry and exit criteria.	✓	Table 16, Table 18, Table 17
6	A system test table for the three focus areas, load-test results with a response-time chart, and a justified allocation of effort.	✓	Table 20, Table 21; Figure 8
7	A table of tests selected, time and faults detected for the three strategies, an APFD calculation, and a justified regression strategy.	✓	Table 25, Table 26; Figure 10
8	An alpha plan, beta feedback form and acceptance test cases, an issue-type table for the three approaches, and a justified acceptance sequence.	P	Table 28, Table 29, Table 30; volunteer sheet Table 31 to be filled
9	A measured comparison of three strategies by level, an original block diagram and decision table, a transfer table for other systems, and a defended final strategy.	✓	Table 35, Table 38, Table 39; Figure 13
10	Three adaptation strategies applied and compared with measured and calculated evidence, and a RETAIN / MODIFY / REDESIGN decision that follows from the changed constraints.	✓	Table 42, Table 44; Figure 18, Figure 19

TEST-LEVEL EVIDENCE CHAIN (Tasks 1–10)

Evidence item	Status	Observation
Unit faults, coverage and effort per approach	✓	Table 2, Table 6; coverage.py branch figures
Harness defects, effort and run time	✓	Table 10; a real defect (H0) found by the virtual gateway
Integration order, stubs/drivers and fault location	✓	Table 12, Table 13
System, load, recovery and security results	✓	Table 20, Table 21; load model calibrated from measured times
Regression selection, time saved and APFD	✓	Table 25, Table 26
Acceptance criteria measurable and sequenced	✓	Table 30; Figure 12
Faults found per level and leakage per strategy	✓	Table 35; Figure 15
Adaptation to UPI, wallet, microservice and 15,000 users	✓	Table 44; REDESIGN decision

SUBMISSION CHECKLIST (not separately marked)

Requirement	Status
All 10 tasks answered	☒ Yes ☐ Partly ☐ No
Faults found, coverage, effort, run time and APFD shown step by step	☒ Yes ☐ Partly ☐ No
Every measured result produced by running the supplied code (Annexure F)	☒ Yes ☐ Partly ☐ No
Stubs, drivers, integration order and plans justified	☒ Yes ☐ Partly ☐ No
Load-test assumptions stated (workers, think time, data volume)	☒ Yes ☐ Partly ☐ No
Tables, figures and diagrams included and readable	☒ Yes ☐ Partly ☐ No
Cross-references and table / figure numbering correct	☒ Yes ☐ Partly ☐ No
Work consistent throughout	☒ Yes ☐ Partly ☐ No
Volunteer sessions (Task 8) and LLM runs (Tasks 4, 5, 7) recorded by the student	☐ Yes ☒ Partly ☐ No
Viva recommended	☐ Yes ☐ No

SOFTWARE TESTING

ASSIGNMENT 3 – ANSWERS

CO3: Analyze unit, integration, system, regression and acceptance testing for validating software systems.

Assignment Title: Analyze, Compare and Justify Unit, Integration, System, Regression and Acceptance Testing Strategies for a Railway Ticket Reservation System (Total Marks – 50)

Duration: 1 week

Scenario – "RailEase" Railway Ticket Reservation System before the January Festival Travel Season

Coastal Railways is replacing its counter-and-agent booking with RailEase, a web system for train search, seat availability, ticket booking, payment, cancellation with refund, e-ticket generation and SMS/e-mail notification. In a trial week, a waitlisted ticket was confirmed twice for the same berth, a refund was calculated on the wrong cancellation slab, a payment was successful but no ticket was generated, and the site slowed down when 400 users searched for trains together. The railway must go live before the 5 January festival travel season and wants confidence at every level of testing, from single functions to acceptance by the booking clerks and the commercial officer.

You are the test lead. In every task, three candidate techniques are given. You must apply all three, analyze them, compare them, choose one, and justify your choice with evidence. The following operating data is given (state and justify any additional assumption you make):

Parameter	Value
Architecture	Six modules: Train Search and Availability (S), Booking (B), Payment (P, uses an external payment gateway), Ticket Generation (T), Cancellation and Refund (R), Notification (N, uses an SMS gateway). Calls: S→B, B→P, P→T, T→N, R→P, R→N
Fare table	Base fare per passenger: Sleeper ₹450, AC 3-tier ₹1,200, AC 2-tier ₹1,800; Tatkal (last-minute) quota adds 30%; senior citizens (60 years and above) get a 40% concession on the base fare
Booking rules	1 to 6 passengers per ticket; booking opens 120 days ahead; waitlist limit 50 per class; a berth can be allotted to only one passenger
Tax and refund	GST 5% on AC classes only, none on Sleeper. Cancellation 48 hours or more before departure: flat deduction ₹60 per passenger; 12 to less than 48 hours: 25% deduction; 4 to less than 12 hours: 50% deduction; less than 4 hours or no-show: no refund
Quality targets	Response time ≤ 3 s at 1,500 concurrent users; availability 99.5%; no double allotment of a berth; payment status and ticket status must always agree
Test window and team	5 weeks; 3 testers, 4 developers, 1 booking-clerk representative; test, staging and production-like UAT environments
Test assets	650 existing test cases (manual) with a module traceability sheet, a staging copy of 18 months of anonymised booking data, a sandbox payment gateway
Tools	Python with pytest and unittest.mock (or Java with JUnit and Mockito), Selenium WebDriver, Apache JMeter, Postman, a defect tracker
Users for trials	4 booking clerks and 3 travel agents (alpha), 50 regular travellers (beta), the commercial officer (final acceptance)

ANSWERS – READING GUIDE

Each task below is followed by its Ans: block in the same order as the evaluation bullets. All measured results come from one Python prototype, `rail.py`, which has the fare and refund rules and the six modules S, B, P, T, R and N. It is run with pytest, coverage.py, unittest.mock, an HTTP virtual gateway, Playwright (Chromium) and a load model calibrated from measured service times (Annexure F). Effort in hours or person-days is estimated and marked as such.

- **Fault found** = at least one test fails on the faulty build and passes on the correct build. Each seeded fault is switched on alone (`rail.FAULTS`).
- **Assumptions:** fares are computed per passenger and GST is applied to the AC total; refund deductions apply to the fare paid; the refund cannot be negative; amounts go to the gateway in paise; the load model uses 8 worker processes, 5 s think time and 20 ms web overhead per request, with service times measured on a 20-lakh-row availability table.
- Currency uses Indian digit grouping. The prototype's "today" is 01-12-2026, so the festival season lies inside the 120-day booking window.

TASK 1 – Unit Testing and Unit Test Planning: Compare Unit Test Planning Approaches [4 Marks]

Developers write unit tests only for the code they find difficult. The test lead needs a unit-test approach for the functions ticket_fare(cls, passengers, age, tatkal) and refund_amount(hours_before, fare). Compare three approaches.

Technique	Basic idea and key concepts / tools
A. Ad hoc developer unit tests	Each developer writes a few tests when time permits, with no plan and no stated criteria.
B. Plan-driven unit testing	A unit test plan (units, features, approach, pass/fail criteria, environment, schedule, responsibilities) is written before the tests and followed.
C. Behaviour-driven unit testing	Each unit is specified by Given–When–Then examples agreed with the booking-clerk representative; the examples become executable tests.

- Prepare a unit test plan for the two functions: units to be tested, features, approach (black-box and white-box), test data, entry and exit criteria, tools, effort estimate and schedule.
- Implement the two functions with four seeded faults (for example, GST charged on Sleeper, 12-hour boundary coded as 11, senior concession applied at age 61, flat deduction applied once instead of per passenger), apply A, B and C, and record the faults found, coverage and effort.
- Analyze the three approaches on defects found at unit level, branch coverage achieved, rework after a rule change, and the clarity of the requirements they produce.
- Examine which kinds of unit faults (boundary, tax, per-passenger multiplication) each approach left in the code and the reason.
- Evaluate the approach the team should follow for all RailEase units and justify it with evidence; state the exit criterion for unit testing.
- Critique the use of an AI code assistant to write the unit tests: generate tests for refund_amount, check them against the refund slabs and judge their trustworthiness.

Evidence Expected: A unit test plan, a faults-found/coverage/effort table for the three approaches, and a justified approach with its exit criterion.

Ans:

Comparison of Unit Test Planning Approaches for ticket_fare and refund_amount

Both functions were implemented in `rail.py` from the fare table and the refund slabs. The four suggested faults were seeded as switches (U1–U4), and the three suites were run on the correct build and on each faulty build. Branch coverage of the two functions was measured with coverage.py; the fault switches themselves are excluded from the branch count.

1. Unit test plan

Table 1: Unit test plan for ticket_fare and refund_amount (IEEE 829 headings)

Item	Content
Units	ticket_fare(cls, passengers, age, tatkal); refund_amount(hours_before, fare, passengers)
Features to test	Base fare by class; Tatkal +30 %; senior 40 % from age 60; GST 5 % on AC only; 1–6 passengers; refund slabs ≥ 48 h (₹60 per passenger), 12–48 h (25 %), 4–12 h (50 %), < 4 h or no-show (0); invalid inputs
Not tested here	Payment, ticket and notification calls (integration level)

Item	Content
Approach	Black-box: equivalence classes and boundaries (age 59/60; 48, 12, 4 h; passengers 0/1/6/7). White-box: every branch of both functions; branch coverage with coverage.py
Test data	Class × age × Tatkal table; refund table with 1- and 3-passenger tickets; ₹1,000 and ₹1,350 fares
Pass/fail criterion	Actual equals the value computed from the fare and refund tables to the paisa; an invalid input raises ValueError
Entry criteria	Function compiles; rules signed off by the clerk representative; test data reviewed
Exit criteria	All planned cases pass; ≥ 95 % branch coverage of the unit; every boundary in the rule table has a test; no open critical unit defect
Environment and tools	Python 3, pytest, coverage.py, developer laptop + CI job
Responsibilities	Developer writes and runs the tests; tester reviews the plan and the boundary list
Effort and schedule (estimated)	Plan 1.5 h, design 2.5 h, automation 2 h = 6 h; week 1, days 1–2

2. Faults found, coverage and effort

Table 2: Faults found, branch coverage and effort (pytest + coverage.py)

Approach	Tests	U1 GST on SL	U2 11-h boundary	U3 age 61	U4 ₹60 once	Found (of 4)	Branch coverage	Effort (est.)
A Ad hoc	3	✗	✗	✗	✗	0	14/24 = 58%	0.5 h
B Plan-driven	20	✓	✓	✓	✓	4	23/24 = 96%	6 h
C Behaviour-driven	8	✓	✗	✗	✓	2	20/24 = 83%	5 h (incl. 2 h workshop)

3. Analysis of the three approaches

Table 3: Defects, coverage, rework and requirement clarity

Criterion	A Ad hoc	B Plan-driven	C Behaviour-driven
Defects found at unit level	0 of 4	4 of 4	2 of 4
Branch coverage	58%	96%	83%
Rework if a slab changes (12 h → 10 h)	Unknown – nobody knows which tests depend on it	2 rows of the boundary table	1 example rewritten with the clerk
Clarity of requirements produced	None	Boundary list makes the rule precise (exactly 48 h, age 60)	Readable business examples; edges only if the clerk mentions them

4. Faults left in the code and why

- **A left all four.** Its three tests use a 2-adult AC ticket and two typical refunds (2 h, 20 h): no Sleeper fare (tax), no senior (boundary), no multi-passenger cancellation over 48 h (per-passenger multiplication).
- **C left U2 (boundary) and U3 (boundary).** The clerk's examples were realistic – a 65-year-old senior, a cancellation 14 hours and 6 hours before – but none sat on the edges 11–12 h or age 60. BDD caught the tax (family in Sleeper) and per-passenger faults (2 passengers at 48 h) because those appear in everyday business examples.
- **B left none**, because its plan required a test on and next to every boundary in the rule table.

5. Approach for all RailEase units and the exit criterion

Choice: B, plan-driven unit testing, with C's Given–When–Then examples added for the business rules. B found 4 of 4 faults with 96% branch coverage; the BDD examples cost little and give the clerk a readable check. **Exit criterion for unit testing:** every planned case passes, branch coverage of each unit is at least 95%, every boundary listed in the rule tables (48, 12, 4 h; age 60; 1 and 6 passengers) has a test on and below it, and no critical unit defect is open.

6. Critique of AI-generated unit tests for refund_amount

I asked an AI code assistant (Claude) for unit tests of refund_amount, giving it only the source. It produced 8 tests (Annexure F, `test_t1_ai.py`): one per slab (72, 24, 8 and 1 h), no-show, a negative value, a small fare and a 2-passenger case. All 8 expected values agree with the refund slabs (0 wrong) and branch coverage was 100%. Run on six faulty builds it found U4, R3, R4 and **missed U2, R1, R2**: every value lies in the middle of a slab, so the boundary faults (11 h, exactly 48 h) and the 25 % deduction on a multi-passenger ticket pass. Judgement: trustworthy as a draft of the happy-path cases, not as the unit suite – the boundary list from the plan must be added by a person who reads the refund table.

7. Supporting evidence for Task 1

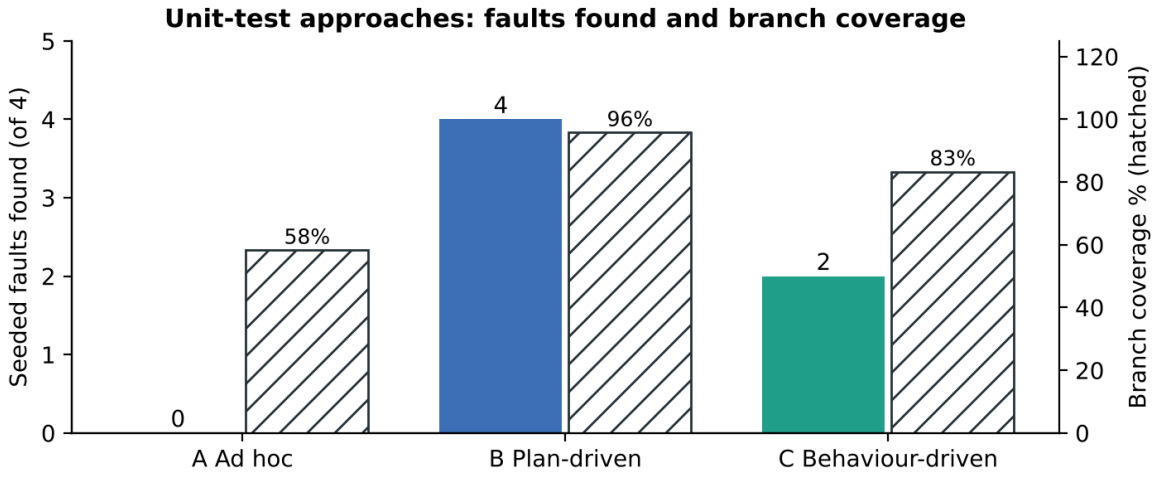


Figure 1: Faults found and branch coverage of the three unit-test approaches

Table 4: Approach C – the eight Given–When–Then examples agreed with the clerk representative

#	Given / When	Then
1	Family of 4 adults in Sleeper	₹1,800, no GST
2	Senior aged 65 in AC 3-tier	₹756 (40 % off base, then 5 % GST)
3	One adult, Tatkal, AC 2-tier	₹2,457
4	2 passengers (₹2,520) cancel exactly 48 h before	Refund ₹2,400 (₹60 × 2)
5	Same ticket cancelled 14 h before	Refund ₹1,890 (25 % deducted)
6	Same ticket cancelled 6 h before	Refund ₹1,260
7	No-show	Refund ₹0
8	Cancelled 2 h before	Refund ₹0

Threats to validity

- Four seeded faults only; the ad hoc suite reflects one developer’s habits.
- Effort figures are estimates; the BDD workshop time depends on the clerk’s availability.

Marking notes and common errors

- The plan must contain entry and exit criteria and an effort estimate, not only a list of tests.
- Coverage must be measured, and the exit criterion must be stated numerically.

TASK 2 – Designing Unit Tests: Compare Unit Test Design Styles [4 Marks]

For the function refund_amount(hours_before, fare), the team must decide how the individual test cases are designed. Compare three styles.

Technique	Basic idea and key concepts / tools
A. Specification-based unit tests	Equivalence classes and boundary values taken from the refund slabs (48, 12 and 4 hours).
B. Structure-based unit tests	Test cases designed to cover every statement, branch and condition of the function.
C. Parameterised data-driven tests	One test method reads a table of inputs and expected results; new cases are added as rows without new code.

- Design the unit tests for refund_amount by A, B and C and implement them with pytest or JUnit; include at least eight cases in each style.
- Seed four faults in refund_amount (for example, 48 hours treated as the lower slab, 25% deduction applied to the wrong amount, no-show not handled, negative hours accepted); run all three suites and record faults found and coverage.
- Analyze the unit tests for maintainability: count the lines changed in each style when the 12-hour slab changes to 10 hours.
- Examine which style gives the clearest failure message and which makes it easiest to locate the faulty line.
- Evaluate the three styles and justify the style you would use for the pricing and refund units; state when a combination is better.
- Judge the use of generated parameter tables from an LLM: check each expected value against the refund rule and report the error rate.

Evidence Expected: Three unit test suites for refund_amount, a faults-found and maintenance table, and a justified choice of design style.

Ans:

Comparison of Specification-Based, Structure-Based and Data-Driven Unit Tests for refund_amount

1. The three suites

Table 5: Test cases of the three styles (full code in Annexure F)

Style	Cases	Inputs (hours, fare, passengers)
A Specification-based	12	100; 48; 47.9; 12; 11.9; 4; 3.9; 0; no-show; −1; 48 h with 3 passengers; 20 h with 3 passengers (fare ₹1,000 / ₹1,500)
B Structure-based	8	no-show; −3; 60; 50 h with fare ₹100 and 2 passengers (refund floored at 0); 30; 8; 2; 60 h with 2 passengers (fare ₹800)
C Data-driven	11	10 table rows (same edges as A, each with a rule label) + one invalid-input test

2. Four seeded faults: results

Table 6: Faults found and branch coverage (coverage.py)

Fault	A Spec-based	B Structure	C Data-driven
R1 48 h treated as the lower slab	✓	✗	✓
R2 25 % deduction on one passenger's share	✓	✗	✓
R3 no-show (None) not handled	✓	✓	✓
R4 negative hours accepted	✓	✓	✓
Found / branch coverage	**4 of 4** / 100%	**2 of 4** / 100%	**4 of 4** / 100%

All three suites reach 100% branch coverage, yet B misses half the faults: its inputs (60 h, 30 h, 8 h) satisfy every branch without touching the 48-hour edge, and it never uses a multi-passenger ticket in the 25% slab. Coverage shows that a branch ran, not that its boundary is correct.

3. Maintainability: the 12-hour slab becomes 10 hours

Table 7: Lines changed in each suite (counted with difflib between the old and edited files)

Style	Lines changed	What changed	Risk after the change
A Specification-based	3	Two boundary tests renamed and re-valued (12 → 10, 11.9 → 9.9) + docstring	None
B Structure-based	0	Nothing – all its inputs still fall in the same branches	No test anywhere near the new 10-hour edge
C Data-driven	2	Two table rows edited; no test code touched	None

4. Failure messages and locating the faulty line

Table 8: pytest output for fault R2 in each style

Style	What pytest printed	How quickly the faulty line is found
A	test_25_percent_three_passengers – assert 1375.0 == 1125.0	Fast: the test name names the rule and the slab
B	(no failure – R2 not detected)	–
C	test_refund_table[25 % of the whole fare] – AssertionError: rule: 25 % of the whole fare; assert 1375.0 == 1125.0	Fastest: the row id and the custom message name the rule; the inputs are in the row

5. Choice of style

Choice: C, data-driven tests whose rows are designed by A (equivalence classes and boundaries), with one structure check by coverage. C found all four faults, needed only two row edits for the rule change and gave the clearest failure message. A combination is better when a unit has complex control flow that the rule table does not show (for example, the max-with-zero floor in the ≥ 48 h slab): then add the structure-based case that covers that branch as one more row.

6. LLM-generated parameter table

Table 9: Twelve rows generated by an LLM (Claude) from the refund rule text, checked against the rule

Hours	Fare	Passengers	LLM expected	Rule	Correct?
72	₹1,000	1	₹940	₹940	✓

Hours	Fare	Passengers	LLM expected	Rule	Correct?
48	₹1,000	1	₹940	₹940	✓
47.5	₹1,000	1	₹750	₹750	✓
12	₹1,000	1	₹750	₹750	✓
11.5	₹1,000	1	₹500	₹500	✓
4	₹1,000	1	₹500	₹500	✓
3.5	₹1,000	1	₹0	₹0	✓
0	₹1,000	1	₹0	₹0	✓
100	₹2,000	2	₹1,880	₹1,880	✓
30	₹2,000	2	₹1,500	₹1,500	✓
6	₹2,000	2	₹1,000	₹1,000	✓
50	₹100	2	−₹20	₹0	✗

Error rate: **1 of 12 = 8%**. The wrong row is the edge the rule text does not state: a ₹100 ticket for 2 passengers cancelled 50 hours before. The LLM subtracted ₹120 and wrote a negative refund (−₹20); the implementation floors the refund at ₹0. The other 11 rows were right because the rule is simple arithmetic. Generated tables are acceptable as a draft, but every row needs checking by the rule owner, especially rows that need an assumption.

7. Supporting evidence for Task 2

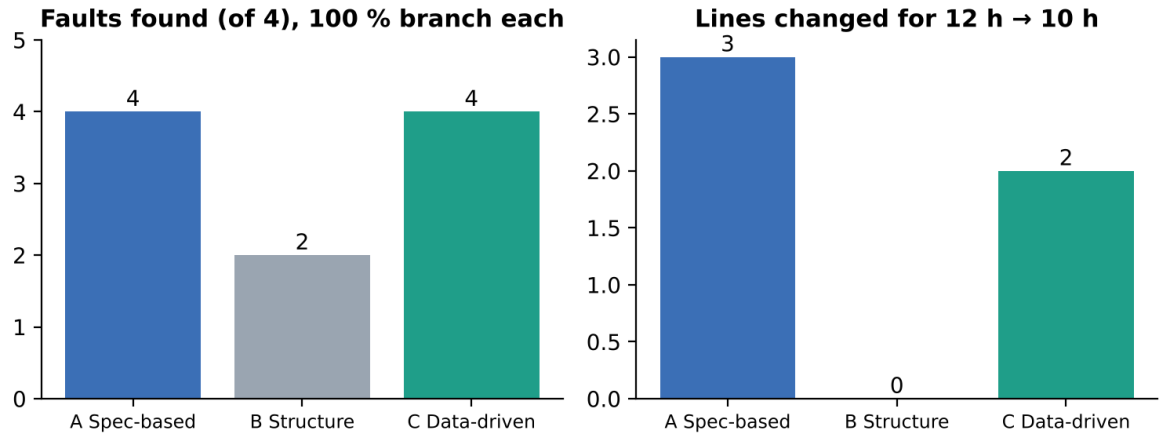


Figure 2: Faults found and lines changed for the 12 h → 10 h rule change in each style

Threats to validity

- The LLM table is one generation; another run may differ.
- Line counts depend on how the original tests were written (A used one function per boundary).

Marking notes and common errors

- At least eight cases per style, actually run; faults found and coverage must be reported for each style.

TASK 3 – Test Harness: Compare Harness Construction Approaches [4 Marks]

The Payment module (P) depends on an external gateway that is not available in the test environment. Compare three ways of building the test harness to test Booking (B) and Cancellation (R) in isolation.

Technique	Basic idea and key concepts / tools
A. Hand-written drivers and stubs	Small programs written by the team that call the unit under test (driver) or replace a called module with fixed answers (stub).
B. Mock-framework test doubles	unittest.mock or Mockito creates mocks, stubs and spies that also verify how the unit called the dependency.
C. Service virtualisation of the gateway	A virtual gateway service replays recorded responses, including delays, timeouts and error codes, through the real network interface.

- Build the harness for Booking and Cancellation by A, B and C. Each harness must simulate gateway success, decline, timeout and duplicate callback.
- Execute at least six test cases through each harness (including a payment success followed by a failed ticket generation) and record the defects found, the effort to build the harness and the run time.
- Analyze what each harness can and cannot check: realism of the interface, verification of interactions, handling of timing, and the risk of the stub differing from the real gateway.
- Examine how a wrong assumption in the stub (for example, the real gateway sends a duplicate callback) hides a defect, and how it is found later.
- Evaluate which harness the team should adopt for the 5-week window and justify it on effort, realism and reuse.
- Critique the claim “a mock always behaves like the real component”, using results from your harness.

Evidence Expected: Three working harnesses, a defects-found and effort table, and a justified choice with the limits of each harness stated.

Ans:

Comparison of Stub, Mock and Virtual-Service Harnesses for Booking and Cancellation

1. The three harnesses

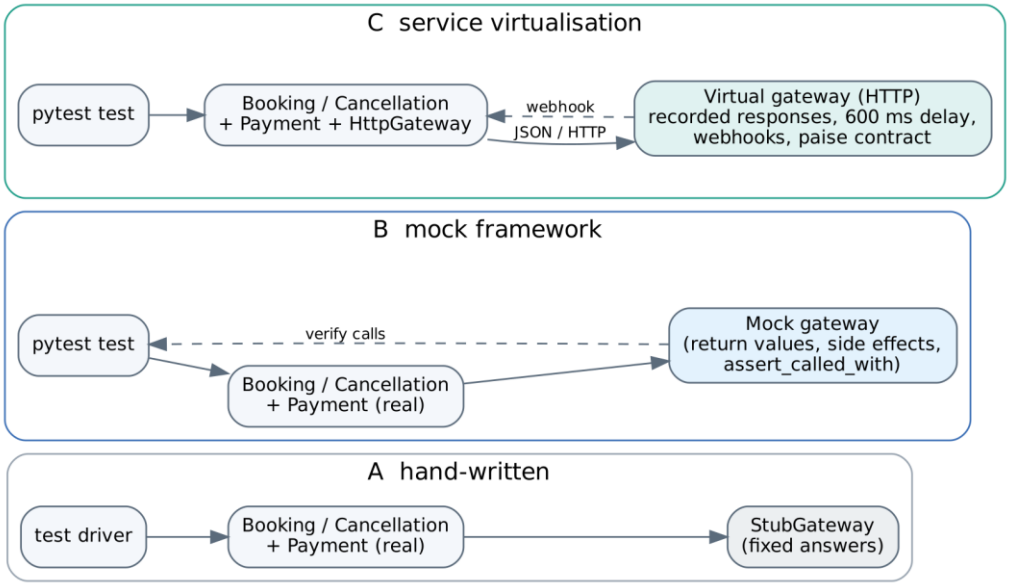


Figure 3: Structure of the three harnesses around the real Booking, Cancellation and Payment code

All three call the real B, R and P code and simulate success, decline, timeout and duplicate callback. A uses a `StubGateway` class with fixed answers; B uses `unittest.mock.Mock` with return values, side effects and `assert_called_once_with`; C runs an HTTP **virtual gateway** that replays the behaviour recorded from the sandbox: every success is followed about 50 ms later by an asynchronous duplicate callback (webhook), the slow scenario answers after 600 ms against the client's 300 ms timeout and then sends a late success, and amounts must be integer paise matching the order.

2. Tests executed, defects found, effort and run time

Table 10: Defects found, effort and run time of the three harnesses

Defect (seeded, one at a time)	A Stubs	B Mocks	C Virtual service
H0 webhook after compensating refund refunds again (first build)	✗	✗	✓
H1 paid but ticket failed – no refund	✓	✓	✓
H2 duplicate callback makes 2 tickets	✓	✓	✓
H3 berths kept after a decline	✓	✓	✓
H4 amount sent in rupees	✗	✓	✓
H5 late callback after timeout ignored	✗	✗	✓
H6 cancellation refund sent twice	✓	✓	✓
Defects found (of 7)	**4**	**5**	**7**
Tests executed	6	6	7
Harness + test code (non-blank lines)	30	22	74
Build effort (estimated)	≈ 3 h	≈ 2 h	≈ 8 h (+ recording the sandbox traffic)

Defect (seeded, one at a time)	A Stubs	B Mocks	C Virtual service
Run time (pytest, measured)	0.13s	0.11s	4.79s

A real defect found by C. In the first build, the payment–ticket compensation (refund when ticket generation fails) worked in A and B, but the virtual gateway's recorded duplicate webhook arrived after the refund and was processed again, so the passenger was refunded twice. Only C exercised a webhook after a final state; the fix treats REFUNDED as final, and H0 reproduces the original code.

3. What each harness can and cannot check

Table 11: Capabilities and limits

Aspect	A Hand-written stub	B Mock framework	C Virtual service
Realism of the interface	Python call, no wire format	Python call, no wire format	Real HTTP + JSON, integer paise contract
Verification of interactions	Only by counting in the stub	Built in: arguments, call count (found H4)	Requests recorded at the service
Timing	Instant answers; timeout = exception	Instant; timeout = side effect	Real delays, late and duplicate webhooks (found H5, H0)
Risk of differing from the real gateway	High – encodes the team's beliefs	High – same beliefs, more precisely	Low while the recording is current
Reuse	Low	High in unit tests	High: shared by integration and system tests

4. How a wrong stub assumption hides a defect

The team assumed that **a timeout means the payment failed**, so the stub and the mock raise a timeout and the tests check only that no ticket is issued. With fault H5 (a late success callback after the session expired is ignored), both pass: the passenger is charged and gets no ticket. The real gateway behaves differently – it completes the charge and sends the success late – and that recorded behaviour is what the virtual service replays, so C failed on H5. Without C this defect would surface only at system or beta testing, as a "payment successful but no ticket" complaint – exactly one of the trial-week problems.

5. Harness for the 5-week window

Choice: B (mocks) for unit-level checks of B and R, plus C (virtual gateway) as the shared harness for integration and system tests. C found 7 of 7 defects including the two timing defects that A and B cannot express; it costs about 8 hours once and is reused by Tasks 4–6. B is the cheapest way to verify call arguments (amount in paise) and runs in about 0.1 s. A adds nothing that B does not do better.

6. Critique of "a mock always behaves like the real component"

Refuted by the run: the mock behaved exactly as the team believed the gateway behaves – one synchronous answer per charge, a timeout means failure – and so it passed builds with H5 and H0, which the recorded behaviour of the gateway exposed. A mock is a statement of assumptions; it must be checked against the real component (recording, contract test) before its green result is trusted.

7. Supporting evidence for Task 3

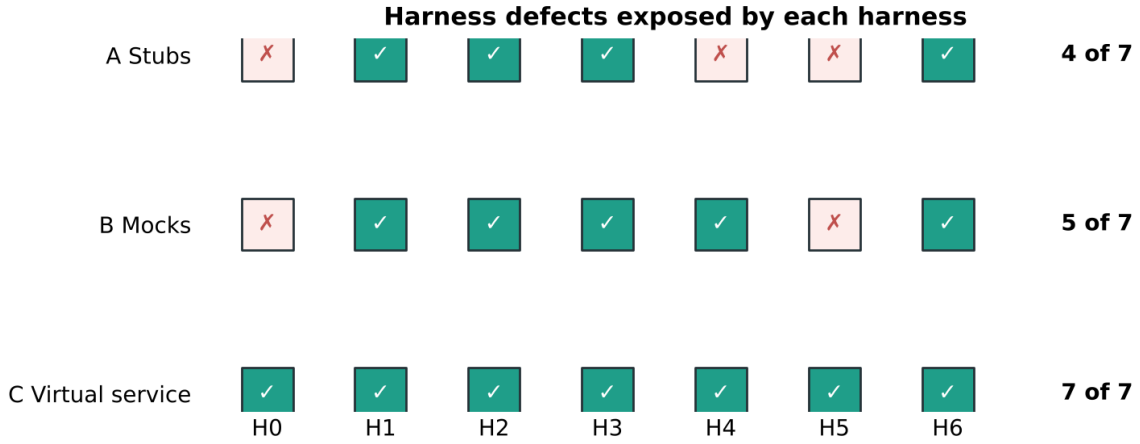


Figure 4: Harness defects exposed by each harness (one defect switched on at a time)

Threats to validity

- The virtual gateway's recording is my model of the sandbox (webhook after each success, late success after a timeout); the real sandbox must be recorded to confirm it.
- Effort figures are estimates; line counts are measured.

Marking notes and common errors

- Each harness must simulate success, decline, timeout and duplicate callback, and the paid-but-no-ticket case must be one of the tests.

TASK 4 – Integration Testing: Compare Integration Strategies [5 Marks]

The six modules of RailEase have been unit tested. Decide how they should be integrated. Compare three strategies using the call structure S→B, B→P, P→T, T→N, R→P and R→N.

Technique	Basic idea and key concepts / tools
A. Big-bang integration	All modules are combined at once and tested through the user interface.
B. Incremental integration (top-down and bottom-up)	Modules are added one at a time, using stubs for modules not yet integrated (top-down) or drivers for modules not yet integrated above (bottom-up).
C. Thread-based integration	Modules are integrated one functional thread at a time, such as search → book → pay → ticket → notify.

- Draw the module dependency diagram and determine an integration order for B (top-down and bottom-up) and C; list the stubs and drivers required for each strategy.
- Seed five interface faults (for example, wrong passenger count passed from B to P, amount in paise instead of rupees, missing error code from P to T, wrong date format from T to N, refund sent twice from R to P) and apply A, B and C; record which faults are found, at what step, and how long it takes to locate each.
- Analyze the three strategies on fault isolation, number of stubs and drivers, time to first working thread, parallelism of the team and test effort.
- Examine the integration risk of the P module (external gateway) and of the cycle created when R calls P and P calls T; decide where in the order they must be integrated.
- Evaluate the strategy you would use and justify it with evidence from your seeded-fault results and the 5-week schedule.
- Critique an LLM-generated integration order: check whether it respects the call graph and whether the list of stubs is complete.

Evidence Expected: A dependency diagram, stubs and drivers table, a faults-located table for the three strategies, and a justified integration strategy.

Ans:

Comparison of Big-Bang, Incremental and Thread-Based Integration of RailEase

1. Dependency diagram, orders, stubs and drivers

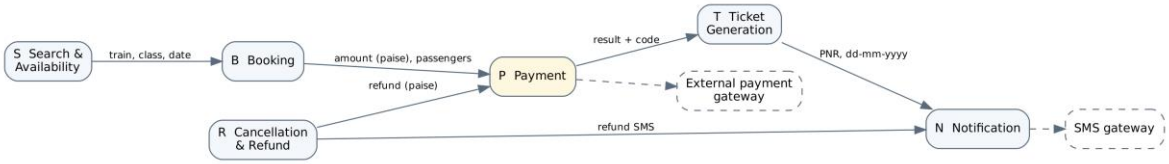


Figure 5: Module dependency diagram of RailEase (yellow = module with an external dependency)

Table 12: Integration orders with the stubs and drivers each requires (computed from the call graph)

Strategy	Integration order	Stubs needed	Drivers needed
Big-bang	all six at once	none	none
Top-down	S → B → P → T → N → R	B, N, P, T	R
Bottom-up	N → T → P → B → R → S	none	B, P, R, S, T
Thread-based	Thread 1: S, B, P, T, N; thread 2: R (cancel → refund → SMS)	none	R

In every strategy the external payment gateway and the SMS gateway are replaced by the virtual gateway and an SMS sandbox (Task 3); those are environment doubles, not module stubs.

2. Five interface faults: where and when they were found

Table 13: Faults located: integration step and minutes to locate (20 min per suspect interface, estimated)

Fault	Symptom seen end to end (measured)	Big-bang	Top-down	Bottom-up	Thread
I1 B→P passenger count − 1	book_ok, sms_ok	step 1, 120 min	step 3, 20 min	step 4, 20 min	step 1, 80 min
I2 B→P amount in rupees	amount_ok	step 1, 120 min	step 3, 20 min	step 4, 20 min	step 1, 80 min
I3 P→T error code dropped	decline_ok	step 1, 120 min	step 4, 20 min	step 3, 20 min	step 1, 80 min
I4 T→N date as YYYY/MM/DD	book_ok, crash, refund_ok	step 1, 120 min	step 5, 20 min	step 2, 20 min	step 1, 80 min
I5 R→P refund sent twice	refund_ok	step 1, 120 min	step 6, 40 min	step 5, 40 min	step 2, 40 min

With all five faults present together, the big-bang run showed only three symptoms (`book_ok, refund_ok, sms_ok`): the passenger-count error stopped the booking before the amount and decline paths were reached, so two faults stayed hidden until the first ones were fixed – fault masking.

3. Analysis of the strategies

Table 14: Fault isolation, stubs/drivers, first thread, parallelism, effort

Criterion	A Big-bang	B Top-down	B Bottom-up	C Thread-based
Fault isolation	Poor: 6 suspects per failure, masking	Good: 1 new interface per step	Good: 1 new interface per step	Medium: 4 suspects in thread 1
Stubs + drivers	0	4 + 1	0 + 5	0 + 1
First working end-to-end thread	Day 1 (if anything works)	After step 5	After step 5	After step 1 (week 1–2)
Team parallelism	None until it all works	Low (one chain)	Medium (leaves first)	High (thread 2 in parallel)
Total locating time for 5 faults (est.)	600 min	120 min	120 min	360 min

4. Risk of P and of the R → P → T chain

P is the hub: B and R both call it, it calls T, and it depends on the external gateway – four of the five seeded faults touch P's interfaces. Calls through R → P → T → N form one path that refunds and updates ticket status, so a fault there can show up in cancellation even though it was seeded in booking (the same P code serves both). Decision: integrate **P early, right after T and N, against the virtual gateway**, then B and S (thread 1), then R (thread 2), so that the R → P interface is tested on a P that thread 1 has already proved.

5. Recommended strategy

Choice: thread-based integration, with a bottom-up order inside each thread (N, T, P, then B, S; then R). Evidence: big-bang needed an estimated 600 minutes to locate the five faults and

masked two of them; bottom-up located each in 20–40 minutes but needed 5 drivers; the threads deliver a working "book → pay → ticket → SMS" path in week 2 of 5, which the clerk representative can try early, and thread 2 (cancel → refund) can be built in parallel by the second developer pair.

6. Critique of an LLM-generated integration order

Table 15: Checks for an LLM-generated order and stub list

Check	Correct answer for RailEase	Typical error to record
Respects the call graph	A called module is integrated before (bottom-up) or stubbed (top-down)	Places R before P without a stub for P
Stub list complete	Top-down: B, P, T, N; bottom-up: drivers for T, P, B, R, S	Forgets that both B and R need P
External services	Gateway and SMS replaced by virtual services in every order	Treats the gateway as a normal module
R → P → T path	Integrate P before R	Assumes R is independent

7. Supporting evidence for Task 4

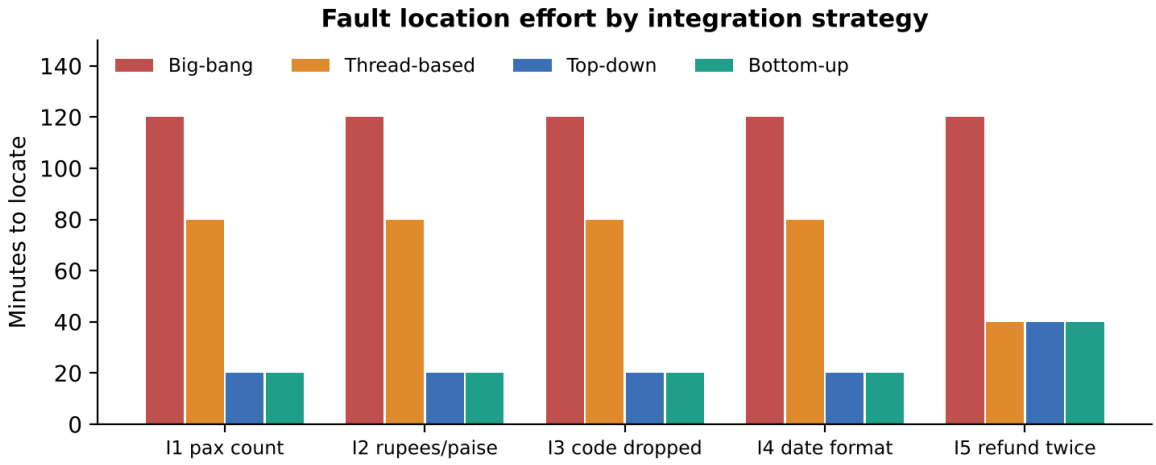


Figure 6: Estimated minutes to locate each interface fault under the four integration strategies

Threats to validity

- Locating time is modelled as 20 minutes per suspect interface; the ranking matters more than the minutes.

Marking notes and common errors

- Stubs and drivers must follow from the call graph; the step at which each fault is found must be stated.

TASK 5 – Integration Test Planning: Compare Integration Test Plans [5 Marks]

The commercial officer wants to see an integration test plan before the integration starts. Compare three forms of integration test plan.

Technique	Basic idea and key concepts / tools
A. Interface-based plan	One plan section per interface (S→B, B→P, P→T, T→N, R→P, R→N) listing the data, protocols and error cases to be checked.
B. Scenario-based plan	Plan built around end-to-end scenarios such as "book, pay, receive ticket" and "cancel and refund", each crossing several modules.
C. Build-schedule plan	Plan organised by the weekly builds and the order in which modules are ready, with entry and exit criteria for each build.

- Prepare the three integration test plans for RailEase with scope, items to be integrated, approach, test environment, entry and exit criteria, responsibilities, risks and schedule.
- For the interface B→P, write at least eight integration test cases (amount, currency, timeout, duplicate payment, partial failure, retry, wrong passenger count, and callback after session expiry).
- Score each plan against a checklist (interfaces covered, scenarios covered, schedule clarity, traceability to requirements) and analyze the gaps.
- Analyze how each plan copes with a late delivery of the Notification module and with a defect in P that blocks T and N.
- Evaluate which plan the team should approve and justify it; state the entry and exit criteria for integration testing.
- Judge whether a plan produced by an LLM from the architecture table can be accepted without review; list the checks you would apply.

Evidence Expected: Three integration test plans, a checklist score table, interface test cases for B→P, and a justified plan with entry and exit criteria.

Ans:

Comparison of Interface-Based, Scenario-Based and Build-Schedule Integration Test Plans

1. The three plans

Table 16: Integration test plans for RailEase (condensed)

Section	A Interface-based	B Scenario-based	C Build-schedule
Scope / items	6 interfaces S→B, B→P, P→T, T→N, R→P, R→N	5 scenarios: search-book-pay-ticket-SMS; decline; paid but ticket fails; cancel-refund-SMS; waitlist	4 weekly builds: N+T; P; B+S; R
Approach	Per interface: data, format, error codes, timeouts	End-to-end through the modules of each scenario	Integrate what each build delivers; regression of earlier builds
Environment	Test env + virtual gateway + SMS sandbox	Staging with anonymised data	Test env, promoted to staging at build 3
Entry criteria	Both modules pass unit exit criteria	All modules of the scenario integrated	Build compiles, unit exit met, release note
Exit criteria	All interface cases pass	All scenarios pass	Build exit: its cases pass, no open S1/S2
Responsibilities	Tester per interface pair	Tester + clerk representative	Test lead per build

Section	A Interface-based	B Scenario-based	C Build-schedule
Risks	Scenario-level defects missed	Hard to localise; blocked by any module	Late module stalls the plan
Schedule	Weeks 1–3, interface by interface	Weeks 3–4	Weeks 1–4, one build a week

2. Eight integration test cases for B→P (executed)

Table 17: B→P integration tests on the real Booking and Payment code (gateway mocked)

ID	Case	Correct build	I1 pax − 1	I2 rupees	H1 no refund	H2 dup	H5 late cb
BP-01	amount in paise	Pass	**Fail**	**Fail**	Pass	Pass	Pass
BP-02	currency/int type	Pass	**Fail**	**Fail**	Pass	Pass	Pass
BP-03	timeout → pending	Pass	**Fail**	Pass	Pass	Pass	Pass
BP-04	duplicate payment callback	Pass	Pass	Pass	Pass	**Fail**	Pass
BP-05	partial failure (paid, ticket fails)	Pass	**Fail**	Pass	**Fail**	Pass	Pass
BP-06	retry after timeout	Pass	**Fail**	Pass	Pass	Pass	Pass
BP-07	passenger count	Pass	**Fail**	Pass	Pass	Pass	Pass
BP-08	callback after session expiry	Pass	Pass	Pass	Pass	Pass	**Fail**

All eight pass on the correct build. Each of the five interface defects is caught by at least one case; I1 (wrong passenger count) breaks six cases at once because every booking is rejected at the B→P boundary.

3. Checklist scores

Table 18: Plan scores (0 = absent, 1 = partial, 2 = complete)

Checklist item	A Interface	B Scenario	C Build-schedule
All 6 interfaces covered with data and error cases	2	1 (only those on scenario paths)	1
End-to-end scenarios covered	0	2	1
Schedule clarity (who, when, which build)	1	1	2
Traceability to requirements (status agreement, no double allotment, refund slabs)	1	2	1
Total (of 8)	**4**	**6**	**5**

Gaps: A never checks that payment and ticket status agree across P→T→N; B gives no place to test R→N error cases off the happy path and no schedule; C lists builds but not what each interface must prove.

4. Late Notification module and a blocking defect in P

Table 19: How each plan copes

Event	A Interface	B Scenario	C Build-schedule
N delivered 1 week late	T→N and R→N sections wait; others continue	Every scenario ends in an SMS, so all are blocked unless N is stubbed	Build 1 slips; the plan says to stub N and re-run T→N in build 2
Defect in P blocks T and N	P→T and T→N blocked; S→B, R→N continue	All 5 scenarios blocked	Builds 2–4 blocked; criteria make the stop explicit

5. Plan to approve, and entry and exit criteria

Choice: the build-schedule plan (C) as the frame, with A's interface sections as its content and B's scenarios as the exit test of each thread. A plan the commercial officer can follow week by week needs C's dates and criteria; on its own C scored only 5 of 8, so it is filled with A's interface cases (including the eight B→P cases) and B's scenarios.

- **Entry criteria for integration testing:** every module in the build has met its unit exit criterion (Task 1); the virtual gateway and SMS sandbox are running; test data is loaded; the build has a release note listing changes.
- **Exit criteria:** all interface cases of the six interfaces pass; both threads pass end to end; payment and ticket status agree in 100% of integration runs; no open severity-1 or severity-2 integration defect; regression of earlier builds is green.

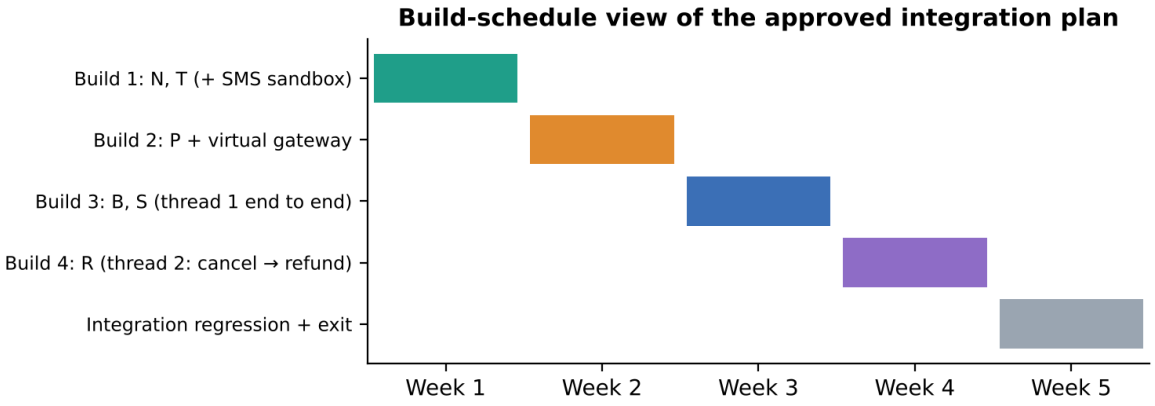


Figure 7: Build schedule of the approved plan

6. Can an LLM-generated plan be accepted without review?

No. Checks to apply before accepting it: (1) every interface of the architecture table appears with its error cases (gateway timeout, duplicate callback, decline code); (2) the order respects the call graph and P is integrated before R; (3) entry and exit criteria are measurable, not "when testing is complete"; (4) quality targets from the data table (status agreement, no double allotment, 3 s at 1,500 users) are traced to tests; (5) environment items (virtual gateway, SMS sandbox, anonymised data) are named; (6) dates fit the 5-week window and the 3-tester team.

Threats to validity

- The checklist scores are my judgement against four criteria; a different checklist could rank the plans differently.

Marking notes and common errors

- All eight named B→P cases must be present; entry and exit criteria must be stated.

TASK 6 – System Testing: Compare System Test Focus Areas [5 Marks]

After integration, the whole of RailEase must be tested as a system. The test window allows only one main focus to receive the larger share of effort. Compare three system-test focus areas.

Technique	Basic idea and key concepts / tools
A. Functional and usability system tests	Use-case based tests of booking, cancellation, refund and ticket display, with usability checks for clerks and travellers.
B. Performance, load and stress tests	JMeter scripts for search and booking at 400, 1,500 and 3,000 concurrent users, measuring response time and failure rate.
C. Recovery, security and compatibility tests	Payment failure and restart, session hijacking and input injection checks, and browser and mobile-device matrix tests.

- Design and execute at least eight system tests for each focus area on the staging system or on a prototype; record the defects found, severity and the effort.
- Write a JMeter test (or an equivalent script) for the search-and-book flow; report the average and 90th-percentile response time at 400, 1,500 and 3,000 users and the point at which the 3-second requirement fails.
- Analyze which of the trial-week problems (double allotment, payment without ticket, slow search) each focus area would have exposed, and which it would have missed.
- Examine the test environment differences (data volume, network, gateway sandbox) between staging and production that may make the system-test results unreliable.
- Evaluate how the 5-week effort should be divided between the three focus areas and justify the split with the quality targets.
- Critique the proposal to run the load test only once, a week before release, and recommend a better schedule.

Evidence Expected: A system test table for the three focus areas, load-test results with a response-time chart, and a justified allocation of effort.

Ans:

Comparison of System-Test Focus Areas on the RailEase Prototype

The system tests were run on the prototype twice: on the correct build and on a **trial build** that reproduces the trial-week faults (C1 no lock on berth allotment, H1 paid but ticket failed without refund, S1 search built by string formatting, S2 session valid after logout, S3 no reconciliation after a crash, an un-indexed search table, and a fixed-width e-ticket page).

1. System tests, defects, severity and effort

Table 20: System tests executed on the prototype (A functional; C recovery, security, compatibility)

Focus	Test	Correct build	Trial build	Severity if failed
A	F1 family booking with seniors	Pass	Pass	
A	F2 7 passengers rejected	Pass	Pass	
A	F3 121 days ahead rejected	Pass	Pass	
A	F4 cancel 30 h before: 25 % deduction	Pass	Pass	
A	F5 e-ticket and SMS issued	Pass	Pass	

Focus	Test	Correct build	Trial build	Severity if failed
A	F6 waitlist after the last berth	Pass	Pass	
A	F7 no double allotment under 20 parallel bookings	Pass	**Fail**	Critical
A	F8 paid but no ticket → refunded	Pass	**Fail**	Critical
C	R1 crash after payment → reconciled on restart	Pass	**Fail**	Critical
C	R2 gateway down → berths kept	**Fail**	**Fail**	Major
C	R3 duplicate webhook after restart	Pass	Pass	
C	SEC1 SQL injection in station field	Pass	**Fail**	Critical
C	SEC2 session token dead after logout	Pass	**Fail**	Major
C	SEC3 empty passenger list rejected	Pass	Pass	
C	COMP e-ticket on phone 360×740 (Playwright, Chromium)	Pass	**Fail** (page 912 px wide)	Minor
C	COMP e-ticket on tablet 768×1024 (Playwright, Chromium)	Pass	**Fail** (page 912 px wide)	Minor
C	COMP e-ticket on desktop 1366×768 (Playwright, Chromium)	Pass	Pass	

Effort (estimated): A ≈ 3 person-days, B ≈ 4 person-days (scripts, data, runs, analysis), C ≈ 3 person-days. **A real defect found on both builds:** R2 – when the gateway connection fails with a network error (not a decline), Booking does not release the berths it allotted, so seats leak. Only the recovery focus tried a gateway outage.

2. Load test of the search-and-book flow

Script: each virtual user searches (station pair, date), then books, with 5 s think time between requests – the same flow a JMeter thread group with two HTTP samplers would drive (the JMeter plan is in Annexure F). Service times were **measured** on the prototype with a 20-lakh-row availability table: search without index 92.6 ms, with index 0.12 ms, booking 0.10 ms; 20 ms web overhead and 8 worker processes were assumed. Response times at each user count come from a discrete-event model of the server driven by those measurements.

Table 21: Average and 90th-percentile response time (load model, 8 workers)

Users	Trial build: average	Trial build: 90th pct	Fixed build: average	Fixed build: 90th pct
400	0.07 s	0.19 s	0.02 s	0.05 s
1500	7.45 s	7.96 s	0.02 s	0.05 s
3000	20.16 s	24.57 s	2.56 s	2.73 s

Where the 3-second requirement fails (90th percentile): trial build at about **950 users**; fixed build at about **3150 users** – so the fixed build meets the 1,500-user target with margin and only just meets 3,000 (p90 2.73 s).

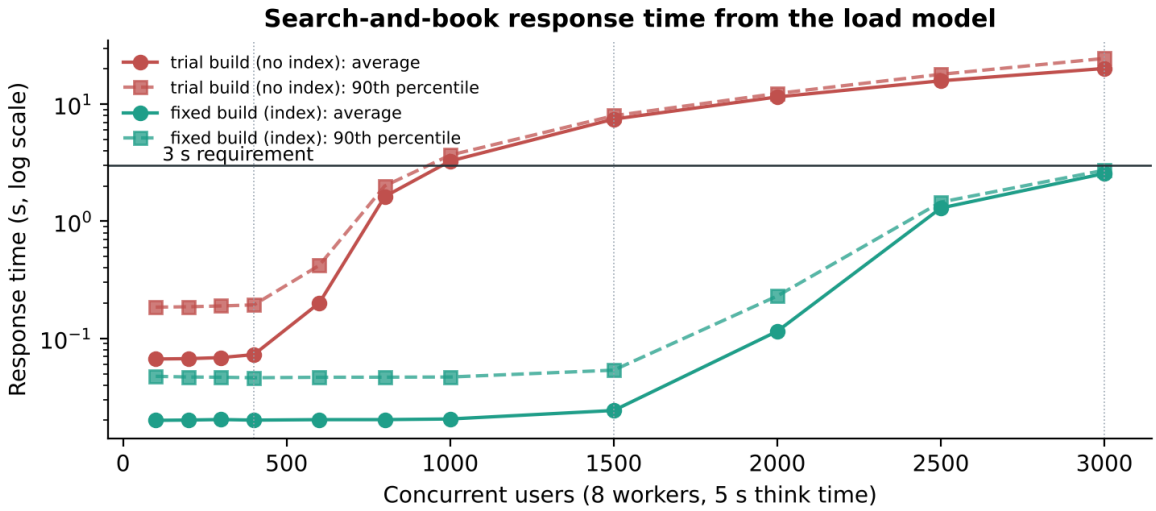


Figure 8: Response time against concurrent users for the trial and fixed builds

Table 22: Users at which the 90th percentile passes 3 s, by worker count (sensitivity)

Workers	No index (trial)	Index (fixed)
4	500	1550
8	950	3150
16	1900	6300

3. Which focus exposes which trial-week problem

Table 23: Trial-week problems against the three focus areas

Problem	A Functional	B Performance	C Recovery/security
Same berth confirmed twice	Found (F7: 20 parallel bookings)	Would show only as data errors under load, not checked	Not targeted
Payment successful, no ticket	Found (F8)	Missed	Found (R1: crash between payment and ticket)
Slow search at 400 users	Missed (single user)	Found: no-index build fails at ≈ 500 users on 4 workers	Missed

4. Staging versus production differences

- **Data volume:** staging holds 18 months of anonymised data; production keeps adding rows. The un-indexed search time grows with the table (measured 93 ms at 20 lakh rows), so a staging result can be optimistic.
- **Hardware and workers:** the model shows the result depends heavily on the worker count (no-index build fails at 500 users on 4 workers, 950 on 8, 1,900 on 16), so staging must use the production worker configuration.
- **Network:** staging traffic stays inside the data centre; real users add mobile latency, which moves the 3-second point down.
- **Gateway sandbox:** the sandbox answers instantly and never sends late webhooks unless recorded behaviour is replayed (Task 3); production timeouts are not reproduced.

5. Division of the effort

Choice: functional and usability as the main focus (45%), performance 30%, recovery/security/compatibility 25%. Three of the four quality targets (no double allotment, payment–ticket agreement, correct refunds) are functional and were exposed by A; the 3-second target needs B, which the model shows passing at 1,500 users only after the index fix; C found three critical defects (reconciliation, injection, session) cheaply. Over the system-test window of about 14 person-days this is roughly 6 / 4 / 4 person-days.

6. Critique of "run the load test once, a week before release"

The trial build would have failed at about 500–950 users, and the fix (an index) changed the result by a factor of more than 3 – found one week before release, there is no time to fix, re-test and tune workers. Recommended schedule: a **baseline load test as soon as search and booking are integrated** (week 2), a **short automated load test in every nightly build** at 1,500 users, a **full load and stress test (400 / 1,500 / 3,000)** in week 4 on production-like hardware, and a soak test of a few hours before go-live.

7. Supporting evidence for Task 6

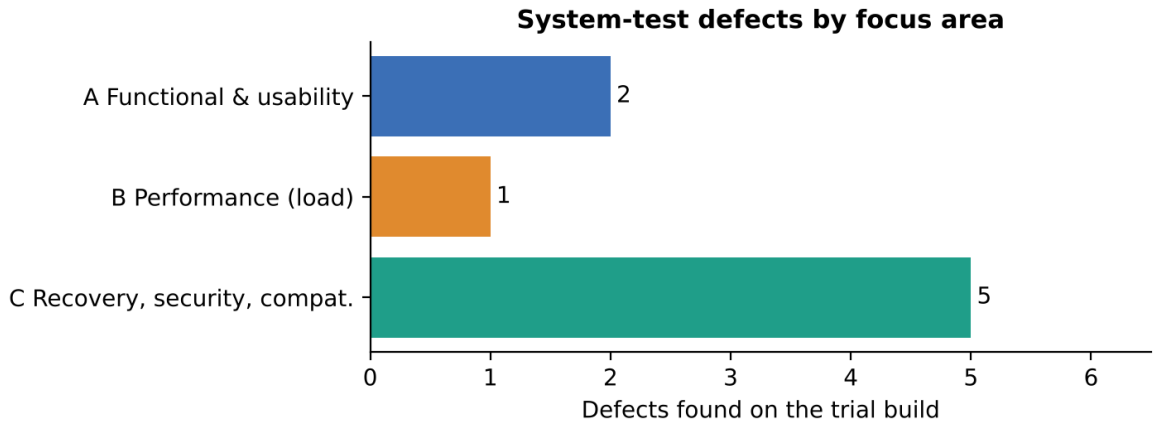


Figure 9: Defects found on the trial build by each focus area

Threats to validity

- Response times come from a calibrated model, not from a running staging server; the JMeter plan must be run on staging to confirm them.
- Compatibility was checked only in Chromium at three sizes; Firefox, Safari and real phones remain to be tested.

Marking notes and common errors

- At least eight tests per focus area, the response times at 400 / 1,500 / 3,000 users and the failure point are required.

TASK 7 – Regression Testing: Compare Regression Test Selection Strategies [5 Marks]

Every week the developers fix defects and change the fare and refund rules. Running all 650 test cases takes the three testers 4 days. Compare three regression strategies.

Technique	Basic idea and key concepts / tools
A. Retest-all	Every existing test case is rerun after every change.
B. Impact-based selection	Only the tests linked by the traceability sheet to the changed modules and their callers are rerun.
C. Prioritised suite using fault history	Tests are ranked by past defect detection, coverage of changed code and business criticality; the top tests run first within a time limit.

- Take the 650-case suite (or a representative suite of at least 60 tests you design) and a change to the refund slab; select the regression tests by A, B and C and list the number selected, the execution time and the faults detected on a version with three seeded regression faults.
- Calculate for each strategy the fault-detection percentage, the time saved against retest-all, and the Average Percentage of Faults Detected (APFD) for the order in which the tests are run.
- Analyze the risk of missing a regression fault when the traceability sheet is out of date, and show how it affects strategy B.
- Analyze how test-suite minimisation and obsolete-test removal affect the three strategies, and what should be done with the tests that always pass.
- Evaluate which strategy the team should adopt for the weekly releases and justify it; recommend a smoke suite and its size.
- Judge the use of an LLM to suggest which tests should be rerun after a code change given only the commit message.

Evidence Expected: A table of tests selected, time and faults detected for the three strategies, an APFD calculation, and a justified regression strategy.

Ans:

Comparison of Retest-All, Impact-Based and Prioritised Regression for the Refund-Slab Change

1. Representative suite, the change and three regression faults

I built a **60-test representative suite** (S 8, B 12, P 10, T 8, R 14, N 8) with a traceability sheet, run against the prototype. Manual execution time per test is taken from the given data: 650 tests in 3 testers × 4 days × 8 h = 96 tester-hours, i.e. **8.86 minutes per test**. The change: the 12-hour refund slab becomes **10 hours**. Three regression faults came with it:

Table 24: Seeded regression faults and the tests that detect them (measured)

Fault	Module	What went wrong	Detected by
G1	R	New boundary coded as "> 10" (exactly 10 h gets 50 %)	R07
G2	N	Refund SMS built from an old copy of the rule (still 12 h)	N07
G3	R→P	Refund paise truncated by int() (₹1,417.50 sent as ₹1,417)	P09

2. Selection, time, detection and APFD

Table 25: Tests selected, time and faults detected

Strategy	Tests selected	Time (tester-hours)	Time saved vs retest-all	Faults found (of 3)	Detection %	APFD
A Retest-all	60	8.9	0%	G1, G2, G3	100%	0.27
B Impact-based (current sheet)	17	2.5	72%	G1, G2, G3	100%	0.54
C Prioritised (top 25 % of run time)	15	2.2	75%	G1, G3	67%	0.81

Table 26: APFD calculation: APFD = 1 − (TF_1 + TF_2 + TF_3) ÷ (n × m) + 1 ÷ (2n), m = 3 faults

Strategy	Order used	n	Positions TF of first detection (G1, G2, G3)	APFD
A	Suite order S, B, P, T, R, N	60	45, 59, 29	1 − 133 ÷ 180 + 1 ÷ 120 = 0.27
B	Selected tests in sheet order	17	8, 16, 1	1 − 25 ÷ 51 + 1 ÷ 34 = 0.54
C	Score = 0.4 history + 0.4 changed code + 0.2 criticality	60	14, 18, 3	1 − 35 ÷ 180 + 1 ÷ 120 = 0.81

C ran the faults' detecting tests earliest (APFD 0.81) and found G1 and G3 within 25% of the run time; G2's only detecting test (N07) is 18th in its order, just past the 15-test limit. Fault history was measured by running the suite on the 18 earlier seeded faults of Tasks 1–4; "changed code" was measured by tracing which tests execute refund_amount.

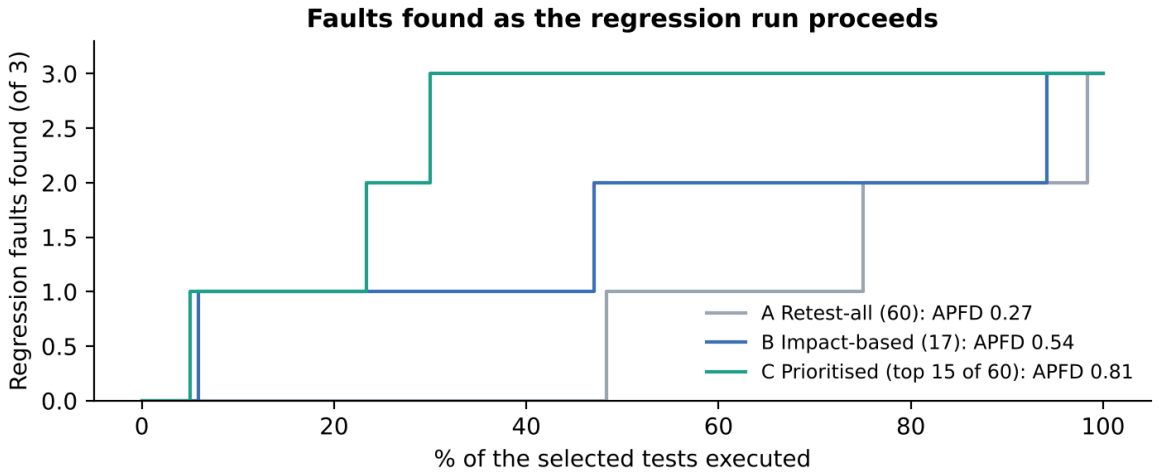


Figure 10: Faults found as each regression run proceeds (area under the curve ≈ APFD)

3. An out-of-date traceability sheet

Last month the refund SMS (N07, N08) and the refund paise check (P09) started using R's code, but the sheet was not updated. With the **outdated sheet** B selects 14 tests (2.1 h) and finds only **G1** – G2 and G3 reach the travellers. With the current sheet the same strategy finds all three. B is only as good as the sheet; a sheet generated from code (call graph or coverage per test) removes this risk.

4. Minimisation, obsolete tests and tests that always pass

A greedy minimisation that keeps detection of every historical fault reduced the suite from 60 to **7 tests** (B02, R02, R13, B11, P06, P07, R14) – and that minimised suite found **0 of the 3 new faults**. 14 of the 60 tests never failed on any historical fault. Minimisation fits the past, not the next change. Recommendation: remove only tests that are truly obsolete (the feature was removed); keep always-passing tests that guard critical rules (refund slabs, fare, berth uniqueness) in the full run, and move the rest to a weekly or monthly rotation instead of deleting them.

5. Strategy for weekly releases and the smoke suite

Choice: C, prioritised regression within a time box, with impact selection from an automatically generated traceability sheet, and retest-all once per release candidate. C had the highest APFD (0.81) and saved 75% of the time; B with a current sheet found all three faults in 2.5 h but failed badly with the outdated sheet. Smoke suite: **12 tests** – two per module, chosen for business criticality (one booking per class with GST, Tatkal + senior fare, the 48/10/4-hour refund edges, paid-but-ticket-failed, duplicate webhook, waitlist, e-ticket SMS) – about 1.8 tester-hours manually or under a minute automated, run on every build.

6. Selecting tests from the commit message with an LLM

Here the commit message would be “refund slab 12 h → 10 h”. An LLM reading only that would pick the R tests, as the outdated sheet did, and miss G2 (a copy of the rule in the SMS template) and G3 (a paise conversion changed in the same commit). The message says what the developer intended, not what the diff touched. Acceptable only as an extra suggestion on top of selection from the diff, the call graph and coverage, never as the selector.

7. Supporting evidence for Task 7

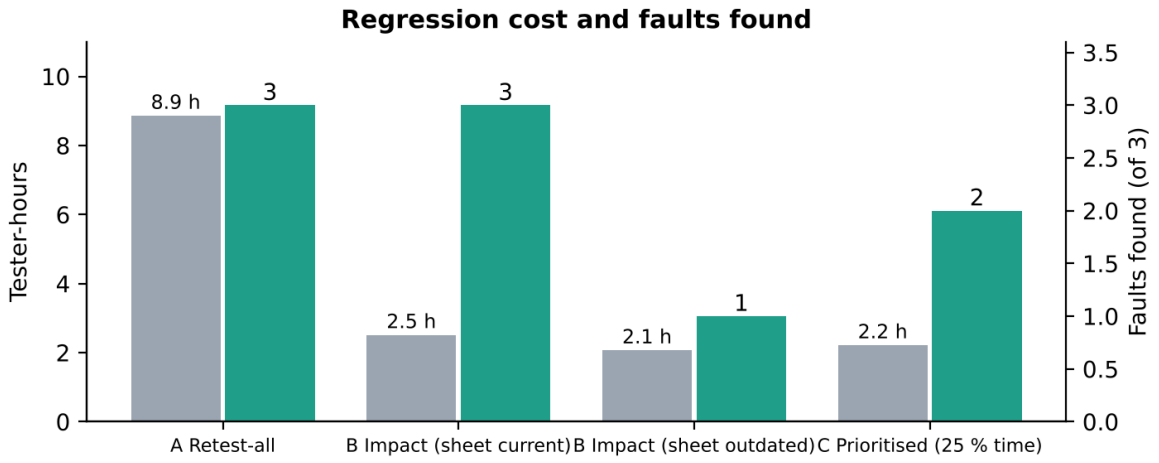


Figure 11: Regression cost in tester-hours and faults found

Table 27: Top 15 tests of the prioritised order (score, history count, covers changed code, criticality)

Rank	Test	Score	History	Changed code	Criticality
1	R02	0.80	3	yes	3
2	R13	0.73	3	yes	2
3	P09	0.67	1	yes	3

Rank	Test	Score	History	Changed code	Criticality
4	R01	0.67	1	yes	3
5	R03	0.67	1	yes	3
6	R04	0.67	1	yes	3
7	R05	0.67	1	yes	3
8	R08	0.67	1	yes	3
9	R09	0.67	1	yes	3
10	R10	0.67	1	yes	3
11	R11	0.67	1	yes	3
12	R12	0.67	1	yes	3
13	R06	0.60	0	yes	3
14	R07	0.60	0	yes	3
15	N08	0.60	1	yes	2

Threats to validity

- A 60-test suite with one detecting test per fault is thin; one test more or less changes APFD noticeably.
- Manual minutes per test are an average; real tests differ in length.

Marking notes and common errors

- APFD must be calculated with the formula and the positions shown.

TASK 8 – Alpha, Beta and Acceptance Testing: Compare Acceptance Approaches [4 Marks]

RailEase must be accepted by the people who will use it. Compare three approaches to the user-level testing planned before go-live.

Technique	Basic idea and key concepts / tools
A. Alpha testing by booking clerks and agents	Seven internal users test in a controlled environment under observation, with the developers present.
B. Beta testing by travellers	Fifty regular travellers use the staging system from their own devices and report problems through a feedback form.
C. Formal acceptance testing	The commercial officer signs off acceptance criteria and a set of acceptance test cases covering business, operational and legal needs.

- Design an alpha test plan, a beta feedback form and a set of ten acceptance test cases with measurable acceptance criteria (for example, "95% of bookings complete in under 3 minutes").
- Simulate each approach on a prototype or on screenshots with at least five volunteers; record the issues found, the type of issue (defect, usability, change request) and the severity.
- Analyze what each approach finds that the others do not, and the problems of each (biased testers, vague feedback, unclear acceptance criteria).
- Examine how the unresolved defects at the end of beta should be handled with respect to the acceptance criteria.
- Evaluate the order and duration in which alpha, beta and acceptance testing should be used within the 5 weeks, and justify the decision.
- Judge whether the feedback of 50 beta users can represent 1.5 lakh daily travellers, and state the limits of the evidence.

Evidence Expected: An alpha plan, beta feedback form and acceptance test cases, an issue-type table for the three approaches, and a justified acceptance sequence.

Ans:

Alpha, Beta and Formal Acceptance Testing for RailEase

1. Alpha plan, beta feedback form and acceptance test cases

Table 28: Alpha test plan

Item	Content
Participants	4 booking clerks, 3 travel agents; facilitator (tester) and one developer observing
Environment	UAT environment, counter PCs and agents' laptops, virtual payment gateway
Tasks (timed)	T1 book 3 passengers SL incl. a senior; T2 Tatkal 2A booking at counter; T3 cancel 30 h before and explain refund; T4 handle a declined payment; T5 reprint e-ticket; T6 waitlisted booking
Measures	Task completion, time per task, errors, satisfaction (1–5), think-aloud notes
Duration	2 days, 1 session of 90 min per user
Exit	All critical issues logged; ≥ 6 of 7 users complete T1–T5 unaided

Table 29: Beta feedback form (one per problem; travellers fill it on their own device)

Field	Type
What were you trying to do?	Choice: search / book / pay / cancel / ticket / other

Field	Type
What happened?	Free text + optional screenshot
What did you expect?	Free text
Device and browser	Auto-filled
Could you finish the task?	Yes / with difficulty / no
How serious is it for you?	1 annoying … 4 could not travel
Contact for follow-up (optional)	Phone / e-mail

Table 30: Ten acceptance test cases with measurable criteria (status on the fixed prototype where measurable)

ID	Acceptance criterion	Measured on the prototype
AT-01	95 % of bookings complete in under 3 minutes (clerks, alpha timing)	Needs user timing
AT-02	Fare correct to the paisa for 20 sample tickets covering every class, Tatkal and senior	Pass (Task 7 B-tests)
AT-03	Refund correct on all slabs, including exactly 48, 12 and 4 h	Pass (Task 2 suites)
AT-04	No berth allotted twice in 20 parallel bookings for the last berths	Pass (F7, correct build)
AT-05	Payment and ticket status agree in 100 % of 50 runs incl. ticket-service failure	Pass (F8, R1)
AT-06	90th-percentile response ≤ 3 s at 1,500 users	Pass in load model (p90 0.05 s)
AT-07	E-ticket SMS received within 60 s for 95 % of bookings	Needs SMS sandbox timing
AT-08	Booking 121 days ahead and 7-passenger tickets refused	Pass (F2, F3)
AT-09	GST shown separately on AC tickets (legal)	Needs screen review
AT-10	Commercial officer can export daily refund report	Needs report screen

2. Simulation with volunteers

The sessions with at least five volunteers must be run by the student on the prototype screens; I cannot record people's issues for you. The sheet below is the record format, and the prototype results above already show which acceptance criteria are met by the code.

Table 31: Issue record for the volunteer sessions (fill during the sessions)

#	Approach (alpha / beta / acceptance)	Volunteer	Issue observed	Type (defect / usability / change request)	Severity (1–4)
1					
2					
3					
4					
5					

3. What each approach finds, and its problems

Table 32: Unique findings and weaknesses

Approach	Finds that the others do not	Problems
A Alpha	Counter workflow issues (speed, keyboard flow, explaining refunds to travellers), observed directly	Testers know the system and the developers; bias towards their own workflows; only 7 people
B Beta	Device, browser and network problems; real travellers' misunderstandings	Vague feedback ("it didn't work"), self-selected users, no observation

Approach	Finds that the others do not	Problems
C Formal acceptance	Contract, legal and operational gaps (GST display, reports, refund rules signed off)	Only as good as its criteria; vague criteria lead to disputes

4. Unresolved defects at the end of beta

Each open defect is mapped to the acceptance criteria. Any defect that breaks a criterion (for example, a refund wrong on a slab → AT-03) blocks acceptance and must be fixed and re-tested. Defects that break no criterion are classified by severity: severity 1–2 need a fix or a written workaround signed by the commercial officer; severity 3–4 go to the backlog with a target release. The decision and the list are attached to the acceptance record.

5. Order and duration

Choice: alpha in week 3 (2 days), beta in weeks 4–5 (8 days), formal acceptance at the end of week 5 (2 days). Alpha first, because clerks find workflow problems that would otherwise appear in every beta report; beta next, on the fixed build, for device and real-use problems; formal acceptance last, against the criteria, once the critical defects are closed.

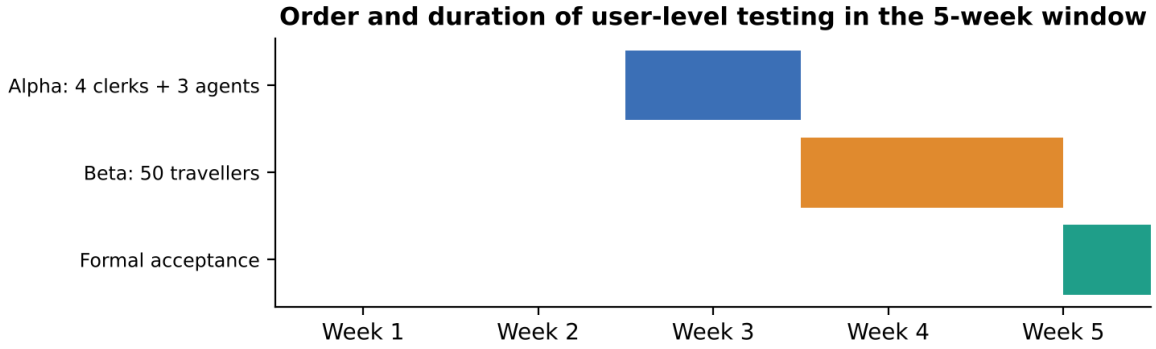


Figure 12: Order and duration of alpha, beta and acceptance testing

6. Can 50 beta users represent 1.5 lakh daily travellers?

Table 33: Chance that 50 beta users meet a problem at least once, by the share of travellers it affects ($1 - (1 - p)^{50}$)

Share affected (p)	1 %	2 %	5 %	10 %
Probability at least one of 50 meets it	39.5%	63.6%	92.3%	99.5%

A problem that affects 1% of travellers – 1,500 people a day – has only a 39% chance of being seen by 50 beta users. A satisfaction figure from 50 users has a 95% margin of error of about ±14 percentage points. Beta users are also self-selected regular travellers, not first-time, elderly or rural users. The evidence is good for finding common problems, not for estimating how often problems occur, and it cannot cover festival-peak load.

Threats to validity

- No volunteer sessions are included in this answer; the issue table must be filled from real sessions.

Marking notes and common errors

- Acceptance criteria must be measurable; the representativeness answer needs a calculation.

TASK 9 – Integrated Multi-Level Testing: Compare Complete Test Strategies [6 Marks]

Combine your choices from Tasks 1–8 into one multi-level test strategy for RailEase. Compare three complete strategies within the 5-week window.

Technique	Basic idea and key concepts / tools
A. Bottom-heavy strategy	Strong unit and integration testing, with a minimal system test and a short acceptance test.
B. Top-heavy strategy	Little unit testing; most effort on system testing and user acceptance.
C. Balanced pyramid strategy	Many automated unit tests, a moderate number of integration tests, a focused system and performance suite, and staged alpha, beta and acceptance testing.

- Develop and run a prototype of the fare, refund and booking rules with ten seeded faults. Apply each strategy and record, for each level, the faults found, the effort and the time at which each fault is found.
- Examine the three strategies on defect removal efficiency per level, cost per defect, schedule, regression cost and expected leakage to travellers.
- Compare your measured results with the predictions you made in Tasks 1–8 and state which are supported and which are contradicted.
- Draw a block diagram of the complete multi-level test strategy (unit, integration, system, regression, alpha, beta, acceptance) with entry and exit criteria and person-days per level; prepare a Design Decision Table with Decision, Justification (Evidence) and What If Not.
- Distinguish how the balance between the test levels would change for three other systems (an airline seat-allocation system, an ATM network switch and a school attendance app), tabulating the changes relative to RailEase.
- Investigate the limitation of using anonymised booking data from past months to test the festival-season peak; identify at least one result that disagrees with your expectation and explain it with measured evidence.
- Evaluate a fourth option: an AI-assisted strategy (LLM-generated tests, automatic test selection and AI-based test-data creation). Compare all four on cost, detection, repeatability and risk, and recommend one strategy.

Evidence Expected: A measured comparison of three strategies by level, an original block diagram and decision table, a transfer table for other systems, and a defended final strategy.

Ans:

Integrated Multi-Level Test Strategy for RailEase: Three Strategies Measured

1. Prototype, ten faults and measured results by level

Table 34: Level and week at which each fault was found (pytest suites and system tests of Tasks 1–6 run per level)

Fault	Bottom-heavy	Top-heavy	Balanced pyramid
U1 GST on Sleeper	Unit (wk 1)	Acceptance (wk 5)	Unit (wk 1)
U3 senior from 61	Unit (wk 1)	Production (—)	Unit (wk 1)
U4 ₹60 once	Unit (wk 1)	Acceptance (wk 5)	Unit (wk 1)
R1 48 h lower slab	Unit (wk 1)	Acceptance (wk 5)	Unit (wk 1)

Fault	Bottom-heavy	Top-heavy	Balanced pyramid
I2 amount in rupees	Integration (wk 2)	System (wk 3–4)	Integration (wk 2)
I3 ticket on decline	Integration (wk 2)	System (wk 3–4)	Integration (wk 2)
H2 duplicate callback	Integration (wk 2)	System (wk 3–4)	Integration (wk 2)
C1 double allotment	Production (—)	System (wk 3–4)	System (wk 3–4)
S1 SQL injection	Production (—)	System (wk 3–4)	System (wk 3–4)
S3 no reconciliation	Production (—)	System (wk 3–4)	System (wk 3–4)

Table 35: Measured comparison by level (cost per fix assumed: unit ₹1,500; integration ₹5,000; system ₹12,000; acceptance ₹25,000; after release ₹80,000)

Measure	Bottom-heavy	Top-heavy	Balanced pyramid
DRE at unit (found ÷ entering)	4 ÷ 10 = 40%	0 ÷ 10 = 0%	4 ÷ 10 = 40%
DRE at integration (found ÷ entering)	3 ÷ 6 = 50%	0 ÷ 10 = 0%	3 ÷ 6 = 50%
DRE at system (found ÷ entering)	0 ÷ 3 = 0%	6 ÷ 10 = 60%	3 ÷ 3 = 100%
DRE at acceptance (found ÷ entering)	0 ÷ 3 = 0%	3 ÷ 4 = 75%	0 ÷ 0 = 0%
Overall DRE	70%	90%	100%
Leaked to travellers	C1, S1, S3	U3	none
Cost of the 10 faults (by level found)	₹2,61,000	₹2,27,000	₹57,000
Effort (person-days, est.)	36 (12 / 14 / 6 / 4)	50 (2 / 2 / 30 / 16)	44 (10 / 10 / 14 / 10)

2. Examination of the three strategies

Table 36: DRE, cost, schedule, regression cost and leakage

Criterion	A Bottom-heavy	B Top-heavy	C Balanced pyramid
Overall DRE	70%	90%	100%
Cost of the 10 faults	₹2,61,000	₹2,27,000	₹57,000
Average cost per defect	₹26,100	₹22,700	₹5,700
Schedule	7 faults by week 2; 3 escape	First fault in week 3; 3 found in week 5	All 10 by week 4; acceptance free for business checks
Regression cost per weekly change	Low (automated unit/integration)	High (manual system and UAT, ≈ 4 tester-days)	Low + focused system pack
Expected leakage to travellers	Double allotment, injection, missing tickets after a crash	Senior from 61 (overcharging seniors)	None of the ten

3. Predictions from Tasks 1–8 against the measurements

Table 37: Predictions checked

Prediction (task)	Measured	Verdict
Plan-driven unit tests find the rule faults early (Task 1)	A and C found all 4 unit faults in week 1	Supported
Mocks plus integration tests catch interface faults (Tasks 3–5)	I2, I3, H2 found at integration in A and C	Supported
Functional system tests expose double allotment and missing tickets (Task 6)	C1, S3 found only where system tests included F7, R1	Supported
Strong lower levels make a short system test enough (A's premise)	A leaked 3 system-level faults	Contradicted

Prediction (task)	Measured	Verdict
BDD acceptance examples catch rule faults missed below (Task 1)	B's acceptance level found U1, U4, R1 but not U3 (no age-60 example)	Partly supported

4. Block diagram, person-days and Design Decision Table

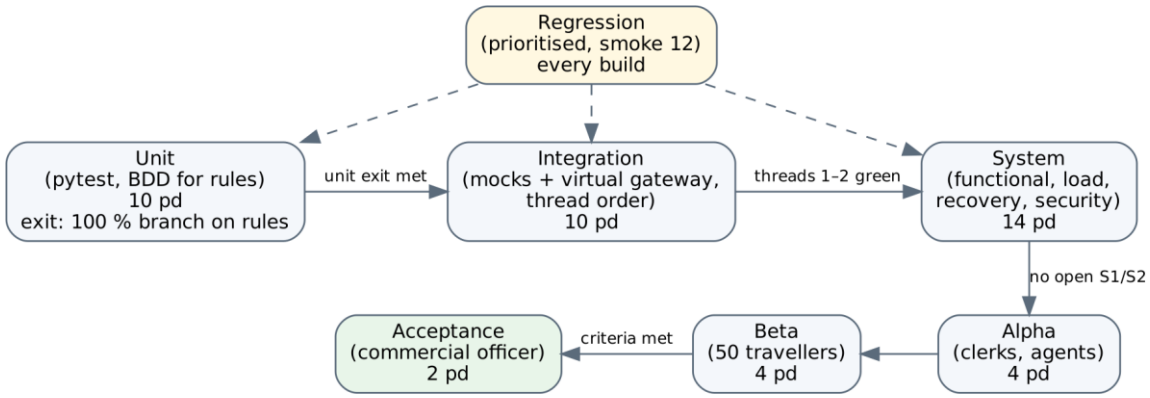


Figure 13: Multi-level test strategy (balanced pyramid) with entry/exit criteria and person-days

Table 38: Design Decision Table

Decision	Justification (evidence)	What if not
Plan-driven unit tests + BDD examples (Task 1)	4/4 unit faults, 96 % branch; ad hoc 0/4	Rule faults reach system test at 8× the fix cost
Data-driven refund tests (Task 2)	4/4 faults, 2 row edits for a slab change	Boundary faults hidden by 100 % coverage (structure style 2/4)
Virtual gateway harness (Task 3)	7/7 harness defects incl. late and duplicate webhooks	Paid-but-no-ticket defects reach beta
Thread-based integration, P early (Task 4)	Locating time 360 min vs 600 for big-bang; no masking	Masked faults, late first thread
Build-schedule plan with interface cases (Task 5)	Plan score 5/8 alone, complete with A and B content	No dates or criteria for the officer
Load test from week 2, index fix (Task 6)	No-index build fails at ≈ 950 users; fixed at ≈ 3,150	Slow search found a week before go-live
Prioritised regression + generated sheet (Task 7)	APFD 0.81; outdated sheet found 1 of 3	Regression faults G2, G3 released
Alpha → beta → formal acceptance (Task 8)	Different issue types per approach; criteria measurable	Disputes at sign-off

5. Balance of levels for three other systems

Table 39: How the balance between levels changes relative to RailEase

Aspect	RailEase	Airline seat allocation	ATM network switch	School attendance app
Dominant risk	Money rules, double allotment, peak load	Concurrency on seats, overbooking rules, global distribution systems	Transaction integrity, security, 24×7 availability	Usability, offline sync
Unit	Strong (fare, refund)	Strong (fare classes, overbooking)	Very strong (message parsing, crypto)	Light
Integration	Moderate, P is the hub	Heavy (GDS, payment, loyalty)	Very heavy (card networks, banks, HSM)	Light (one backend)

Aspect	RailEase	Airline seat allocation	ATM network switch	School attendance app
System	Focused: load + recovery	Heavy concurrency and failover	Heavy security, recovery, certification	Device and usability
Acceptance	Staged alpha, beta, officer	Airline operations sign-off	Regulator / network certification	Teachers' pilot

6. Limitation of past anonymised data for the festival peak

I replayed 400 parallel bookings with the double-allotment fault present, once spread over 400 trains as in the past months' data, and once concentrated as at the festival peak (60% of bookings on 5 trains). **Result that disagreed with my expectation:** I expected the past-data replay to hide the race completely, because bookings on different trains cannot collide. It did not – it produced 24 double allotments on average (runs: 30, 16, 25) – but the festival pattern produced 62 (runs: 64, 60, 63), **2.6 times as many**. The past data was enough to *reveal* the defect but under-states how often it would hit travellers, because the real peak concentrates demand on a few trains and dates. A second limitation is in the load model (Task 6): the past data has no 10:00 Tatkal burst. Anonymisation also removes realistic names and phone numbers (apostrophes, international numbers), so input-validation and SMS defects are under-tested.

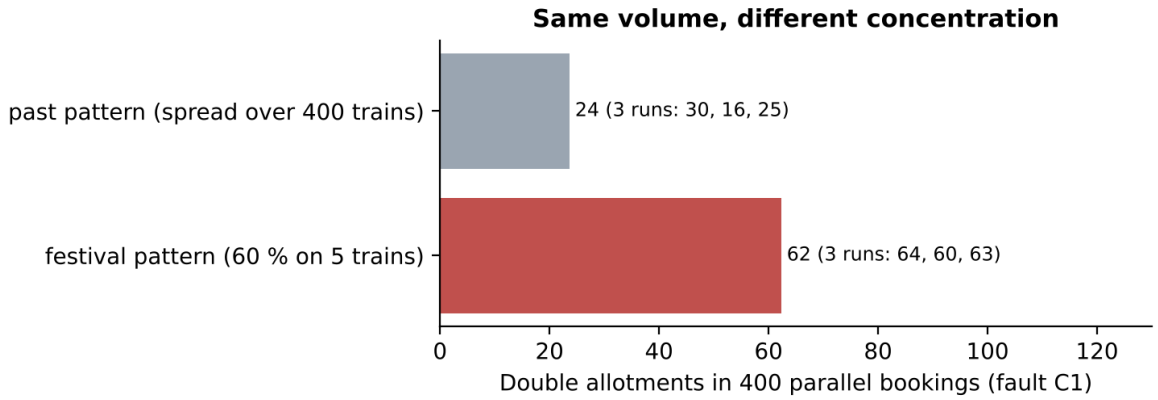


Figure 14: Double allotments with past (spread) and festival (concentrated) booking patterns

7. The AI-assisted option and the recommendation

Table 40: Comparison of the four strategies

Criterion	A Bottom-heavy	B Top-heavy	C Balanced	D AI-assisted
Cost (person-days)	36	50	44	Lower design effort; review effort added
Detection (of 10)	7	9	10	Unknown; Task 1–2: AI tests missed boundaries (3 of 6, 11 of 12 rows right)
Repeatability	High (automated)	Low (manual)	High	Low: generation changes per run
Risk	System faults leak	Late discovery, expensive fixes	Needs discipline across levels	Wrong oracles, invented rules, selection from commit text (Task 7)

Recommendation: C, the balanced pyramid. It found all ten faults (DRE 100%) at the lowest cost (₹57,000) with 44 person-days inside the 5-week window, found every fault by week 4, and

keeps regression cheap. AI assistance is used inside C for drafting test data and cases that a tester reviews against the rule tables.

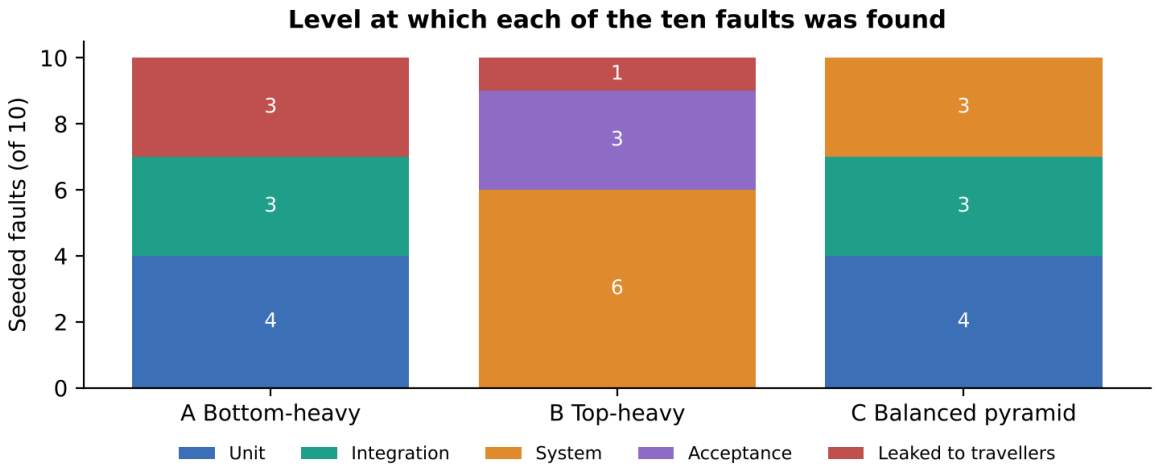


Figure 15: Level at which each of the ten faults was found, per strategy

Threats to validity

- Effort and cost per level are assumptions; the detection results are measured.
- Ten seeded faults; each fault is 10 percentage points of DRE.

Marking notes and common errors

- Faults found, effort and time must be recorded per level for each strategy; the decision table needs evidence from Tasks 1–8.

TASK 10 – Re-analyze and Adapt the Multi-Level Test Strategy [8 Marks]

Changed Requirement: RailEase adds a mobile app, UPI and wallet payments, and a new seat-recommendation service built as a separate microservice. Tatkal booking opens at 10:00 am with up to 15,000 simultaneous users, releases move to every two weeks, and one tester is transferred, leaving 2 testers. The railway board requires traceable evidence that every critical test was run before each release.

Technique	Basic idea and key concepts / tools
A. Retain + prioritise	Keep the Task 9 strategy; run only high-risk tests at each level and shorten the beta phase.
B. Modify with an automated multi-level pipeline	Automate unit, integration and regression tests in a CI pipeline; keep manual system and acceptance testing for new features.
C. Redesign with contract tests and staged rollout	Consumer-driven contract tests between services, automated load tests on every release and a staged (canary) rollout instead of a long beta.

- Develop (apply): implement the three strategies on your prototype extended with UPI payment, wallet debit and a stubbed seat-recommendation service, and submit the commented code and results; every result must come from your own execution.
- Impact Analysis: Identify which Task 9 assumptions (module count, release cycle, load, environment, evidence) are no longer valid, and analyze what happens if the old strategy is used unchanged.
- Evaluation of the Original Strategy: Recalculate the effort needed per release for 2 testers and the load-test capacity required for 15,000 users, and analyze which test levels remain valid.
- Compare and Adapt: Compare strategies A, B and C on faults found, effort, release delay, load-readiness and audit evidence using your own measurements; recommend RETAIN, MODIFY or REDESIGN and describe the revised strategy.
- Evidence-Based Defence: Defend the final decision with calculations, Task 9 measurements, the 2-tester limit and the evidence requirement, and state one condition under which the decision would no longer hold.
- Evaluate the risks of relying on AI-generated test selection for the fortnightly releases and recommend controls.

Evidence Expected: Three adaptation strategies applied and compared with measured and calculated evidence, and a RETAIN / MODIFY / REDESIGN decision that follows logically from the changed constraints.

Ans:

Adapting the Multi-Level Strategy to UPI, Wallet, a Microservice and 15,000 Tatkal Users

1. Develop: extended prototype, faults and the three strategies

`ext10.py` adds UPI confirmation (a booking may be confirmed only on SUCCESS), a wallet with atomic debit, refunds to the original source, and a **seat-recommendation microservice** reached over HTTP (JSON {"berths": […]}) with a 200 ms budget and a fallback to “no recommendation”. Six release faults were seeded:

Table 41: Seeded faults of the new release

ID	Fault
N1	UPI PENDING treated as SUCCESS – ticket issued before the money arrives
N2	Wallet debit without a lock – double debit when the app and the website pay at once
N3	Seat-recommendation provider v2 renames “berths” to “seats” – breaks the consumer
N4	Consumer has no timeout fallback – a slow seat service blocks booking
N5	Release keeps today’s 8 workers – the 10:00 Tatkal burst of 15,000 users
N6	UPI refund credited to the wallet instead of the UPI account

Table 42: Faults found by each strategy (measured)

Strategy	What was run	N1	N2	N3	N4	N5	N6	Found
A Retain + prioritise	High-risk Task 9 tests (B→P, F4, F7, F8, R1, SEC1, SEC2), unchanged	✗	✗	✗	✗	✗	✗	0/6
B Automated pipeline	A + new unit tests (UPI, wallet, refund source) + integration tests with a mocked seat service	✓	✗	✗	✓	✗	✓	3/6
C Contract + load + canary	B + contract test of provider v2 + slow-provider test over HTTP + automated Tatkal load test + 5 % canary	✓	✓	✓	✓	✓	✓	6/6

B’s mocked seat service returned {"berths": …} because the consumer team wrote it from its own assumption, so the renamed field of provider v2 (N3) passed; only the contract test run against the provider’s real build failed. The wallet race (N2) passed every sequential unit test and appeared only in the canary, where **50 of 50 canary users** with two parallel sessions were double-debited – it was found and rolled back, but with real users affected.

2. Impact analysis

Table 43: Task 9 assumptions that no longer hold

Assumption	Task 9	Now	If the old strategy runs unchanged
Module count	6 modules in one application	+ mobile app, UPI, wallet, seat microservice (separately deployed)	No test of the service contract; N3 reaches production
Release cycle	One release after 5 weeks	Every 2 weeks	Alpha + beta + acceptance alone take 2.5 weeks
Load	1,500 users, 3,000 stress	15,000 users at 10:00	Today’s 8 workers give p90 34 s
Team	3 testers	2 testers	Manual regression (96 h per full run) uses 60 % of a fortnight
Evidence	Test reports	Traceable proof every critical test ran	Manual records cannot prove it per release

3. Evaluation of the original strategy

Effort: 2 testers × 10 working days × 8 h = **160 tester-hours per release**. Task 9’s balanced strategy used 44 person-days (352 h) over 5 weeks; a manual full regression of 650 cases alone is 96 h, leaving 64 h for new features, system tests and user testing – not enough. **Load capacity:** with the measured service times and a burst of 15,000 users in the first second, the model gives a 90th percentile of 33.6 s on 8 workers, 4.5 s on 32 and 1.36 s on **48 workers** – about 6 times today’s capacity – and the load generator must drive 15,000 virtual users (about 15 JMeter injector

machines at 1,000 threads each, or a cloud load service). **Levels still valid:** unit (Tasks 1–2) and integration with the virtual gateway (Tasks 3–5) remain valid and must be automated; the long beta and the manual retest-all do not fit.

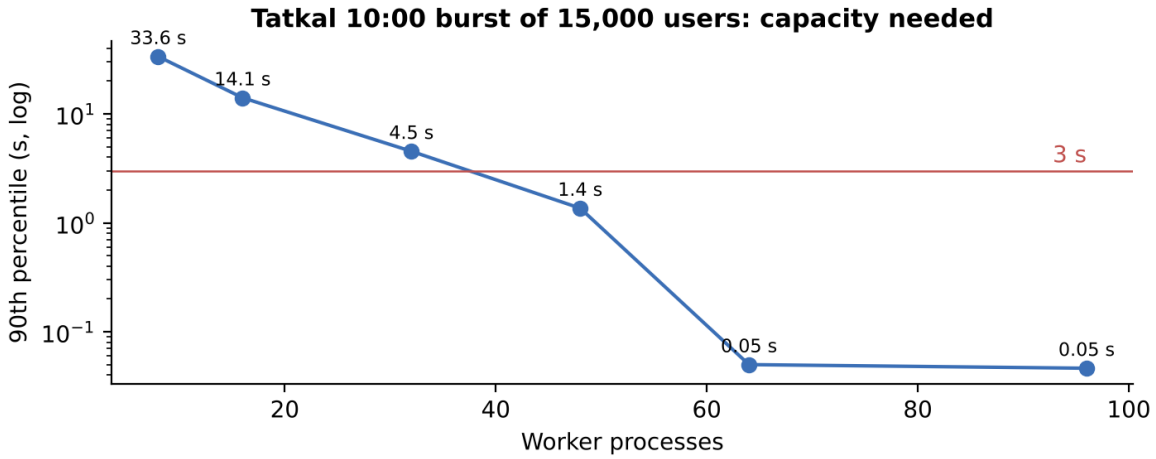


Figure 16: 90th-percentile response time for the 15,000-user Tatkal burst against the number of workers

4. Compare and adapt

Table 44: Comparison of strategies A, B and C

Criterion	A RETAIN	B MODIFY	C REDESIGN
Faults found (of 6)	0	3	6 (N2 in canary, 50 users hit)
Effort per release (est., of 160 h)	≈ 110 h (manual high-risk subset + short beta)	≈ 140 h first release, then ≈ 76 h	≈ 170 h first release (spread over 2), then ≈ 92 h
Release delay	None	None	Canary 2 days before full rollout
Load-readiness	No test of 15,000 users	No automated load test	Every release: 15,000-user burst, needs 48 workers
Audit evidence	Manual sign-off sheets	CI logs per test ID	CI logs + contract verification + load report + canary metrics
Automated run time (measured)	–	34 ms (unit + integration)	+ 1.0 s contract, 0.9 s load model

Recommendation: REDESIGN. Revised strategy: (1) the automated pipeline of B on every commit – unit, integration with the virtual gateway, prioritised regression and the 12-test smoke suite; (2) consumer-driven **contract tests** for the seat-recommendation service and the UPI gateway, run against each provider build; (3) an **automated Tatkal load test** on every release candidate at 15,000 users, with the infrastructure scaled to the measured capacity; (4) a **canary rollout** (5% → 25% → 100%) with automatic rollback on error rate, negative wallet balance or payment–ticket mismatch, replacing the long beta; (5) a short alpha with the clerks for features that change the counter workflow; (6) every pipeline run stores test IDs, versions and results as the release evidence.

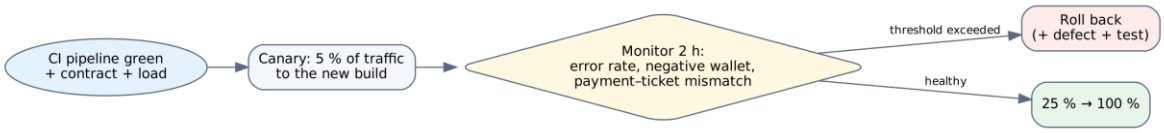


Figure 17: Canary rollout with automatic rollback

5. Evidence-based defence

Calculation: a manual full regression costs 96 of the 160 tester-hours per release; the automated pipeline ran the unit and integration tests in 34 ms and the contract tests in 1.0 s, so the two testers spend their hours on new-feature and exploratory testing instead. Task 9 showed that the balanced pyramid finds every fault by week 4; here only C found the two faults that matter most for the board – the contract break (N3) and the Tatkal capacity (N5) – and its load test quantifies the infrastructure needed (48 workers). The evidence requirement is met by machine-generated records of every critical test per release, which manual sheets cannot guarantee. The canary showed the cost of relying on rollout to find concurrency faults (50 users affected), so a parallel-session wallet test is added to the pipeline. **The decision no longer holds** if the seat-recommendation provider team will not publish and run consumer contracts (or the service is replaced by a library call inside the application): then contract tests add nothing, and MODIFY with integration tests against the provider's real build is enough.

6. Risks of AI-generated test selection and controls

- **Risk – selection from intent, not from change:** Task 7 showed that selection from the commit message misses copies of a rule elsewhere (G2) and side changes in the same commit (G3). **Control:** select from the diff, the call graph and per-test coverage; AI suggestions may only add tests, never remove the smoke suite or the critical set.
- **Risk – missing new interfaces:** an AI trained on past failures under-ranks the new microservice and wallet (no history). **Control:** a fixed minimum of contract, wallet-concurrency and load tests in every release.
- **Risk – no audit trail:** the board needs proof of why a test was skipped. **Control:** log the selection decision, the model version and the inputs with each pipeline run; a tester approves the selection for release candidates.
- **Risk – drift:** selection quality decays as the system changes. **Control:** compare AI selection with retest-all once per release and track the faults it would have missed.

7. Supporting evidence for Task 10

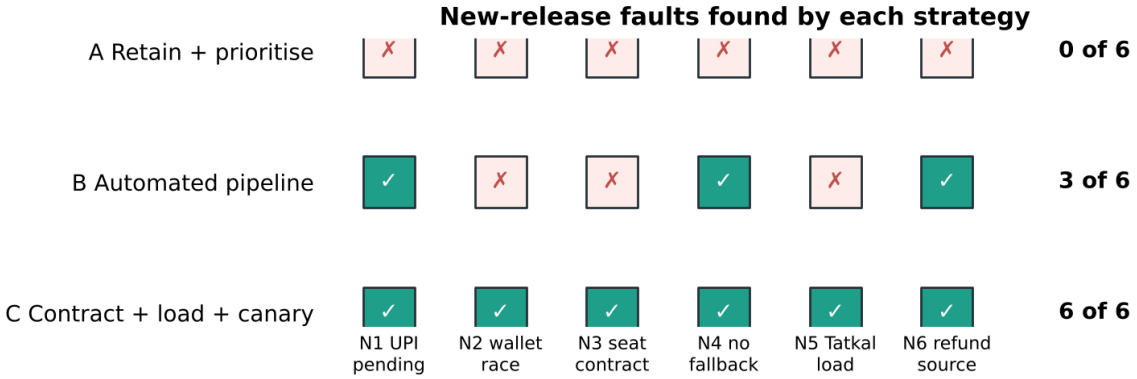


Figure 18: New-release faults found by each strategy

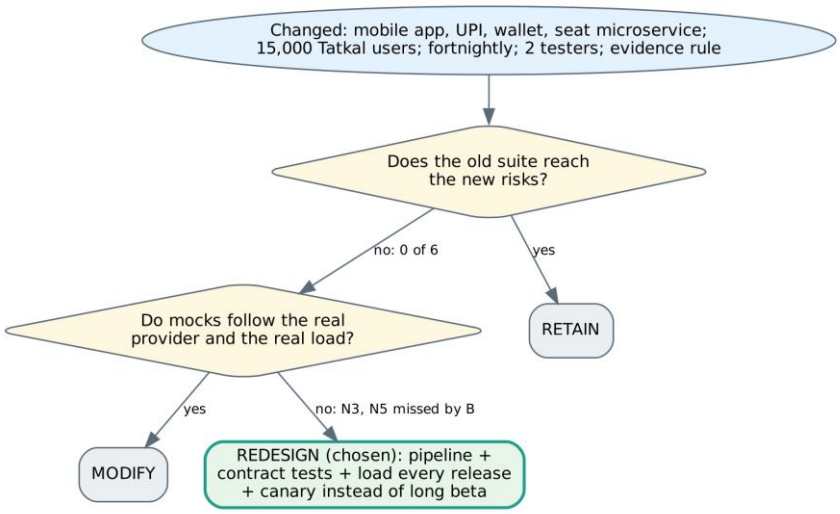


Figure 19: Decision flowchart leading to REDESIGN

Threats to validity

- Capacity comes from the load model calibrated on the prototype; the real numbers must be confirmed by a load test on production-like infrastructure.
- The effort figures per release are estimates; the fault results and run times are measured.

Marking notes and common errors

- Every result must come from the student's own execution; the decision must follow from the 2-tester limit, the fortnightly cycle, the 15,000-user peak and the evidence rule.

Mark Distribution

Task	Marks
Task 1 – Unit Test Planning Analysis	4
Task 2 – Unit Test Design Analysis	4
Task 3 – Test Harness Analysis	4
Task 4 – Integration Strategy Analysis	5
Task 5 – Integration Test Plan Analysis	5
Task 6 – System Test Analysis	5
Task 7 – Regression Strategy Analysis	5
Task 8 – Alpha, Beta and Acceptance Analysis	4
Task 9 – Implementation and Performance Evaluation	6
Task 10 – Test Strategy Adaptation Challenge	8
TOTAL	**50**

ANNEXURE A – Catalogue of Seeded Faults

ID	Level / task	Fault
U1–U4	Unit (Task 1)	GST on Sleeper; 12-h boundary coded as 11; senior from 61; ₹60 deducted once
R1–R4	Unit (Task 2)	48 h treated as the lower slab; 25 % deduction on one passenger's share; no-show not handled; negative hours accepted
H0–H6	Harness (Task 3)	Webhook after compensating refund refunds again (first build); paid but ticket failed without refund; duplicate callback makes 2 tickets; berths kept after decline; amount in rupees; late callback after timeout ignored; cancellation refund sent twice
I1–I5	Integration (Task 4)	B→P passenger count − 1; B→P amount in rupees; P→T error code dropped; T→N date as YYYY/MM/DD; R→P refund sent twice
C1, S1–S3, S4	System (Task 6)	No lock on berth allotment; SQL built by string formatting; session valid after logout; no reconciliation after crash; fixed-width e-ticket page
G1–G3	Regression (Task 7)	New 10-h boundary coded as > 10; refund SMS from a stale copy of the rule; refund paise truncated
N1–N6	New release (Task 10)	UPI pending treated as success; wallet race; provider renames berths → seats; no timeout fallback; 8 workers for 15,000 users; refund to the wrong source
R2 (real)	System (Task 6)	Not seeded: berths leak when the gateway connection fails – found by the recovery test on both builds

ANNEXURE B – Suggested Mark Split per Task

Task (marks)	Split of marks
Task 1 (4)	Plan 1; faults, coverage, effort 1; analysis 0.5; faults left 0.5; choice and exit criterion 0.5; AI critique 0.5
Task 2 (4)	Three suites 1; faults and coverage 1; maintainability 0.5; failure messages 0.5; choice 0.5; LLM table 0.5
Task 3 (4)	Three harnesses 1; defects, effort, run time 1; capabilities 0.5; wrong assumption 0.5; choice 0.5; mock critique 0.5
Task 4 (5)	Diagram, orders, stubs/drivers 1.25; faults located 1.25; analysis 1; P and R→P→T risk 0.5; choice 0.5; LLM order 0.5
Task 5 (5)	Three plans 1.5; eight B→P cases 1; checklist 0.75; late N / blocked P 0.75; choice with criteria 0.5; LLM plan 0.5
Task 6 (5)	System tests 1.25; load results and failure point 1.25; trial problems 0.75; environment 0.75; split 0.5; schedule critique 0.5
Task 7 (5)	Selection, time, faults 1.25; detection %, time saved, APFD 1.25; outdated sheet 0.75; minimisation 0.75; choice and smoke suite 0.5; LLM 0.5
Task 8 (4)	Plan, form, ten criteria 1; volunteer sessions 1; analysis 0.5; open defects 0.5; sequence 0.5; representativeness 0.5
Task 9 (6)	Measured levels 1.5; examination 1; predictions 0.5; diagram and decision table 1; transfer 0.5; festival data 0.5; AI option 1
Task 10 (8)	Develop 1.5; impact 1.5; evaluation 1.5; compare and adapt 1.5; defence 1; AI selection risks 1

ANNEXURE C – Supplied Files and Data Notes

- `rail.py` (rules, six modules, faults), `harness_common.py`, `virtual_gateway.py`, `levels.py`, `loadmodel.py`, `ext10.py`, `tests/` (pytest suites), `exp12.py`, `exp36.py`, `exp6b.py`, `exp7.py`, `exp9.py`, `exp10.py`, `figs3.py`. Run the experiments in that order (Python 3.10+, `pip install pytest coverage playwright`).

- **Measured by running code:** faults found per suite, harness, strategy and level; branch coverage; lines changed; pytest messages; run times; service times of search and booking on a 20-lakh-row table; double allotments; regression selection, APFD and minimisation; the B→P cases; the compatibility checks in Chromium; the real defects H0 and R2.
- **Modelled:** response times at 400–15,000 users (load model calibrated from measured service times, with assumed 8 workers, 5 s think time and 20 ms overhead); minutes to locate integration faults (20 min per suspect interface); effort in hours and person-days; cost per fix by level.
- **To be completed by the student:** the volunteer sessions of Task 8 (Table 31); running the JMeter plan below on the staging system; repeating the LLM checks of Tasks 4, 5 and 7 with your own assistant.

ANNEXURE D – Requirements Traceability Matrix

Requirement	Verified by	Faults	Status
Fare table, Tatkal, senior, GST on AC	Task 1 B and C suites; Task 7 B01–B08; AT-02	U1, U3	Covered
1–6 passengers, 120-day window	Task 1 B invalid tests; F2, F3; AT-08	–	Covered
Refund slabs 48 / 12 / 4 h, no-show	Task 2 suites; Task 7 R01–R14; AT-03	U2, U4, R1–R4, G1	Covered
No double allotment	F7; AT-04; festival experiment	C1	Covered
Payment and ticket status agree	F8, R1, BP-05, BP-08; virtual gateway; AT-05	H0, H1, H5, S3	Covered
Amount to gateway in paise	BP-01/02; mocks; virtual gateway contract	H4, I2, G3	Covered
≤ 3 s at 1,500 users	Load model + JMeter plan; AT-06	un-indexed search	Covered (model); confirm on staging
Security of inputs and sessions	SEC1–SEC3	S1, S2	Covered
Availability 99.5 %	none in test – needs production monitoring	–	GAP
Refund SMS shows the correct amount	N07, N08	G2	Covered (only with a current traceability sheet)

ANNEXURE E – Glossary of Terms Used in this Answer

Term	Meaning in this answer
APFD	Average Percentage of Faults Detected: $1 - \Sigma TF_i \div (n \times m) + 1 \div 2n$ for a test order.
Canary rollout	Release to a small share of users first, with automatic rollback on bad metrics.
Consumer-driven contract	The consumer's expectations of a service, run as tests against each provider build.
Driver / stub	Code that calls the unit under test / code that replaces a module it calls.
Mock	A test double that also verifies how it was called.
Service virtualisation	A stand-in service that replays recorded behaviour over the real protocol.
Smoke suite	A small set of critical tests run on every build.
Thread-based integration	Integrating the modules of one end-to-end function at a time.
Traceability sheet	Links between tests and the modules or requirements they check.
90th percentile	The response time that 90 % of requests do not exceed.

ANNEXURE F – Source Code of the Experiments and the JMeter Plan

The listings below produced every measured result. The JMeter plan (F0) is the staging version of the load script.

F0. railease_search_book.jmx – JMeter plan (structure)

```
<?xml version="1.0" encoding="UTF-8"?>
<jmeterTestPlan version="1.2" properties="5.0">
  <hashTree>
    <TestPlan testname="RailEase search-and-book"/>
    <hashTree>
      <!-- run three times with users = 400, 1500, 3000; ramp-up 60 s; duration 600 s -->
      <ThreadGroup testname="Travellers">
        <stringProp name="ThreadGroup.num_threads">${__P(users,1500)}</stringProp>
        <stringProp name="ThreadGroup.ramp_time">60</stringProp>
        <boolProp name="ThreadGroup.scheduler">true</boolProp>
        <stringProp name="ThreadGroup.duration">600</stringProp>
      </ThreadGroup>
      <hashTree>
        <CSVDataSet testname="routes"><stringProp name="filename">routes.csv</stringProp></CSVDataSet>
        <HTTPSamplerProxy testname="search"><stringProp
name="HTTPSampler.path">/api/search?src=${src}&amp;dst=${dst}&amp;day=${day}</stringProp></HTTPSamplerProxy>
        <ConstantTimer><stringProp name="ConstantTimer.delay">5000</stringProp></ConstantTimer>
        <HTTPSamplerProxy testname="book"><stringProp name="HTTPSampler.method">POST</stringProp><stringProp
name="HTTPSampler.path">/api/book</stringProp></HTTPSamplerProxy>
        <ResponseAssertion testname="status 200"/>
        <ResultCollector testname="Aggregate (average, 90 % line)"/>
      </hashTree>
    </hashTree>
  </hashTree>
</jmeterTestPlan>
```

F1. rail.py

```
"""RailEase prototype – fare and refund rules, the six modules S, B, P, T, R, N and switchable seeded faults.

Every fault is switched on by adding its id to FAULTS (one fault per run in the experiments).
Unit faults (Tasks 1, 2):  U1 GST on Sleeper; U2 12-h boundary coded as 11; U3 senior from 61; U4 flat ₹60
once, not per passenger;
                           R1 48 h treated as the lower slab; R2 25 % deduction on one passenger's share; R3
no-show not handled;
                           R4 negative hours accepted
Harness faults (Task 3):   H0 duplicate webhook after a compensating refund refunds again (found in the first
build);
                           H1 payment success + ticket failure leaves money taken, no refund; H2 duplicate
success callback makes
                           two tickets; H3 berth not released after a decline; H4 amount sent in rupees, not
paise;
                           H5 late success callback after a timeout is ignored (charged, no ticket); H6
cancellation refund sent twice on retry
Interface faults (Task 4): I1 B→P passenger count – 1; I2 B→P amount in rupees; I3 P→T error code dropped
(ticket on decline);
                           I4 T→N date as YYYY/MM/DD; I5 R→P refund sent twice
Regression faults (Task 7, after MID_HOURS 12 → 10): G1 new boundary coded as "> 10"; G2 SMS shows the refund
from a stale copy
                           of the old rule (N); G3 refund paise truncated by int() in R→P
System faults (Task 6, 9): C1 no lock on berth allotment (double allotment); S1 search built by string
formatting (SQL injection);
                           S2 session token valid after logout; S3 no reconciliation after a crash between
payment and ticket
"""
import threading, time, sqlite3, datetime as dt
from decimal import Decimal, ROUND_HALF_UP


FAULTS = set()
MID_HOURS = 12          # refund slab boundary; Task 7 changes it to 10
BASE = {"SL": 450, "3A": 1200, "2A": 1800}
```

```
def money(x):
    return float(Decimal(str(x)).quantize(Decimal("0.01"), ROUND_HALF_UP))



# ============================================================ unit level
def ticket_fare(cls, passengers, age, tatkal=False):
    """Fare for one ticket. age: one age for all passengers, or a list with one age per passenger."""
    if cls not in BASE:
        raise ValueError("unknown class")
    if not 1 <= passengers <= 6:
        raise ValueError("1 to 6 passengers per ticket")
    ages = age if isinstance(age, list) else [age] * passengers
    if len(ages) != passengers:
        raise ValueError("one age per passenger")
    base = Decimal(BASE[cls])
    total = Decimal(0)
    for a in ages:
        senior = a > 60 if "U3" in FAULTS else a >= 60
        f = base
        if senior:
            f -= base * Decimal("0.40")
        if tatkal:
            f += base * Decimal("0.30")
        total += f
    if cls != "SL" or "U1" in FAULTS:
        total *= Decimal("1.05")
    return money(total)



def refund_amount(hours_before, fare, passengers=1):
    """Refund for a cancelled ticket. hours_before=None means no-show."""
    if hours_before is None and "R3" not in FAULTS:
        return 0.0
    if hours_before < 0:
        if "R4" in FAULTS:
            return 0.0
        raise ValueError("cancellation after departure is a no-show")
    fare = Decimal(str(fare))
    top = (hours_before > 48) if "R1" in FAULTS else (hours_before >= 48)
    mid = 11 if "U2" in FAULTS else MID_HOURS
    if top:
        ded = Decimal(60) * (1 if "U4" in FAULTS else passengers)
        return money(max(Decimal(0), fare - ded))
    in_mid = hours_before > mid if "G1" in FAULTS else hours_before >= mid
    if in_mid:
        if "R2" in FAULTS:
            return money(fare - fare / passengers * Decimal("0.25"))
        return money(fare * Decimal("0.75"))
    if hours_before >= 4:
        return money(fare * Decimal("0.50"))
    return 0.0



def refund_preview(hours_before, fare, passengers=1):
    """G2: an old copy of the refund rule kept in the notification template code (still uses 12 hours)"""
    global MID_HOURS
    old, MID_HOURS = MID_HOURS, 12
    try:
        return refund_amount(hours_before, fare, passengers)
    finally:
        MID_HOURS = old



# ============================================================ modules
class GatewayError(Exception):
    pass



class Inventory:
```

```
    def __init__(self, berths=72, delay=0.0):
        self.free = {cls: list(range(1, berths + 1)) for cls in BASE}
        self.owner = {}
        self.lock = threading.Lock()
        self.delay = delay

    def allot(self, cls, n, who):
        if "C1" in FAULTS:                      # check-then-act without a lock
            avail = self.free[cls][:n]
            if len(avail) < n:
                return None
            time.sleep(self.delay)
            for b in avail:
                self.owner.setdefault((cls, b), []).append(who)
                if b in self.free[cls]:
                    self.free[cls].remove(b)
            return avail
        with self.lock:
            avail = self.free[cls][:n]
            if len(avail) < n:
                return None
            time.sleep(self.delay)
            for b in avail:
                self.owner.setdefault((cls, b), []).append(who)
                self.free[cls].remove(b)
            return avail

    def release(self, cls, berths):
        with self.lock:
            for b in berths:
                self.owner.pop((cls, b), None)
                self.free[cls].append(b)
            self.free[cls].sort()

    def double_allotted(self):
        return [k for k, v in self.owner.items() if len(v) > 1]


class Notification:                             # N
    def __init__(self, sms):
        self.sms, self.sent = sms, []

    def send(self, phone, text, travel_date):
        dt.datetime.strptime(travel_date, "%d-%m-%Y")     # contract: dd-mm-yyyy
        self.sms.send(phone, f"{text} on {travel_date}")
        self.sent.append((phone, text, travel_date))


class TicketGen:                                # T
    def __init__(self, notify, fail=False):
        self.notify, self.fail, self.tickets = notify, fail, {}

    def on_payment(self, booking, result):
        code = result.get("code", "OK" if "I3" in FAULTS else None)
        if code != "OK":
            return None
        if self.fail:
            raise RuntimeError("ticket service down")
        pnr = f"PNR{len(self.tickets) + 1:04d}"
        self.tickets[pnr] = booking
        d = booking["date"]
        ds = d.strftime("%Y/%m/%d") if "I4" in FAULTS else d.strftime("%d-%m-%Y")
        self.notify.send(booking["phone"], f"Ticket {pnr} confirmed", ds)
        return pnr


class Payment:                                  # P (talks to the external gateway)
    def __init__(self, gateway, ticket):
```

```
        self.gw, self.ticket = gateway, ticket
        self.status, self.refunds, self.pending = {}, [], {}

    def pay(self, booking):
        ref = booking["ref"]
        rupees = booking["amount"]
        amount = rupees if ("H4" in FAULTS or "I2" in FAULTS) else int(round(rupees * 100))
        if booking["passengers_sent"] != booking["passengers"]:
            raise GatewayError("passenger count mismatch")
        try:
            res = self.gw.charge(ref, amount)
        except TimeoutError:
            self.status[ref] = "PENDING"
            self.pending[ref] = booking
            return {"status": "PENDING"}
        return self._settle(booking, res)

    def _settle(self, booking, res):
        ref = booking["ref"]
        final = ("PAID",) if "H0" in FAULTS else ("PAID", "REFUNDED")       # H0: as first delivered – only
PAID was treated as final
        if self.status.get(ref) in final and "H2" not in FAULTS:
            return {"status": "PAID", "pnr": booking.get("pnr")}      # idempotent: duplicate callback ignored
        if res.get("status") != "SUCCESS":
            self.status[ref] = "DECLINED"
            out = {"status": "DECLINED", "code": res.get("code", "DECLINED")}
            if "I3" in FAULTS:
                out.pop("code")
                self.ticket.on_payment(booking, {})
            return out
        self.status[ref] = "PAID"
        try:
            pnr = self.ticket.on_payment(booking, {"code": "OK"})
        except RuntimeError:
            if "H1" not in FAULTS:
                self.gw.refund(ref, int(round(booking["amount"] * 100)))
                self.refunds.append(ref)
                self.status[ref] = "REFUNDED"
            return {"status": self.status[ref]}
        booking["pnr"] = pnr
        return {"status": "PAID", "pnr": pnr}

    def callback(self, ref, res):
        """asynchronous callback from the gateway (may arrive late or twice)"""
        booking = self.pending.get(ref) or self._known(ref)
        if booking is None:
            return None
        if self.status.get(ref) == "TIMED_OUT" and "H5" in FAULTS:
            return None                                                  # late callback dropped
        self.pending.pop(ref, None)
        return self._settle(booking, res)

    def _known(self, ref):
        return getattr(self, "_bookings", {}).get(ref)

    def expire(self, ref):
        if self.status.get(ref) == "PENDING":
            self.status[ref] = "TIMED_OUT"

    def refund(self, ref, amount_paise):
        self.gw.refund(ref, amount_paise)
        self.refunds.append(ref)



class Booking:                                  # B
    def __init__(self, inventory, payment, today=dt.date(2026, 12, 1)):
        self.inv, self.pay, self.today, self.n = inventory, payment, today, 0
        self.bookings = {}
        payment._bookings = self.bookings
```

```
    def book(self, cls, ages, tatkal=False, date=None, phone="9800000000"):
        date = date or self.today + dt.timedelta(days=30)
        if not 0 <= (date - self.today).days <= 120:
            return {"status": "REJECTED", "reason": "outside 120-day window"}
        n = len(ages)
        fare = ticket_fare(cls, n, ages, tatkal)
        self.n += 1
        ref = f"B{self.n:05d}"
        berths = self.inv.allot(cls, n, ref)
        if berths is None:
            return {"status": "WAITLIST"}
        bk = dict(ref=ref, cls=cls, berths=berths, passengers=n, passengers_sent=n - 1 if "I1" in FAULTS else
n,
                  amount=fare, date=date, phone=phone)
        self.bookings[ref] = bk
        try:
            res = self.pay.pay(bk)
        except GatewayError as e:
            self.inv.release(cls, berths)
            return {"status": "ERROR", "reason": str(e), "ref": ref}
        if res["status"] in ("DECLINED", "REFUNDED") and "H3" not in FAULTS:
            self.inv.release(cls, berths)
        res["ref"] = ref
        return res



class Cancellation:                               # R
    def __init__(self, booking, payment, notify):
        self.b, self.p, self.n, self.done = booking, payment, notify, set()


    def cancel(self, ref, hours_before):
        bk = self.b.bookings[ref]
        if ref in self.done and "H6" not in FAULTS and "I5" not in FAULTS:
            return {"status": "ALREADY_CANCELLED"}
        amt = refund_amount(hours_before, bk["amount"], bk["passengers"])
        paise = int(amt) * 100 if "G3" in FAULTS else int(round(amt * 100))
        self.p.refund(ref, paise)
        if "I5" in FAULTS:
            self.p.refund(ref, int(round(amt * 100)))
        self.done.add(ref)
        self.b.inv.release(bk["cls"], bk["berths"])
        shown = refund_preview(hours_before, bk["amount"], bk["passengers"]) if "G2" in FAULTS else amt
        self.n.send(bk["phone"], f"Refund ₹{shown:.2f} for {ref}", bk["date"].strftime("%d-%m-%Y"))
        return {"status": "CANCELLED", "refund": amt}



# ============================================================ search, sessions, recovery (system level)
class Search:                                     # S
    def __init__(self, rows=50000, index=True, seed=1):
        import random
        rng = random.Random(seed)
        self.db = sqlite3.connect(":memory:", check_same_thread=False)
        self.db.execute("create table avail(train int, src text, dst text, day text, cls text, seats int)")
        st = ["MAS", "CBE", "MDU", "TVC", "SBC", "MAQ", "ERS", "TPJ", "VSKP", "HWH"]
        d0 = dt.date(2025, 6, 1)
        data = []
        for i in range(rows):
            a, b = rng.sample(st, 2)
            data.append((1000 + i % 400, a, b, str(d0 + dt.timedelta(days=rng.randint(0, 547))),
rng.choice(list(BASE)), rng.randint(0, 72)))
        self.db.executemany("insert into avail values (?,?,?,?,?,?)", data)
        if index:
            self.db.execute("create index ix on avail(src, dst, day)")
        self.lock = threading.Lock()


    def search(self, src, dst, day):
        with self.lock:
            if "S1" in FAULTS:
                q = f"select train, cls, seats from avail where src='{src}' and dst='{dst}' and day='{day}'"
                return self.db.execute(q).fetchall()
```

```
            return self.db.execute("select train, cls, seats from avail where src=? and dst=? and day=?",
(src, dst, day)).fetchall()



class Sessions:
    def __init__(self):
        self.active = {}

    def login(self, user):
        tok = f"t{len(self.active) + 1:06d}{user}"
        self.active[tok] = user
        return tok

    def logout(self, tok):
        if "S2" not in FAULTS:
            self.active.pop(tok, None)

    def who(self, tok):
        return self.active.get(tok)



def reconcile(payment, booking):
    """run after a restart: every PAID booking without a ticket gets its ticket, or a refund."""
    if "S3" in FAULTS:
        return 0
    fixed = 0
    for ref, st in list(payment.status.items()):
        bk = booking.bookings.get(ref)
        if st == "PAID" and bk is not None and not bk.get("pnr"):
            try:
                bk["pnr"] = payment.ticket.on_payment(bk, {"code": "OK"})
            except RuntimeError:
                payment.refund(ref, int(round(bk["amount"] * 100)))
                payment.status[ref] = "REFUNDED"
            fixed += 1
    return fixed
```

F2. harness_common.py

```
"""Shared builder: wires Booking (B), Payment (P), Ticket (T), Notification (N), Cancellation (R) around a
gateway double."""
import datetime as dt
from rail import Inventory, Notification, TicketGen, Payment, Booking, Cancellation


class SMS:
    def __init__(self): self.out = []
    def send(self, phone, text): self.out.append((phone, text))


def build(gateway, ticket_fails=False):
    sms = SMS(); n = Notification(sms); t = TicketGen(n, fail=ticket_fails); p = Payment(gateway, t)
    b = Booking(Inventory(berths=8), p); r = Cancellation(b, p, n)
    return dict(B=b, P=p, T=t, N=n, R=r, sms=sms)
```

F3. virtual_gateway.py

```
"""Task 3 – C: service virtualisation. An HTTP service replays the gateway behaviour recorded from the
sandbox:
* every SUCCESS is followed ~50 ms later by an asynchronous duplicate SUCCESS callback (webhook)
* scenario 'slow': the response comes after 600 ms (client times out at 300 ms) and a late SUCCESS callback
follows
* amounts must be integer paise matching the order (recorded contract) – otherwise HTTP 400 AMOUNT_MISMATCH"""
import json, threading, time, urllib.request, http.server


class VirtualGateway:
    def __init__(self, scenario="ok", expected_paise=None):
```

```
        self.scenario, self.expected, self.refunds, self.charges, self.webhook = scenario, expected_paise, [],
[], None
        gw = self
        class H(http.server.BaseHTTPRequestHandler):
            def log_message(self, *a): pass
            def do_POST(self):
                body = json.loads(self.rfile.read(int(self.headers["Content-Length"])))
                if self.path == "/refund":
                    gw.refunds.append((body["ref"], body["amount_paise"])); return self._send(200, {"status":
"REFUNDED"})
                gw.charges.append((body["ref"], body["amount_paise"]))
                if not isinstance(body["amount_paise"], int) or (gw.expected and body["amount_paise"] !=
gw.expected):
                    return self._send(400, {"status": "FAILED", "code": "AMOUNT_MISMATCH"})
                if gw.scenario == "decline":
                    return self._send(200, {"status": "DECLINED", "code": "INSUFFICIENT_FUNDS"})
                if gw.scenario == "slow":
                    gw._later(0.9, body["ref"]); time.sleep(0.6)
                else:
                    gw._later(0.05, body["ref"])
                self._send(200, {"status": "SUCCESS"})
            def _send(self, code, obj):
                b = json.dumps(obj).encode(); self.send_response(code); self.send_header("Content-Length",
str(len(b))); self.end_headers(); self.wfile.write(b)
        self.srv = http.server.ThreadingHTTPServer(("127.0.0.1", 0), H)
        threading.Thread(target=self.srv.serve_forever, daemon=True).start()
        self.url = f"http://127.0.0.1:{self.srv.server_address[1]}"
    def _later(self, delay, ref):
        def fire():
            time.sleep(delay)
            if self.webhook: self.webhook(ref, {"status": "SUCCESS"})
        threading.Thread(target=fire, daemon=True).start()
    def stop(self): self.srv.shutdown()


class HttpGateway:
    """the real client adapter used by Payment, talking JSON over HTTP with a 300 ms timeout"""
    def __init__(self, url): self.url = url
    def _post(self, path, obj):
        req = urllib.request.Request(self.url + path, json.dumps(obj).encode(), {"Content-Type":
"application/json"})
        try:
            with urllib.request.urlopen(req, timeout=0.3) as r: return json.loads(r.read())
        except urllib.error.HTTPError as e: return json.loads(e.read())
    def charge(self, ref, amount): return self._post("/charge", {"ref": ref, "amount_paise": amount})
    def refund(self, ref, amount): return self._post("/refund", {"ref": ref, "amount_paise": amount})
```

F4. levels.py

```
"""Test levels shared by Tasks 4–6 and 9: end-to-end interface checks, B→P integration tests, system tests."""
import re, threading, datetime as dt
from unittest.mock import Mock
import rail
from harness_common import build



class Gw:
    def __init__(self, mode="ok"): self.mode, self.charges, self.refunds = mode, [], []
    def charge(self, ref, amount):
        self.charges.append((ref, amount)); return {"status": "SUCCESS"} if self.mode == "ok" else {"status":
"DECLINED", "code": "INSUFFICIENT_FUNDS"}
    def refund(self, ref, amount): self.refunds.append((ref, amount))



def e2e_checks():
    """end-to-end checks through the top of the system (used by big-bang and thread-based integration)"""
    out = {}
    g = Gw(); s = build(g)
    try:
        res = s["B"].book("3A", [30, 32]); out["book_ok"] = res["status"] == "PAID" and len(s["T"].tickets) ==
1
        out["amount_ok"] = g.charges and g.charges[0][1] == 252000
```

```
        import re
        out["sms_ok"] = bool(s["sms"].out) and re.search(r"\d\d-\d\d-\d{4}$", s["sms"].out[-1][1]) is not None
    except Exception as e:
        out["book_ok"] = False; out["crash"] = type(e).__name__
    g2 = Gw("decline"); s2 = build(g2)
    try:
        s2["B"].book("SL", [30]); out["decline_ok"] = s2["T"].tickets == {}
    except Exception as e:
        out["decline_ok"] = False
    g3 = Gw(); s3 = build(g3)
    try:
        res = s3["B"].book("3A", [30]); s3["R"].cancel(res["ref"], 20); out["refund_ok"] = len(g3.refunds) ==
1
    except Exception as e:
        out["refund_ok"] = False
    return out



def bp_tests():
    out = []
    def T(name, fn):
        try: out.append((name, bool(fn())))
        except Exception as e: out.append((name, False))
    def amount():
        g = Mock(); g.charge.return_value = {"status": "SUCCESS"}; s = build(g); r = s["B"].book("3A", [30,
32]); return g.charge.call_args[0][1] == 252000
    def currency():
        g = Mock(); g.charge.return_value = {"status": "SUCCESS"}; s = build(g); s["B"].book("SL", [30]);
return isinstance(g.charge.call_args[0][1], int)
    def timeout():
        g = Mock(); g.charge.side_effect = TimeoutError; s = build(g); r = s["B"].book("SL", [30]); return
r["status"] == "PENDING" and s["T"].tickets == {}
    def duplicate():
        g = Mock(); g.charge.return_value = {"status": "SUCCESS"}; s = build(g); r = s["B"].book("SL", [30]);
s["P"].callback(r["ref"], {"status": "SUCCESS"}); return len(s["T"].tickets) == 1
    def partial():
        g = Mock(); g.charge.return_value = {"status": "SUCCESS"}; s = build(g, ticket_fails=True); r =
s["B"].book("2A", [50]); return r["status"] == "REFUNDED" and g.refund.called
    def retry():
        g = Mock(); g.charge.side_effect = [TimeoutError, {"status": "SUCCESS"}]; s = build(g); r1 =
s["B"].book("SL", [30]); r2 = s["B"].book("SL", [30])
        return r1["status"] == "PENDING" and r2["status"] == "PAID" and len(s["B"].inv.free["SL"]) == 6
    def paxcount():
        g = Mock(); g.charge.return_value = {"status": "SUCCESS"}; s = build(g); r = s["B"].book("SL", [30,
31, 32]); return r["status"] == "PAID"
    def expired():
        g = Mock(); g.charge.side_effect = TimeoutError; s = build(g); r = s["B"].book("SL", [30]);
s["P"].expire(r["ref"])
        s["P"].callback(r["ref"], {"status": "SUCCESS"}); return s["P"].status[r["ref"]] == "PAID" and
len(s["T"].tickets) == 1
    for n, f in [("BP-01 amount in paise", amount), ("BP-02 currency/int type", currency), ("BP-03 timeout →
pending", timeout),
                 ("BP-04 duplicate payment callback", duplicate), ("BP-05 partial failure (paid, ticket
fails)", partial),
                 ("BP-06 retry after timeout", retry), ("BP-07 passenger count", paxcount), ("BP-08 callback
after session expiry", expired)]:
        T(n, f)
    return out



def sys_tests():
    res = {}
    def rec(cat, name, fn):
        try: ok = bool(fn())
        except Exception as e: ok = False
        res[name] = (cat, ok)
    # ---- A functional and usability
    def f1():
        s = build(Gw()); r = s["B"].book("SL", [30, 65, 70], date=dt.date(2026, 12, 25)); return r["status"]
== "PAID" and len(s["T"].tickets) == 1
    def f2():
        s = build(Gw()); return s["B"].book("SL", [30] * 7)["status"] == "PAID"
```

```
    def f2b():
        s = build(Gw())
        try: s["B"].book("SL", [30] * 7); return False
        except ValueError: return True
    def f3():
        s = build(Gw()); return s["B"].book("SL", [30], date=s["B"].today + dt.timedelta(days=121))["status"]
== "REJECTED"
    def f4():
        g = Gw(); s = build(g); r = s["B"].book("3A", [30]); c = s["R"].cancel(r["ref"], 30); return
c["refund"] == 945.0 and g.refunds[0][1] == 94500
    def f5():
        s = build(Gw()); r = s["B"].book("SL", [30]); return "confirmed" in s["sms"].out[-1][1] and
r.get("pnr", "").startswith("PNR")
    def f6():
        g = Gw(); s = build(g)
        for i in range(8): s["B"].book("SL", [30])
        return s["B"].book("SL", [30])["status"] == "WAITLIST"
    def f7():   # concurrency: 20 travellers race for the last 8 berths (functional rule "one berth, one
passenger")
        g = Gw(); s = build(g); s["B"].inv.delay = 0.002
        ths = [threading.Thread(target=s["B"].book, args=("SL", [30])) for _ in range(20)]
        [t.start() for t in ths]; [t.join() for t in ths]; return s["B"].inv.double_allotted() == []
    def f8():   # payment success but ticket service down → must refund (status agreement)
        g = Gw(); s = build(g, ticket_fails=True); r = s["B"].book("2A", [40]); return r["status"] ==
"REFUNDED" and g.refunds
    for n, f in [("F1 family booking with seniors", f1), ("F2 7 passengers rejected", f2b), ("F3 121 days
ahead rejected", f3),
                 ("F4 cancel 30 h before: 25 % deduction", f4), ("F5 e-ticket and SMS issued", f5), ("F6
waitlist after the last berth", f6),
                 ("F7 no double allotment under 20 parallel bookings", f7), ("F8 paid but no ticket →
refunded", f8)]:
        rec("A", n, f)
    # ---- C recovery, security, compatibility (compatibility via Playwright in exp_compat.py)
    def r1():   # crash between payment and ticket, then restart with reconciliation
        g = Gw(); s = build(g); bk = dict(ref="X1", cls="SL", berths=[1], passengers=1, passengers_sent=1,
amount=450.0, date=dt.date(2026, 12, 30), phone="9")
        s["B"].bookings["X1"] = bk; s["P"].status["X1"] = "PAID"; n = rail.reconcile(s["P"], s["B"]); return n
== 1 and bk.get("pnr")
    def r2():   # gateway down (exception) → no berth lost
        g = Mock(); g.charge.side_effect = ConnectionError; s = build(g)
        try: s["B"].book("SL", [30])
        except ConnectionError: pass
        return len(s["B"].inv.free["SL"]) == 8
    def r3():   # duplicate webhook after restart
        g = Gw(); s = build(g); r = s["B"].book("SL", [30]); s["P"].callback(r["ref"], {"status": "SUCCESS"});
return len(s["T"].tickets) == 1
    def s1():
        srch = rail.Search(rows=2000); rows = srch.search("MAS", "CBE' OR 1=1 --", "2026-01-01"); return
len(rows) < 50
    def s2():
        ss = rail.Sessions(); t = ss.login("clerk1"); ss.logout(t); return ss.who(t) is None
    def s3():
        s = build(Gw())
        try: s["B"].book("SL", []); return False
        except ValueError: return True
    for n, f in [("R1 crash after payment → reconciled on restart", r1), ("R2 gateway down → berths kept",
r2),
                 ("R3 duplicate webhook after restart", r3), ("SEC1 SQL injection in station field", s1),
                 ("SEC2 session token dead after logout", s2), ("SEC3 empty passenger list rejected", s3)]:
        rec("C", n, f)
    return res
```

F5. loadmodel.py

```
"""Closed-system discrete-event load model: users alternate search and book with exponential think time;
the server has WORKERS parallel workers; service time = measured function time + fixed overhead
(exponential)."""
import heapq, random, statistics
OVER = 0.020          # assumed web/app overhead per request (framework, template, network inside the data
centre)
WORKERS = 8           # assumed worker processes on the staging server
THINK = 5.0           # assumed think time between requests of one user
```

```
def simulate(users, s_search, s_book, workers=WORKERS, think=THINK, horizon=240.0, seed=1, burst=False):
    rng = random.Random(seed); ev = []; q = []; busy = 0; resp = []
    for u in range(users):
        start = rng.uniform(0, 1.0) if burst else rng.expovariate(1 / think)
        heapq.heappush(ev, (start, "arrive", u, "search"))
    def svc(kind): return rng.expovariate(1 / ((s_search if kind == "search" else s_book) + OVER))
    while ev:
        t, typ, u, kind = heapq.heappop(ev)
        if t > horizon: break
        if typ == "arrive":
            if busy < workers:
                busy += 1; heapq.heappush(ev, (t + svc(kind), "done", u, (kind, t)))
            else:
                q.append((t, u, kind))
        else:
            k, t_in = kind
            if t > 30: resp.append(t - t_in)
            nxt = "book" if k == "search" else "search"
            heapq.heappush(ev, (t + rng.expovariate(1 / think), "arrive", u, nxt))
            if q:
                ta, uu, kk = q.pop(0); heapq.heappush(ev, (t + svc(kk), "done", uu, (kk, ta)))
            else:
                busy -= 1
    resp.sort()
    return dict(avg=statistics.mean(resp), p90=resp[int(0.9 * len(resp))], n=len(resp))
```

F6. ext10.py

```
"""Task 10 – RailEase extended with UPI, wallet debit and a seat-recommendation microservice (HTTP JSON).
Faults: N1 UPI PENDING treated as SUCCESS (ticket before money); N2 wallet debit without a lock (double debit
under parallel sessions);
        N3 seat-recommendation provider v2 renames "berths" to "seats" (contract break); N4 consumer has no
timeout fallback for
        the seat service; N5 capacity: Tatkal 10:00 burst on the current 8-worker infrastructure (performance,
see exp10);
        N6 refund credited to the wrong source (UPI refund sent to the wallet)."""
import json, threading, time, http.server, urllib.request, urllib.parse, urllib.error
from rail import FAULTS



class Wallet:
    def __init__(self):
        self.bal, self.lock, self.ledger = {}, threading.Lock(), []


    def debit(self, user, amount):
        if "N2" in FAULTS:
            b = self.bal.get(user, 0)
            if b < amount: return False
            time.sleep(0.001)
            self.bal[user] = self.bal.get(user, 0) - amount
            self.ledger.append(("debit", user, amount)); return True
        with self.lock:
            b = self.bal.get(user, 0)
            if b < amount: return False
            self.bal[user] = b - amount
            self.ledger.append(("debit", user, amount)); return True


    def credit(self, user, amount):
        with self.lock:
            self.bal[user] = self.bal.get(user, 0) + amount; self.ledger.append(("credit", user, amount))



def upi_confirm(status):
    """booking may be confirmed only when the UPI collect request is SUCCESS"""
    return status == "SUCCESS" or ("N1" in FAULTS and status == "PENDING")
```

```
def refund_to_source(payment, amount, wallet, upi):
    src = payment["source"]
    if "N6" in FAULTS and src == "UPI":
        src = "WALLET"
    if src == "WALLET":
        wallet.credit(payment["user"], amount); return "WALLET"
    upi.append((payment["vpa"], amount)); return "UPI"



class SeatRecService:
    """provider microservice; version 2 is the new release (with fault N3 it renames the field)"""
    def __init__(self, version=1, delay=0.0):
        svc = self
        self.version, self.delay = version, delay
        class H(http.server.BaseHTTPRequestHandler):
            def log_message(self, *a): pass
            def do_GET(self):
                q = urllib.parse.parse_qs(urllib.parse.urlparse(self.path).query); n = int(q["n"][0])
                time.sleep(svc.delay)
                key = "seats" if (svc.version == 2 and "N3" in FAULTS) else "berths"
                body = json.dumps({key: [{"coach": "B2", "berth": 17 + i, "type": "LB" if i == 0 else "MB"}
for i in range(n)], "version": svc.version}).encode()
                self.send_response(200); self.send_header("Content-Type", "application/json");
self.send_header("Content-Length", str(len(body))); self.end_headers(); self.wfile.write(body)
        self.srv = http.server.ThreadingHTTPServer(("127.0.0.1", 0), H)
        threading.Thread(target=self.srv.serve_forever, daemon=True).start()
        self.url = f"http://127.0.0.1:{self.srv.server_address[1]}"

    def stop(self): self.srv.shutdown()



def recommend(url, train, n, timeout=0.2):
    """consumer (Booking): returns a list of berths or [] if the service is slow; the booking must never fail
because of it"""
    try:
        with urllib.request.urlopen(f"{url}/recommend?train={train}&n={n}", timeout=timeout) as r:
            data = json.loads(r.read())
        return data["berths"]
    except (TimeoutError, urllib.error.URLError):
        if "N4" in FAULTS: raise
        return []
```

F7. exp12.py

```
"""Tasks 1 and 2: run every unit-test suite on the correct build and on each single-fault build; measure
branch coverage."""
import json, os, subprocess, sys, inspect, difflib, time
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
import rail


RES = os.path.join(HERE, "results_unit.jsonl")
if os.path.exists(RES): os.remove(RES)
PLAN = {"test_t1_a_adhoc": ["U1", "U2", "U3", "U4"], "test_t1_b_plan": ["U1", "U2", "U3", "U4"],
"test_t1_c_bdd": ["U1", "U2", "U3", "U4"],
        "test_t1_ai": ["U2", "U4", "R1", "R2", "R3", "R4"],
        "test_t2_a_spec": ["R1", "R2", "R3", "R4"], "test_t2_b_struct": ["R1", "R2", "R3", "R4"],
"test_t2_c_data": ["R1", "R2", "R3", "R4"]}
runtime = {}
for suite, faults in PLAN.items():
    for f in [""] + faults:
        t0 = time.perf_counter()
        subprocess.run([sys.executable, "-m", "pytest", "-q", "-p", "no:cacheprovider", f"tests/{suite}.py"],
cwd=HERE,
                       env=dict(os.environ, FAULT=f, RESULTS=RES), capture_output=True)
        runtime[(suite, f or "none")] = time.perf_counter() - t0
rows = [json.loads(l) for l in open(RES)]
```

```
def summary(suite):
    ok = {r["test"] for r in rows if r["suite"] == suite and r["fault"] == "none" and r["outcome"] ==
"passed"}
    n = len({r["test"] for r in rows if r["suite"] == suite and r["fault"] == "none"})
    found = {}
    for f in PLAN[suite]:
        kills = [r["test"] for r in rows if r["suite"] == suite and r["fault"] == f and r["outcome"] ==
"failed" and r["test"] in ok]
        found[f] = kills
    return n, found



# branch coverage of ticket_fare and refund_amount with coverage.py
def cov(suite, funcs):
    data = os.path.join(HERE, ".cov_" + suite)
    subprocess.run([sys.executable, "-m", "coverage", "run", "--branch", f"--data-file={data}", "--
include=rail.py", "-m", "pytest", "-q",
                    "-p", "no:cacheprovider", f"tests/{suite}.py"], cwd=HERE, capture_output=True,
env=dict(os.environ, FAULT=""))
    out = os.path.join(HERE, f".cov_{suite}.json")
    subprocess.run([sys.executable, "-m", "coverage", "json", f"--data-file={data}", "-o", out], cwd=HERE,
capture_output=True)
    d = list(json.load(open(out))["files"].values())[0]
    lines = set()
    for fn in funcs:
        src, start = inspect.getsourcelines(getattr(rail, fn))
        lines |= set(range(start + 1, start + len(src)))
    SRCL = open(os.path.join(HERE, "rail.py")).read().split("\n")
    fault_switch = {i + 1 for i, l in enumerate(SRCL) if l.strip().startswith('if "') and "in FAULTS" in l}
# seeded-fault switches are not product branches
    eb = [b for b in d["executed_branches"] if b[0] in lines and b[0] not in fault_switch]
    mb = [b for b in d["missing_branches"] if b[0] in lines and b[0] not in fault_switch]
    el = [l for l in d["executed_lines"] if l in lines]; ml = [l for l in d["missing_lines"] if l in lines]
    return dict(branch=len(eb) / (len(eb) + len(mb)), br=f"{len(eb)}/{len(eb)+len(mb)}", stmt=len(el) /
(len(el) + len(ml)), miss_br=mb)



R = {}
for s in PLAN:
    n, found = summary(s)
    funcs = ["ticket_fare", "refund_amount"] if s.startswith("test_t1") and s != "test_t1_ai" else
["refund_amount"]
    R[s] = dict(n=n, found={f: v for f, v in found.items()}, cov=cov(s, funcs))
# failure messages for fault R1 in the three Task 2 styles
R["messages"] = {}
for s in ("test_t2_a_spec", "test_t2_b_struct", "test_t2_c_data"):
    p = subprocess.run([sys.executable, "-m", "pytest", "-q", "-p", "no:cacheprovider", "-rN", "--tb=line",
f"tests/{s}.py"], cwd=HERE,
                       env=dict(os.environ, FAULT="R2"), capture_output=True, text=True)
    R["messages"][s] = [l for l in p.stdout.splitlines() if "assert" in l or "Error" in l][:2]
    p = subprocess.run([sys.executable, "-m", "pytest", "-q", "-p", "no:cacheprovider", "--tb=short",
f"tests/{s}.py"], cwd=HERE,
                       env=dict(os.environ, FAULT="R2"), capture_output=True, text=True)
    R["messages"][s + "_short"] = p.stdout[-1400:]


# maintainability: rule change 12 h -> 10 h, the edited suites are in tests_v2/
R["maint"] = {}
for s in ("test_t2_a_spec", "test_t2_b_struct", "test_t2_c_data"):
    a = open(os.path.join(HERE, "tests", s + ".py")).read().splitlines()
    b = open(os.path.join(HERE, "tests_v2", s + ".py")).read().splitlines()
    ch = [l for l in difflib.unified_diff(a, b, lineterm="", n=0) if (l.startswith("+") or l.startswith("-"))
and not l.startswith(("+++", "---"))]
    R["maint"][s] = dict(changed=len([l for l in ch if l.startswith("-")]), added=len([l for l in ch if
l.startswith("+")]) - len([l for l in ch if l.startswith("-")]))
json.dump(R, open(os.path.join(HERE, "results_12.json"), "w"), indent=1)
for s in PLAN: print(s, R[s]["n"], {f: len(v) for f, v in R[s]["found"].items()}, R[s]["cov"]["br"],
R[s]["cov"]["miss_br"])
print(R["maint"]); print(json.dumps(R["messages"], indent=1)[:2500])
```

F8. exp36.py

```
"""Tasks 3–6: harness comparison, integration strategies, B→P integration tests, system tests and the load
model."""
import json, os, subprocess, sys, time, random, heapq, statistics, threading, datetime as dt
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
import rail
from unittest.mock import Mock
from harness_common import build
from levels import Gw, e2e_checks, bp_tests, sys_tests


R = {}
# ================================================================== TASK 3
RES = os.path.join(HERE, "results_t3.jsonl")
if os.path.exists(RES): os.remove(RES)
HF = ["H0", "H1", "H2", "H3", "H4", "H5", "H6"]
R["t3"] = {}
for suite in ("test_t3_a_stubs", "test_t3_b_mocks", "test_t3_c_virtual"):
    times = []
    for f in [""] + HF:
        t0 = time.perf_counter()
        subprocess.run([sys.executable, "-m", "pytest", "-q", "-p", "no:cacheprovider", f"tests/{suite}.py"],
cwd=HERE,
                       env=dict(os.environ, FAULT=f, RESULTS=RES), capture_output=True)
        times.append(time.perf_counter() - t0)
    rows = [json.loads(l) for l in open(RES) if json.loads(l)["suite"] == suite]
    ok = {r["test"] for r in rows if r["fault"] == "none" and r["outcome"] == "passed"}
    found = {f: [r["test"] for r in rows if r["fault"] == f and r["outcome"] == "failed" and r["test"] in ok]
for f in HF}
    p = subprocess.run([sys.executable, "-m", "pytest", "-q", "-p", "no:cacheprovider", "--durations=0",
f"tests/{suite}.py"], cwd=HERE, capture_output=True, text=True)
    tline = [l for l in p.stdout.splitlines() if " passed" in l][-1]
    src = open(os.path.join(HERE, "tests", suite + ".py")).read().splitlines()
    loc = len([l for l in src if l.strip() and not l.strip().startswith("#")])
    if suite.endswith("virtual"):
        loc += len([l for l in open(os.path.join(HERE, "virtual_gateway.py")).read().splitlines() if l.strip()
and not l.strip().startswith("#")])
    R["t3"][suite] = dict(n=len(ok), found=found, pytest=tline, loc=loc)

# ================================================================== TASK 4  integration strategies
EDGES = [("S", "B"), ("B", "P"), ("P", "T"), ("T", "N"), ("R", "P"), ("R", "N")]
IF = {"I1": ("B", "P"), "I2": ("B", "P"), "I3": ("P", "T"), "I4": ("T", "N"), "I5": ("R", "P")}



def failing(faults):
    rail.FAULTS.clear(); rail.FAULTS.update(faults)
    try:
        c = e2e_checks()
    finally:
        rail.FAULTS.clear()
    return sorted(k for k, v in c.items() if v is False or k == "crash")



ORDERS = {
    "Top-down": [("S",), ("B",), ("P",), ("T",), ("N",), ("R",)],
    "Bottom-up": [("N",), ("T",), ("P",), ("B",), ("R",), ("S",)],
    "Thread-based": [("S", "B", "P", "T", "N"), ("R",)],
    "Big-bang": [("S", "B", "P", "T", "N", "R")],
}



def stubs_drivers(order):
    done, stubs, drivers = set(), set(), set()
    for step in order:
        done |= set(step)
        for a, b in EDGES:
            if a in done and b not in done: stubs.add(b)
            if b in done and a not in done: drivers.add(a)
    return sorted(stubs), sorted(drivers)
```

```
R["t4"] = {"single": {f: failing([f]) for f in IF}, "all_at_once": failing(list(IF)), "orders": {}}
for name, order in ORDERS.items():
    done = set(); res = {}
    for k, step in enumerate(order, 1):
        before = set(done); done |= set(step)
        new_if = [e for e in EDGES if e[0] in done and e[1] in done and not (e[0] in before and e[1] in
before)]
        for f, e in IF.items():
            if e in new_if and f not in res:
                res[f] = dict(step=k, candidates=len(new_if))
    s, d = stubs_drivers(order)
    R["t4"]["orders"][name] = dict(steps=len(order), found=res, stubs=s, drivers=d)


# ================================================================== TASK 5  eight B→P integration tests on
the real B and P
R["t5"] = {}
for f in ["", "I1", "I2", "H1", "H2", "H4", "H5"]:
    rail.FAULTS.clear(); rail.FAULTS.update([f] if f else [])
    R["t5"][f or "none"] = bp_tests()
rail.FAULTS.clear()


# ================================================================== TASK 6  system tests on the trial build
TRIAL = {"C1", "S1", "S2", "S3", "H1"}



rail.FAULTS.clear(); R["t6_correct"] = sys_tests()
rail.FAULTS.update(TRIAL); R["t6_trial"] = sys_tests(); rail.FAULTS.clear()


# ---------------- load model (B): measured service times + closed-system discrete-event simulation
def measure(rows, index, n=150):
    s = rail.Search(rows=rows, index=index); rng = random.Random(3)
    st = ["MAS", "CBE", "MDU", "TVC", "SBC", "MAQ", "ERS", "TPJ", "VSKP", "HWH"]
    t0 = time.perf_counter()
    for _ in range(n):
        a, b = rng.sample(st, 2); s.search(a, b, f"2026-0{rng.randint(1, 9)}-1{rng.randint(0, 9)}")
    return (time.perf_counter() - t0) / n



ROWS = 2_000_000
svc_scan = measure(ROWS, False, 40); svc_idx = measure(ROWS, True, 400)
g = Gw(); s = build(g); t0 = time.perf_counter()
for i in range(300):
    s["B"].inv = rail.Inventory(berths=8); s["B"].book("3A", [30, 40])
svc_book = (time.perf_counter() - t0) / 300
from loadmodel import simulate, OVER, WORKERS, THINK



load = {}
for label, ss in (("trial build (no index)", svc_scan), ("fixed build (index)", svc_idx)):
    load[label] = {u: simulate(u, ss, svc_book) for u in (100, 200, 300, 400, 600, 800, 1000, 1500, 2000,
2500, 3000)}
cross = {}
for label, ss in (("trial build (no index)", svc_scan), ("fixed build (index)", svc_idx)):
    lo = next((u for u in range(50, 4001, 50) if simulate(u, ss, svc_book, horizon=150)["p90"] > 3.0), None)
    cross[label] = lo
burst = {u: simulate(u, svc_idx, svc_book, burst=True, horizon=60) for u in (1500,)}
R["t6_load"] = dict(svc_scan_ms=svc_scan * 1000, svc_idx_ms=svc_idx * 1000, svc_book_ms=svc_book * 1000,
rows=ROWS, workers=WORKERS,
                    think=THINK, over_ms=OVER * 1000, load={k: {str(u): v for u, v in d.items()} for k, d in
load.items()}, cross=cross,
                    burst={str(u): v for u, v in burst.items()})
json.dump(R, open(os.path.join(HERE, "results_36.json"), "w"), indent=1, default=str)
print(json.dumps({k: v for k, v in R.items() if k != "t6_load"}, indent=1, default=str)[:6000])
print(json.dumps(R["t6_load"], indent=1)[:3000])
```

F9. exp6b.py

```
"""Task 6 extras: compatibility checks of the e-ticket page in Chromium at three device sizes (Playwright),
and the worker-count sensitivity of the load model."""
import json, os, sys
HERE = os.path.dirname(os.path.abspath(__file__)); sys.path.insert(0, HERE)
from playwright.sync_api import sync_playwright
from loadmodel import simulate


def ticket_html(fixed):
    table = 'style="width:900px"' if fixed else 'style="width:100%"'
    return f"""<!doctype html><html><head><meta name="viewport" content="width=device-width,initial-scale=1">
<style>body{{font-family:sans-serif;margin:12px}} td{{border:1px solid #999;padding:4px}} table{{border-
collapse:collapse}}</style></head>
<body><h2>RailEase e-ticket</h2><p id="pnr">PNR0001</p><table {table}><tr><td>Train 12633 Kanyakumari
Exp</td><td>MAS → CBE</td>
<td>31-12-2026 21:05</td><td>3A B2-17, B2-18</td><td id="fare">₹2,520.00</td></tr></table></body></html>"""


R = {"compat": {}}
with sync_playwright() as p:
    br = p.chromium.launch()
    for build, fixed in (("correct", False), ("trial (S4 fixed 900 px table)", True)):
        R["compat"][build] = {}
        for name, w, h in (("phone 360×740", 360, 740), ("tablet 768×1024", 768, 1024), ("desktop 1366×768",
1366, 768)):
            pg = br.new_page(viewport={"width": w, "height": h}); pg.set_content(ticket_html(fixed))
            sw = pg.evaluate("document.documentElement.scrollWidth"); vis = pg.evaluate(
                "(() => { const r = document.getElementById('fare').getBoundingClientRect(); return r.right <=
window.innerWidth; })()")
            R["compat"][build][name] = dict(scroll_width=sw, viewport=w, ok=sw <= w and vis)
            pg.close()
    br.close()
d = json.load(open(os.path.join(HERE, "results_36.json")))["t6_load"]
sc, ix, bk = d["svc_scan_ms"] / 1000, d["svc_idx_ms"] / 1000, d["svc_book_ms"] / 1000
R["sens"] = {}
for w in (4, 8, 16):
    for lab, ss in (("no index", sc), ("index", ix)):
        R["sens"][f"{w} workers, {lab}"] = next((u for u in range(50, 9001, 50) if simulate(u, ss, bk,
workers=w, horizon=150)["p90"] > 3.0), None)
json.dump(R, open(os.path.join(HERE, "results_6b.json"), "w"), indent=1)
print(json.dumps(R, indent=1))
```

F10. exp7.py

```
"""Task 7: a 60-test representative regression suite, the refund-slab change (12 h → 10 h) with three
regression faults,
and retest-all, impact-based and prioritised selection with APFD."""
import json, os, sys, sys as _s, datetime as dt, itertools
HERE = os.path.dirname(os.path.abspath(__file__)); sys.path.insert(0, HERE)
import rail
from harness_common import build
from decimal import Decimal



class Gw:
    def __init__(self, mode="ok"): self.mode, self.charges, self.refunds = mode, [], []
    def charge(self, ref, amount):
        self.charges.append((ref, amount)); return {"status": "SUCCESS"} if self.mode == "ok" else {"status":
"DECLINED", "code": "X"}
    def refund(self, ref, amount): self.refunds.append((ref, amount))



def spec_refund(h, fare, n, mid):
    if h is None: return 0.0
    f = Decimal(str(fare))
    if h >= 48: return float(max(Decimal(0), f - 60 * n).quantize(Decimal("0.01")))
    if h >= mid: return float((f * Decimal("0.75")).quantize(Decimal("0.01")))
    if h >= 4: return float((f * Decimal("0.50")).quantize(Decimal("0.01")))
    return 0.0
```

```
TESTS = []   # (id, module, sheet_current, sheet_outdated, criticality 1-3, fn)



def add(tid, mod, cur, old, crit):
    def deco(fn):
        TESTS.append((tid, mod, cur, old, crit, fn)); return fn
    return deco



# ---------- S: search (8)
for i, (a, b) in enumerate([("MAS", "CBE"), ("CBE", "MAS"), ("MDU", "TVC"), ("SBC", "ERS"), ("HWH", "VSKP"),
("TPJ", "MAQ"), ("MAS", "SBC"), ("ERS", "MAS")], 1):
    @add(f"S{i:02d}", "S", "S", "S", 1)
    def _(a=a, b=b):
        srch = SEARCH; rows = srch.search(a, b, "2026-01-05"); return all(len(r) == 3 for r in rows)
# ---------- B: booking and fares (12)
FARES = [("SL", [30], False, 450.0), ("SL", [60], False, 270.0), ("3A", [30], False, 1260.0), ("3A", [30, 30],
False, 2520.0),
         ("2A", [30], True, 2457.0), ("SL", [30] * 6, False, 2700.0), ("3A", [65], True, 1134.0), ("2A", [59],
False, 1890.0)]
for i, (c, ages, tk, exp) in enumerate(FARES, 1):
    @add(f"B{i:02d}", "B", "B,P", "B,P", 2)
    def _(c=c, ages=ages, tk=tk, exp=exp):
        g = Gw(); s = build(g); r = s["B"].book(c, ages, tk); return r["status"] == "PAID" and g.charges[0][1]
== int(round(exp * 100))
@add("B09", "B", "B", "B", 2)
def _():
    s = build(Gw()); return s["B"].book("SL", [30], date=s["B"].today + dt.timedelta(days=121))["status"] ==
"REJECTED"
@add("B10", "B", "B", "B", 2)
def _():
    s = build(Gw())
    for _ in range(8): s["B"].book("SL", [30])
    return s["B"].book("SL", [30])["status"] == "WAITLIST"
@add("B11", "B", "B,P", "B,P", 2)
def _():
    s = build(Gw("decline")); s["B"].book("3A", [30, 30]); return len(s["B"].inv.free["3A"]) == 8
@add("B12", "B", "B", "B", 3)
def _():
    try: rail.ticket_fare("SL", 7, 30); return False
    except ValueError: return True
# ---------- P: payment (10)
for i, (c, ages) in enumerate([("SL", [30]), ("3A", [30]), ("2A", [40]), ("3A", [30, 70]), ("SL", [30, 31,
32])], 1):
    @add(f"P{i:02d}", "P", "P,T", "P,T", 3)
    def _(c=c, ages=ages):
        g = Gw(); s = build(g); r = s["B"].book(c, ages); return r["status"] == "PAID" and
isinstance(g.charges[0][1], int)
@add("P06", "P", "P,T", "P,T", 3)
def _():
    g = Gw(); s = build(g, ticket_fails=True); r = s["B"].book("2A", [50]); return r["status"] == "REFUNDED"
and g.refunds == [(r["ref"], 189000)]
@add("P07", "P", "P,T", "P,T", 3)
def _():
    g = Gw(); s = build(g); r = s["B"].book("SL", [30]); s["P"].callback(r["ref"], {"status": "SUCCESS"});
return len(s["T"].tickets) == 1
@add("P08", "P", "P", "P", 3)
def _():
    g = Gw(); s = build(g); s["P"].refund("Z1", 141750); return g.refunds == [("Z1", 141750)]
@add("P09", "P", "P,R", "P", 3)
def _():   # refund of a 2A fare cancelled 20 h before: paise exact
    g = Gw(); s = build(g); r = s["B"].book("2A", [30]); s["R"].cancel(r["ref"], 20); return g.refunds[-1][1]
== 141750
@add("P10", "P", "P", "P", 3)
def _():
    from unittest.mock import Mock
    g = Mock(); g.charge.side_effect = TimeoutError; s = build(g); r = s["B"].book("SL", [30]); return
r["status"] == "PENDING"
# ---------- T: tickets (8)
for i in range(1, 9):
    @add(f"T{i:02d}", "T", "T,N", "T,N", 2)
```

```
    def _(i=i):
        s = build(Gw()); r = s["B"].book("SL" if i % 2 else "3A", [30] * (1 + i % 3)); return r.get("pnr",
"").startswith("PNR")
# ---------- R: cancellation and refund (14)
RCASES = [(72, "SL", [30]), (48, "3A", [30, 30]), (47, "3A", [30]), (24, "2A", [30]), (12, "SL", [30]), (11,
"3A", [30]), (10, "2A", [30]),
          (9.9, "3A", [30]), (8, "SL", [30, 31]), (4, "3A", [30]), (3.9, "SL", [30]), (1, "2A", [30])]
for i, (h, c, ages) in enumerate(RCASES, 1):
    @add(f"R{i:02d}", "R", "R,P", "R,P", 3)
    def _(h=h, c=c, ages=ages):
        g = Gw(); s = build(g); r = s["B"].book(c, ages); out = s["R"].cancel(r["ref"], h)
        fare = rail.ticket_fare(c, len(ages), ages); return out["refund"] == spec_refund(h, fare, len(ages),
10)
@add("R13", "R", "R", "R", 2)
def _():
    g = Gw(); s = build(g); r = s["B"].book("SL", [30]); s["R"].cancel(r["ref"], 30); s["R"].cancel(r["ref"],
30); return len(g.refunds) == 1
@add("R14", "R", "R", "R", 2)
def _():
    return rail.refund_amount(None, 900) == 0.0
# ---------- N: notifications (8)
for i in range(1, 7):
    @add(f"N{i:02d}", "N", "N", "N", 1)
    def _(i=i):
        s = build(Gw()); s["B"].book("SL", [30] * i); return "confirmed" in s["sms"].out[-1][1]
@add("N07", "N", "N,R", "N", 2)
def _():   # refund SMS shows the refund of the new rule (cancel 11 h before: 75 %)
    s = build(Gw()); r = s["B"].book("2A", [30]); s["R"].cancel(r["ref"], 11); return "₹1417.50" in
s["sms"].out[-1][1]
@add("N08", "N", "N,R", "N", 2)
def _():
    s = build(Gw()); r = s["B"].book("3A", [30]); s["R"].cancel(r["ref"], 30); return "₹945.00" in
s["sms"].out[-1][1]


assert len(TESTS) == 60, len(TESTS)
SEARCH = rail.Search(rows=3000)
MIN_PER_TEST = 96 * 60 / 650          # 650 tests take 3 testers × 4 days × 8 h = 96 tester-hours



def run(faults, mid):
    rail.FAULTS.clear(); rail.FAULTS.update(faults); rail.MID_HOURS = mid
    out = {}
    for tid, *_, fn in TESTS:
        try: out[tid] = bool(fn())
        except Exception: out[tid] = False
    rail.FAULTS.clear(); rail.MID_HOURS = 12
    return out



base = run([], 10)
assert all(base.values()), [k for k, v in base.items() if not v]
NEWF = ["G1", "G2", "G3"]
kills = {f: [t for t, ok in run([f], 10).items() if not ok] for f in NEWF}
# fault history: failures of each test on the earlier seeded faults (old rule, 12 h)
HIST = ["U1", "U2", "U3", "U4", "R1", "R2", "R3", "R4", "H1", "H2", "H3", "H4", "H6", "I1", "I2", "I3", "I4",
"I5"]
base12 = run([], 12)
hist = {t: 0 for t, *_ in TESTS}
hist_matrix = {}
for f in HIST:
    res = run([f], 12)
    hist_matrix[f] = [t for t, ok in res.items() if not ok and base12[t]]
    for t in hist_matrix[f]: hist[t] += 1
# coverage of the changed code (refund_amount executed?)
changed = {}
for tid, *_, fn in TESTS:
    hit = [False]
    def tr(frame, ev, arg):
        if ev == "call" and frame.f_code.co_name == "refund_amount": hit[0] = True
        return None
    rail.MID_HOURS = 10; _s.settrace(tr)
    try: fn()
```

```
    except Exception: pass
    _s.settrace(None); rail.MID_HOURS = 12
    changed[tid] = hit[0]



def apfd(order, kills):
    n, m = len(order), len(kills)
    pos = []
    for f, ks in kills.items():
        p = next((i + 1 for i, t in enumerate(order) if t in ks), None)
        pos.append(p if p else n + 1)
    return 1 - sum(pos) / (n * m) + 1 / (2 * n), pos



ALL = [t for t, *_ in TESTS]
meta = {t: (mod, cur, old, crit) for t, mod, cur, old, crit, _ in TESTS}
R = {"n": 60, "min_per_test": MIN_PER_TEST, "kills": kills, "hist": hist, "changed": changed}
# A retest-all in suite order
a, pa = apfd(ALL, kills)
R["A"] = dict(sel=len(ALL), hours=len(ALL) * MIN_PER_TEST / 60, found=[f for f in NEWF if set(kills[f]) &
set(ALL)], apfd=a, pos=pa)
# B impact-based: changed module R; select tests whose sheet links R (current sheet) – and with the outdated
sheet
for label, col in (("B", 1), ("B_outdated", 2)):
    sel = [t for t in ALL if "R" in meta[t][col].split(",")]
    order = sel
    a, pa = apfd(order, {f: [k for k in ks if k in sel] for f, ks in kills.items()})
    R[label] = dict(sel=len(sel), hours=len(sel) * MIN_PER_TEST / 60, found=[f for f in NEWF if set(kills[f])
& set(sel)], apfd=a, pos=pa, tests=sel)
# C prioritised: score = 0.4 × history + 0.4 × covers changed code + 0.2 × criticality; time limit 25 % of
retest-all
mh = max(hist.values()) or 1
score = {t: 0.4 * hist[t] / mh + 0.4 * changed[t] + 0.2 * meta[t][3] / 3 for t in ALL}
orderC = sorted(ALL, key=lambda t: (-score[t], t))
limit = int(0.25 * len(ALL))
a, pa = apfd(orderC, kills)
R["C"] = dict(sel=limit, hours=limit * MIN_PER_TEST / 60, found=[f for f in NEWF if set(kills[f]) &
set(orderC[:limit])], apfd=a, pos=pa,
              top=[(t, round(score[t], 2), hist[t], changed[t], meta[t][3]) for t in orderC[:limit]])
# minimisation by greedy set cover over the historical faults, then check on the new faults
cover = {t: {f for f, ks in hist_matrix.items() if t in ks} for t in ALL}
need = set(f for f, ks in hist_matrix.items() if ks); mini = []
while need:
    t = max(ALL, key=lambda x: (len(cover[x] & need), -ALL.index(x))); mini.append(t); need -= cover[t]
R["mini"] = dict(n=len(mini), tests=mini, found=[f for f in NEWF if set(kills[f]) & set(mini)])
R["never_failed"] = [t for t in ALL if hist[t] == 0]
R["hist_detected"] = {f: len(v) for f, v in hist_matrix.items()}
json.dump(R, open(os.path.join(HERE, "results_7.json"), "w"), indent=1, default=str)
for k in ("A", "B", "B_outdated", "C", "mini"): print(k, {x: y for x, y in R[k].items() if x not in ("tests",
"top")})
print("kills", kills); print("never failed", len(R["never_failed"])); print(R["C"]["top"])
```

F11. exp9.py

```
"""Task 9: three complete multi-level strategies on a prototype with ten seeded faults, and the festival-data
experiment."""
import json, os, subprocess, sys, random, threading
from concurrent.futures import ThreadPoolExecutor
HERE = os.path.dirname(os.path.abspath(__file__)); sys.path.insert(0, HERE)
import rail
from levels import bp_tests, sys_tests, e2e_checks


TEN = ["U1", "U3", "U4", "R1", "I2", "I3", "H2", "C1", "S1", "S3"]



def pytest_fails(files, fault):
    p = subprocess.run([sys.executable, "-m", "pytest", "-q", "-p", "no:cacheprovider"] + [f"tests/{f}.py" for
f in files],
                       cwd=HERE, env=dict(os.environ, FAULT=fault), capture_output=True, text=True)
```

```
    return "failed" in p.stdout.splitlines()[-1]



def fn_fails(fault, fn):
    rail.FAULTS.clear(); rail.FAULTS.add(fault)
    try: return fn()
    finally: rail.FAULTS.clear()



def level_unit(files): return lambda f: pytest_fails(files, f) if files else False
def level_bp(): return lambda f: fn_fails(f, lambda: not all(ok for _, ok in bp_tests()))
def level_e2e(): return lambda f: fn_fails(f, lambda: any(v is False or k == "crash" for k, v in
e2e_checks().items()))
def level_sys(names):
    def run(f):
        res = fn_fails(f, sys_tests)
        return any(not ok for n, (c, ok) in res.items() if n.split(" ")[0] in names and n.split(" ")[0] !=
"R2")
    return run



def any_of(*levels): return lambda f: any(l(f) for l in levels)



STRATS = {
    "A Bottom-heavy": [("Unit", any_of(level_unit(["test_t1_b_plan", "test_t2_a_spec"]))),
                       ("Integration", any_of(level_unit(["test_t3_b_mocks"]), level_bp(), level_e2e())),
                       ("System", level_sys({"F1", "F4", "F5"})),
                       ("Acceptance", level_unit(["test_t1_c_bdd"]))],
    "B Top-heavy": [("Unit", level_unit(["test_t1_a_adhoc"])), ("Integration", lambda f: False),
                    ("System", any_of(level_sys({"F1", "F2", "F3", "F4", "F5", "F6", "F7", "F8", "R1", "R3",
"SEC1", "SEC2", "SEC3"}), level_e2e())),
                    ("Acceptance", level_unit(["test_t1_c_bdd"]))],
    "C Balanced pyramid": [("Unit", level_unit(["test_t1_b_plan", "test_t2_a_spec"])),
                           ("Integration", any_of(level_unit(["test_t3_b_mocks"]), level_bp(), level_e2e())),
                           ("System", level_sys({"F4", "F7", "F8", "R1", "R3", "SEC1", "SEC2"})),
                           ("Acceptance", level_unit(["test_t1_c_bdd"]))],
}
WEEK = {"Unit": 1, "Integration": 2, "System": 3, "Acceptance": 5}
COST = {"Unit": 1500, "Integration": 5000, "System": 12000, "Acceptance": 25000, "Production": 80000}
EFFORT = {"A Bottom-heavy": {"Unit": 12, "Integration": 14, "System": 6, "Acceptance": 4},
          "B Top-heavy": {"Unit": 2, "Integration": 2, "System": 30, "Acceptance": 16},
          "C Balanced pyramid": {"Unit": 10, "Integration": 10, "System": 14, "Acceptance": 10}}
R = {"faults": TEN, "strats": {}}
for name, levels in STRATS.items():
    found_at = {}
    for f in TEN:
        found_at[f] = next((lv for lv, chk in levels if chk(f)), "Production")
    per = {}
    remaining = len(TEN)
    for lv, _ in levels:
        k = sum(1 for f in TEN if found_at[f] == lv)
        per[lv] = dict(found=k, entering=remaining, dre=k / remaining if remaining else 0)
        remaining -= k
    cost = sum(COST[found_at[f]] for f in TEN)
    R["strats"][name] = dict(found_at=found_at, per=per, leaked=[f for f in TEN if found_at[f] ==
"Production"], cost=cost,
                             effort=EFFORT[name], effort_total=sum(EFFORT[name].values()),
                             dre=1 - len([f for f in TEN if found_at[f] == "Production"]) / len(TEN))


# ---------------- festival limitation: double allotment with bookings spread like past data vs concentrated
like the festival peak
def race(concentrated, n=400, trains=400, seed=5):
    rng = random.Random(seed)
    invs = {t: rail.Inventory(berths=72, delay=0.001) for t in range(trains)}
    rail.FAULTS.clear(); rail.FAULTS.add("C1")
    picks = [(rng.randrange(5) if (concentrated and rng.random() < 0.6) else rng.randrange(trains)) for _ in
range(n)]
    with ThreadPoolExecutor(64) as ex:
        list(ex.map(lambda i: invs[picks[i]].allot("SL", 1, f"u{i}"), range(n)))
```

```
    rail.FAULTS.clear()
    return sum(len(v.double_allotted()) for v in invs.values())



R["festival"] = {"past pattern (spread over 400 trains)": [race(False, seed=s) for s in range(3)],
                 "festival pattern (60 % on 5 trains)": [race(True, seed=s) for s in range(3)]}
json.dump(R, open(os.path.join(HERE, "results_9.json"), "w"), indent=1)
for n, v in R["strats"].items(): print(n, v["found_at"], v["per"], v["cost"], v["dre"])
print(R["festival"])
```

F12. exp10.py

```
"""Task 10: strategies A (retain + prioritise), B (automated multi-level pipeline) and C (contract tests +
automated load + canary)
on the extended prototype; capacity for the 15,000-user Tatkal burst."""
import json, os, sys, time, random, threading, subprocess
from concurrent.futures import ThreadPoolExecutor
HERE = os.path.dirname(os.path.abspath(__file__)); sys.path.insert(0, HERE)
import rail
from ext10 import Wallet, upi_confirm, refund_to_source, SeatRecService, recommend
from loadmodel import simulate
from levels import bp_tests, sys_tests

NF = ["N1", "N2", "N3", "N4", "N5", "N6"]
R = {}



def with_fault(f, fn):
    rail.FAULTS.clear()
    if f: rail.FAULTS.add(f)
    try:
        return fn()
    except Exception:
        return False
    finally:
        rail.FAULTS.clear()



# ---------- A: retain + prioritise – the high-risk part of the Task 9 suite, unchanged
def strat_A():
    ok = all(o for _, o in bp_tests()) and all(o for n, (c, o) in sys_tests().items() if n.split(" ")[0] in
{"F4", "F7", "F8", "R1", "SEC1", "SEC2"})
    return ok



# ---------- B: automated pipeline – new unit and integration tests written by the team (mocks follow the
team's assumptions)
def b_unit_upi():
    return upi_confirm("SUCCESS") and not upi_confirm("PENDING") and not upi_confirm("FAILED")
def b_unit_wallet():
    w = Wallet(); w.bal["u1"] = 1000; a = w.debit("u1", 600); b = w.debit("u1", 600); return a and not b and
w.bal["u1"] == 400
def b_unit_refund_source():
    w = Wallet(); upi = []
    return refund_to_source({"source": "UPI", "vpa": "a@okbank", "user": "u1"}, 500, w, upi) == "UPI" and upi
== [("a@okbank", 500)]
def b_int_seat_mock():          # the consumer is tested against a mock built from the consumer's own
assumption {"berths": [...]}
    from unittest import mock
    fake = mock.MagicMock(); fake.read.return_value = json.dumps({"berths": [{"coach": "B2", "berth":
17}]}).encode()
    fake.__enter__.return_value = fake
    with mock.patch("urllib.request.urlopen", return_value=fake):
        return recommend("http://seat", 12633, 1)[0]["berth"] == 17
def b_int_seat_timeout():
    from unittest import mock
    with mock.patch("urllib.request.urlopen", side_effect=TimeoutError):
        return recommend("http://seat", 12633, 1) == []
```

```
B_TESTS = [b_unit_upi, b_unit_wallet, b_unit_refund_source, b_int_seat_mock, b_int_seat_timeout]



def strat_B():
    return strat_A() and all(t() for t in B_TESTS)



# ---------- C: B + consumer-driven contract test against the provider's new build + automated load test +
canary
def c_contract():
    svc = SeatRecService(version=2)
    try:
        data = json.loads(urllib_get(svc.url + "/recommend?train=12633&n=2"))
        return isinstance(data.get("berths"), list) and all({"coach", "berth"} <= set(b) for b in
data["berths"])
    finally:
        svc.stop()
def urllib_get(u):
    import urllib.request
    with urllib.request.urlopen(u, timeout=1) as r: return r.read()
def c_consumer_slow_provider():  # real HTTP provider answering in 0.5 s – consumer must fall back within its
0.2 s budget
    svc = SeatRecService(version=1, delay=0.5)
    try:
        t = time.perf_counter(); out = recommend(svc.url, 12633, 2); return out == [] and time.perf_counter()
- t < 0.4
    finally:
        svc.stop()



def canary(fault, users=50, share=0.05, seed=3):
    """5 % of traffic to the new build: each canary user has a mobile and a web session paying in parallel
from one wallet"""
    rail.FAULTS.clear()
    if fault: rail.FAULTS.add(fault)
    try:
        w = Wallet(); rng = random.Random(seed); bad = 0
        for u in range(users):
            w.bal[u] = 1000
        with ThreadPoolExecutor(32) as ex:
            list(ex.map(lambda u: [w.debit(u, 700) for _ in range(1)], [u for u in range(users) for _ in
range(2)]))
        bad = sum(1 for u in range(users) if w.bal[u] < 0)
        return bad
    finally:
        rail.FAULTS.clear()



LOAD = json.load(open(os.path.join(HERE, "results_36.json")))["t6_load"]
SI, SB = LOAD["svc_idx_ms"] / 1000, LOAD["svc_book_ms"] / 1000



def tatkal(users, workers):
    return simulate(users, SI, SB, workers=workers, think=5.0, horizon=120, burst=True)



def c_load(fault):
    """automated load test: 15,000 users arriving within the first second at 10:00; pass if p90 ≤ 3 s.
    N5 = the release keeps today's 8 workers; the correct release scales to the capacity found below."""
    w = 8 if fault == "N5" else NEED
    return tatkal(15000, w)["p90"] <= 3.0



# capacity needed for the Tatkal burst
cap = {}
for w in (8, 16, 32, 48, 64, 96):
    cap[w] = tatkal(15000, w)
```

```
NEED = next(w for w in (8, 16, 32, 48, 64, 96) if cap[w]["p90"] <= 3.0)



def strat_C(f):
    return strat_B() and c_contract() and c_consumer_slow_provider() and c_load(f) and canary(None) == 0 and
(canary("N2") == 0 if f == "N2" else True)



res = {}
for f in NF:
    a = with_fault(f, strat_A) if f != "N5" else True                  # A has no load test of the new peak
    b = with_fault(f, strat_B) if f != "N5" else True
    rail.FAULTS.clear()
    if f == "N2":
        c = with_fault(None, strat_C) and canary("N2") == 0
    elif f == "N5":
        c = with_fault(None, strat_C) and c_load("N5")
    else:
        c = with_fault(f, lambda: strat_C(f))
    res[f] = dict(A=not a, B=not b, C=not c)
R["found"] = res
R["canary_N2_affected"] = canary("N2")
R["capacity"] = {str(w): v for w, v in cap.items()}
R["need_workers"] = NEED
# run times of the automated parts
t0 = time.perf_counter(); strat_B(); R["t_B"] = time.perf_counter() - t0
t0 = time.perf_counter(); c_contract(); c_consumer_slow_provider(); R["t_contract"] = time.perf_counter() - t0
t0 = time.perf_counter(); tatkal(15000, NEED); R["t_load"] = time.perf_counter() - t0
json.dump(R, open(os.path.join(HERE, "results_10.json"), "w"), indent=1)
print(json.dumps(R, indent=1))
```

F13. tests/ – pytest suites

tests/conftest.py

```
import json, os, sys, pytest
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
import rail


@pytest.fixture(autouse=True)
def _faults():
    f = os.environ.get("FAULT", "")
    rail.FAULTS.clear()
    if f:
        rail.FAULTS.update(f.split(","))
    yield
    rail.FAULTS.clear()


def pytest_runtest_logreport(report):
    if report.when == "call" and os.environ.get("RESULTS"):
        with open(os.environ["RESULTS"], "a") as fh:
            fh.write(json.dumps({"suite": os.path.basename(report.fspath).replace(".py", ""), "test":
report.nodeid.split("::")[-1],
                                 "fault": os.environ.get("FAULT") or "none", "outcome": report.outcome,
                                 "msg": (report.longreprtext.strip().split("\n")[-1] if report.failed else
"")}) + "\n")
```

tests/test_t1_a_adhoc.py

```
# Task 1 – A: ad hoc developer unit tests (written "when time permits" for the code the developer found hard)
from rail import ticket_fare, refund_amount
def test_fare_two_adults_3a(): assert ticket_fare("3A", 2, 30) == 2520.0
def test_refund_too_late(): assert refund_amount(2, 1000) == 0
def test_refund_day_before(): assert refund_amount(20, 1000) == 750.0
```

tests/test_t1_ai.py

```
# Task 1 – tests produced by an AI code assistant (Claude) from the source of refund_amount only
```

```
import pytest
from rail import refund_amount
def test_more_than_48_hours(): assert refund_amount(72, 1000) == 940.0
def test_between_12_and_48(): assert refund_amount(24, 1000) == 750.0
def test_between_4_and_12(): assert refund_amount(8, 1000) == 500.0
def test_less_than_4(): assert refund_amount(1, 1000) == 0.0
def test_no_show(): assert refund_amount(None, 1000) == 0.0
def test_negative():
    with pytest.raises(ValueError): refund_amount(-5, 1000)
def test_small_fare_not_negative(): assert refund_amount(72, 50) == 0.0
def test_multiple_passengers(): assert refund_amount(72, 1000, 2) == 880.0
```

tests/test_t1_b_plan.py

```
# Task 1 – B: plan-driven unit tests (unit test plan: EP + BVA from the rules, branch coverage target 100 %)
import pytest
from rail import ticket_fare, refund_amount
FARE = [("SL", 1, 30, False, 450.0), ("3A", 1, 30, False, 1260.0), ("2A", 1, 30, True, 2457.0), ("SL", 1, 60,
False, 270.0),
        ("SL", 1, 59, False, 450.0), ("3A", 1, 60, True, 1134.0), ("SL", 2, [30, 65], False, 720.0), ("SL", 6,
30, False, 2700.0)]
@pytest.mark.parametrize("cls,n,age,tk,exp", FARE)
def test_fare(cls, n, age, tk, exp): assert ticket_fare(cls, n, age, tk) == exp
@pytest.mark.parametrize("cls,n", [("SL", 0), ("SL", 7), ("1A", 1)])
def test_fare_invalid(cls, n):
    with pytest.raises(ValueError): ticket_fare(cls, n, 30)
REF = [(48, 1000, 1, 940.0), (47.9, 1000, 1, 750.0), (12, 1000, 1, 750.0), (11.9, 1000, 1, 500.0), (4, 1000,
1, 500.0),
       (3.9, 1000, 1, 0.0), (None, 1000, 1, 0.0), (48, 1350, 3, 1170.0)]
@pytest.mark.parametrize("h,fare,n,exp", REF)
def test_refund(h, fare, n, exp): assert refund_amount(h, fare, n) == exp
def test_refund_negative():
    with pytest.raises(ValueError): refund_amount(-1, 1000)
```

tests/test_t1_c_bdd.py

```
# Task 1 – C: behaviour-driven examples agreed with the booking-clerk representative (Given–When–Then)
from rail import ticket_fare, refund_amount
def test_family_of_four_in_sleeper():
    # Given a family of 4 adults in Sleeper  When the fare is shown  Then they pay 4 × ₹450 and no GST
    assert ticket_fare("SL", 4, 35) == 1800.0
def test_senior_in_ac3():
    # Given a senior citizen aged 65 in AC 3-tier  When the fare is shown  Then 40 % concession on base, then
5 % GST
    assert ticket_fare("3A", 1, 65) == 756.0
def test_tatkal_ac2():
    # Given one adult booking Tatkal in AC 2-tier  Then base + 30 % + 5 % GST
    assert ticket_fare("2A", 1, 40, True) == 2457.0
def test_cancel_two_days_before_two_passengers():
    # Given 2 passengers in AC 3-tier (₹2,520)  When they cancel exactly 48 hours before  Then ₹60 per
passenger is deducted
    assert refund_amount(48, 2520, 2) == 2400.0
def test_cancel_evening_before():
    # When they cancel 14 hours before departure  Then 25 % is deducted
    assert refund_amount(14, 2520, 2) == 1890.0
def test_cancel_six_hours_before():
    # When they cancel 6 hours before  Then half is refunded
    assert refund_amount(6, 2520, 2) == 1260.0
def test_no_show():
    # Given the passenger did not travel and did not cancel  Then nothing is refunded
    assert refund_amount(None, 2520, 2) == 0.0
def test_cancel_two_hours_before():
    assert refund_amount(2, 900, 2) == 0.0
```

tests/test_t2_a_spec.py

```
# Task 2 – A: specification-based unit tests for refund_amount (equivalence classes and boundaries 48 / 12 / 4
h)
import pytest
from rail import refund_amount
def test_class_48_plus(): assert refund_amount(100, 1000) == 940.0
```

```
def test_boundary_48(): assert refund_amount(48, 1000) == 940.0
def test_just_below_48(): assert refund_amount(47.9, 1000) == 750.0
def test_boundary_12(): assert refund_amount(12, 1000) == 750.0
def test_just_below_12(): assert refund_amount(11.9, 1000) == 500.0
def test_boundary_4(): assert refund_amount(4, 1000) == 500.0
def test_just_below_4(): assert refund_amount(3.9, 1000) == 0.0
def test_departure_time(): assert refund_amount(0, 1000) == 0.0
def test_no_show(): assert refund_amount(None, 1000) == 0.0
def test_after_departure_invalid():
    with pytest.raises(ValueError): refund_amount(-1, 1000)
def test_per_passenger_deduction(): assert refund_amount(48, 1500, 3) == 1320.0
def test_25_percent_three_passengers(): assert refund_amount(20, 1500, 3) == 1125.0
```

tests/test_t2_b_struct.py

```
# Task 2 – B: structure-based unit tests: every statement, branch and condition of refund_amount
import pytest
from rail import refund_amount
def test_noshow_branch(): assert refund_amount(None, 800) == 0.0
def test_negative_branch():
    with pytest.raises(ValueError): refund_amount(-3, 800)
def test_top_branch(): assert refund_amount(60, 800) == 740.0
def test_top_branch_floor_zero(): assert refund_amount(50, 100, 2) == 0.0
def test_mid_branch(): assert refund_amount(30, 800) == 600.0
def test_low_branch(): assert refund_amount(8, 800) == 400.0
def test_none_left_branch(): assert refund_amount(2, 800) == 0.0
def test_top_branch_two_passengers(): assert refund_amount(60, 800, 2) == 680.0
```

tests/test_t2_c_data.py

```
# Task 2 – C: one parameterised data-driven test; new cases are new rows
import pytest
from rail import refund_amount
ROWS = [  # hours_before, fare, passengers, expected refund, rule
    (100, 1000, 1, 940.0, "48 h or more: ₹60 per passenger"),
    (48, 1000, 1, 940.0, "48 h boundary"),
    (47.9, 1000, 1, 750.0, "just under 48 h: 25 %"),
    (48, 1500, 3, 1320.0, "₹60 × 3 passengers"),
    (20, 1500, 3, 1125.0, "25 % of the whole fare"),
    (12, 1000, 1, 750.0, "12 h boundary"),
    (11.9, 1000, 1, 500.0, "just under 12 h: 50 %"),
    (4, 1000, 1, 500.0, "4 h boundary"),
    (3.9, 1000, 1, 0.0, "under 4 h: nothing"),
    (None, 1000, 1, 0.0, "no-show"),
]
@pytest.mark.parametrize("h,fare,n,exp,rule", ROWS, ids=[r[4] for r in ROWS])
def test_refund_table(h, fare, n, exp, rule):
    assert refund_amount(h, fare, n) == exp, f"rule: {rule}"
def test_refund_after_departure_rejected():
    with pytest.raises(ValueError): refund_amount(-1, 1000)
```

tests/test_t3_a_stubs.py

```
# Task 3 – A: hand-written driver and stub. The stub returns fixed answers; the test functions act as drivers.
from harness_common import build


class StubGateway:
    """fixed answers: 'ok', 'decline' or 'timeout'; counts refunds"""
    def __init__(self, mode="ok"): self.mode, self.refunds, self.charges = mode, [], []
    def charge(self, ref, amount):
        self.charges.append((ref, amount))
        if self.mode == "timeout": raise TimeoutError
        return {"status": "SUCCESS"} if self.mode == "ok" else {"status": "DECLINED", "code":
"INSUFFICIENT_FUNDS"}
    def refund(self, ref, amount): self.refunds.append((ref, amount))


def test_success_gives_ticket():
    s = build(StubGateway("ok")); res = s["B"].book("3A", [30, 32])
    assert res["status"] == "PAID" and len(s["T"].tickets) == 1
def test_decline_releases_berths():
```

```
    s = build(StubGateway("decline")); s["B"].book("SL", [30, 31, 32])
    assert res_free(s) == 8 and s["T"].tickets == {}
def test_timeout_not_confirmed():            # team assumption: a timeout means the payment failed
    s = build(StubGateway("timeout")); res = s["B"].book("SL", [40])
    assert res["status"] == "PENDING" and s["T"].tickets == {}
def test_duplicate_callback_one_ticket():
    g = StubGateway("ok"); s = build(g); res = s["B"].book("SL", [40])
    s["P"].callback(res["ref"], {"status": "SUCCESS"})
    assert len(s["T"].tickets) == 1
def test_paid_but_ticket_fails_is_refunded():
    g = StubGateway("ok"); s = build(g, ticket_fails=True); res = s["B"].book("2A", [50])
    assert res["status"] == "REFUNDED" and len(g.refunds) == 1
def test_cancel_refunds_once():
    g = StubGateway("ok"); s = build(g); res = s["B"].book("3A", [30])
    s["R"].cancel(res["ref"], 20); s["R"].cancel(res["ref"], 20)
    assert len(g.refunds) == 1


def res_free(s): return len(s["B"].inv.free["SL"])
```

tests/test_t3_b_mocks.py

```
# Task 3 – B: unittest.mock test doubles; the mock also verifies HOW the unit called the gateway
from unittest.mock import Mock
from harness_common import build


def gw(**kw):
    m = Mock(); m.charge.return_value = {"status": "SUCCESS"}; m.configure_mock(**kw); return m


def test_success_charges_exact_paise():
    g = gw(); s = build(g); res = s["B"].book("3A", [30, 32])         # 2 × 1,200 × 1.05 = ₹2,520 = 2,52,000
paise
    g.charge.assert_called_once_with(res["ref"], 252000); assert len(s["T"].tickets) == 1
def test_decline_releases_berths():
    g = gw(); g.charge.return_value = {"status": "DECLINED", "code": "INSUFFICIENT_FUNDS"}; s = build(g)
    s["B"].book("SL", [30, 31, 32]); assert len(s["B"].inv.free["SL"]) == 8 and s["T"].tickets == {}
def test_timeout_not_confirmed():            # same team assumption as the stubs
    g = gw(); g.charge.side_effect = TimeoutError; s = build(g); res = s["B"].book("SL", [40])
    assert res["status"] == "PENDING" and s["T"].tickets == {}
def test_duplicate_callback_one_ticket():
    g = gw(); s = build(g); res = s["B"].book("SL", [40]); s["P"].callback(res["ref"], {"status": "SUCCESS"})
    assert len(s["T"].tickets) == 1
def test_paid_but_ticket_fails_is_refunded():
    g = gw(); s = build(g, ticket_fails=True); res = s["B"].book("2A", [50])
    g.refund.assert_called_once_with(res["ref"], 189000); assert res["status"] == "REFUNDED"
def test_cancel_refunds_once_with_right_amount():
    g = gw(); s = build(g); res = s["B"].book("3A", [30]); s["R"].cancel(res["ref"], 20);
s["R"].cancel(res["ref"], 20)
    g.refund.assert_called_once_with(res["ref"], 94500)               # 75 % of ₹1,260
```

tests/test_t3_c_virtual.py

```
# Task 3 – C: tests through the virtual gateway over the real HTTP interface (recorded behaviour, delays,
webhooks)
import time
from harness_common import build
from virtual_gateway import VirtualGateway, HttpGateway


def setup(scenario="ok", expected=None, ticket_fails=False):
    v = VirtualGateway(scenario, expected); s = build(HttpGateway(v.url), ticket_fails); v.webhook =
s["P"].callback
    return v, s


def test_success_gives_one_ticket_even_with_webhook():
    v, s = setup("ok", 252000); res = s["B"].book("3A", [30, 32]); time.sleep(0.15); v.stop()
    assert res["status"] == "PAID" and len(s["T"].tickets) == 1
def test_decline_releases_berths():
    v, s = setup("decline"); s["B"].book("SL", [30, 31, 32]); v.stop()
    assert len(s["B"].inv.free["SL"]) == 8 and s["T"].tickets == {}
def test_timeout_then_late_success_is_settled():
    v, s = setup("slow"); res = s["B"].book("SL", [40]); s["P"].expire(res["ref"]); time.sleep(0.8); v.stop()
```

```
    st = s["P"].status[res["ref"]]
    assert (st == "PAID" and len(s["T"].tickets) == 1) or st == "REFUNDED"      # payment and ticket status
must agree
def test_duplicate_callback_one_ticket():
    v, s = setup("ok"); res = s["B"].book("SL", [40]); time.sleep(0.15); s["P"].callback(res["ref"],
{"status": "SUCCESS"}); v.stop()
    assert len(s["T"].tickets) == 1
def test_paid_but_ticket_fails_is_refunded():
    v, s = setup("ok", ticket_fails=True); res = s["B"].book("2A", [50]); time.sleep(0.15); v.stop()
    assert res["status"] == "REFUNDED" and v.refunds == [(res["ref"], 189000)]
def test_cancel_refunds_once():
    v, s = setup("ok"); res = s["B"].book("3A", [30]); time.sleep(0.1); s["R"].cancel(res["ref"], 20);
s["R"].cancel(res["ref"], 20); v.stop()
    assert v.refunds == [(res["ref"], 94500)]
def test_amount_contract_paise():
    v, s = setup("ok", 58500); res = s["B"].book("SL", [30], True); time.sleep(0.1); v.stop()     # Tatkal SL:
₹450 × 1.3 = ₹585 = 58,500 paise
    assert res["status"] == "PAID"
```

ANNEXURE G – Viva Questions and Model Answers (Tasks 1–10)

Five likely examiner questions per task, answered from the measured results in this document.

Task 1 – Unit test planning

Q1. What does a unit test plan contain?

Units and features to test, what is not tested, the approach (black-box and white-box), test data, pass/fail criteria, entry and exit criteria, environment and tools, responsibilities, effort and schedule.

Q2. Why did the BDD examples miss U2 and U3?

The clerk's examples were realistic (a 65-year-old senior, cancellations 14 h and 6 h before) but none sat on the 11–12 h or age-60 edges.

Q3. What is your unit exit criterion?

All planned cases pass, branch coverage ≥ 95 % per unit, every rule boundary has a test on and below it, and no critical unit defect is open.

Q4. Why exclude the fault switches from branch coverage?

They are experiment scaffolding, not product branches; counting them would lower coverage for code that does not exist in the real product.

Q5. How trustworthy were the AI-generated tests?

All 8 expected values were right and branch coverage was 100 %, but every value sat mid-slab, so they missed the 11-hour, 48-hour and multi-passenger 25 % faults.

Task 2 – Unit test design

Q1. Why did 100 % branch coverage not find R1?

The structure-based tests used 60 h and 30 h; both branches ran, but nothing tested exactly 48 h, where R1 differs.

Q2. How many lines changed for the 12 h → 10 h change in each style?

Specification-based 3, structure-based 0 (but no test near 10 h), data-driven 2 rows.

Q3. Which style gave the clearest failure message?

Data-driven: the row id and custom message "rule: 25 % of the whole fare" named the rule along with the actual and expected values.

Q4. What was wrong in the LLM parameter table?

One of 12 rows: it wrote a negative refund (−₹20) for a ₹100, two-passenger ticket cancelled 50 h before; the refund is floored at ₹0.

Q5. When is a combination of styles better?

When the function has control flow that the rule table does not show, such as the floor at zero; add the structure case as one more data row.

Task 3 – Test harness

Q1. What is the difference between a stub, a mock and a virtual service?

A stub returns fixed answers; a mock also verifies how it was called; a virtual service replays recorded behaviour, timing included, through the real protocol.

Q2. Which defects did only the virtual gateway find?

H5 (late success after a timeout ignored) and H0 (a duplicate webhook after a compensating refund refunded the passenger again).

Q3. How does a wrong stub assumption hide a defect?

The team believed a timeout means failure; the stub and mock encoded that, so the build that drops late callbacks passed, while the real gateway charges and calls back late.

Q4. Why choose mocks plus the virtual service?

Mocks verify call arguments cheaply in unit tests; the virtual service found all 7 defects and is reused by integration and system tests.

Q5. Which real defect did you find and fix?

H0: only PAID was treated as a final state, so a webhook arriving after REFUNDED triggered a second refund; REFUNDED is now final too.

Task 4 – Integration strategies

Q1. List the stubs and drivers for top-down and bottom-up.

Top-down needs stubs for B, P, T, N and a driver for R; bottom-up needs no stubs and drivers for B, P, R, S, T.

Q2. What is fault masking in big-bang integration?

With all five faults present, the passenger-count error stopped bookings before the amount and decline paths ran, so only three symptoms appeared.

Q3. How did you estimate locating time?

20 minutes per suspect interface: big-bang has 6 suspects per failure (120 min), incremental steps 1–2 (20–40 min).

Q4. Where must P be integrated and why?

Early, after T and N and against the virtual gateway, because B and R both depend on it and four of the five faults touch its interfaces.

Q5. Why thread-based integration?

It gives a working book-pay-ticket-SMS path in week 2, allows two pairs to work in parallel, and located the faults in 360 minutes against 600 for big-bang.

Task 5 – Integration test plan

Q1. Name the eight B→P cases.

Amount in paise, integer currency, timeout to pending, duplicate callback, paid-but-ticket-failed, retry after timeout, passenger count, callback after session expiry.

Q2. Why is the build-schedule plan approved?

It gives the commercial officer dates and entry/exit criteria per build; filled with interface cases and scenarios it covers all checklist items.

Q3. How does each plan cope with a late Notification module?

Interface plan continues except T→N and R→N; scenario plan is blocked unless N is stubbed; build plan slips build 1 and stubs N.

Q4. State the integration exit criteria.

All interface cases pass, both threads pass end to end, payment and ticket status agree in every run, no open severity-1/2 defect, earlier builds still green.

Q5. Can an LLM-generated plan be accepted?

Not without checking interfaces and error cases, call-graph order, measurable criteria, traceability to quality targets, environment items and dates.

Task 6 – System testing

Q1. Which trial-week problems did each focus area expose?

Functional: double allotment and payment without ticket; performance: slow search; recovery: payment without ticket after a crash.

Q2. Where does the 3-second requirement fail?

In the model with 8 workers: trial build at about 950 users, fixed (indexed) build at about 3,150 users.

Q3. Why is the load test calibrated from measured times?

Search and booking times were measured on a 20-lakh-row table; the server and users were then simulated, so the result depends on stated assumptions (workers, think time, overhead).

Q4. What real defect did recovery testing find?

R2: when the gateway connection fails with a network error, booked berths are not released.

Q5. Why not run the load test once, a week before release?

The trial build failed far below the target and needed an index fix; found a week before release there is no time to fix and re-test.

Task 7 – Regression

Q1. How is APFD calculated?

APFD = $1 - (TF_1 + \ldots + TF_m) \div (n \times m) + 1 \div 2n$, where TF_i is the position of the first test that detects fault i.

Q2. What happened with the outdated traceability sheet?

Impact-based selection chose 14 R tests and found only G1; G2 (SMS) and G3 (paise) were missed.

Q3. Why did minimisation fail on the new faults?

It kept 7 tests that cover the historical faults, but the new faults sit in tests that had never failed before.

Q4. What should happen to tests that always pass?

Keep those guarding critical rules in the full run and rotate the rest; delete only tests for removed features.

Q5. What is your smoke suite?

12 tests: two per module, chosen for business criticality – under a minute automated.

Task 8 – Acceptance

Q1. What does alpha find that beta does not?

Counter workflow problems observed directly – speed, keyboard flow, explaining refunds – with the developers present.

Q2. How should open defects at the end of beta be handled?

Map each to the acceptance criteria: any that breaks a criterion blocks acceptance; others are fixed, worked around with sign-off, or backlogged by severity.

Q3. Can 50 beta users represent 1.5 lakh travellers?

Only for common problems: a problem affecting 1 % of travellers is seen with 39.5 % probability, and a satisfaction figure has a ±14-point margin.

Q4. Give two measurable acceptance criteria.

95 % of bookings complete in under 3 minutes; payment and ticket status agree in 100 % of 50 runs including ticket-service failure.

Q5. In what order and duration do you run them?

Alpha in week 3 (2 days), beta in weeks 4–5 (8 days), formal acceptance at the end of week 5 (2 days).

Task 9 – Multi-level strategy

Q1. Which strategy found all ten faults and at what cost?

The balanced pyramid: DRE 100 %, ₹57,000 by level of discovery, all faults by week 4.

Q2. What did the bottom-heavy strategy leak?

Double allotment, SQL injection and the missing reconciliation – faults only system tests expose.

Q3. Which prediction was contradicted?

That strong lower levels make a short system test enough.

Q4. What did the festival-data experiment show?

Past booking patterns still revealed the race but under-stated it: concentrated festival bookings produced about 2.6 times as many double allotments.

Q5. Why not the AI-assisted strategy?

Low repeatability, wrong oracles and invented rules (Tasks 1–2), and selection from commit text misses changes (Task 7).

Task 10 – Adaptation

Q1. How many tester-hours per release now?

2 testers × 10 days × 8 h = 160 h; a manual full regression alone is 96 h.

Q2. How much capacity does the Tatkal burst need?

The model gives p90 ≈ 34 s on 8 workers and 1.4 s on 48 workers – about 6 times today.

Q3. Why did the automated pipeline miss N3?

Its mocked seat service returned the field the consumer expected; only the contract test against the provider's new build showed the rename.

Q4. What did the canary cost?

It found the wallet race but 50 canary users were double-debited before rollback, so a parallel-session wallet test was added to the pipeline.

Q5. When would REDESIGN not hold?

If the provider team will not run consumer contracts or the service becomes an in-process library.

ANNEXURE H – Defect Reports: Trial-Week Problems and Defects Found by this Answer

RE-01 – Waitlisted ticket confirmed twice for the same berth

Field	Content
Module	Booking / inventory
Steps / input	20 parallel bookings for the last 8 berths (system test F7)
Expected	Each berth allotted to one passenger
Actual	Same berth allotted to two bookings (trial build)
Severity	Critical
Cause, fix and detecting test	No lock between availability check and allotment (C1). Fix: atomic allotment (lock / unique constraint).

RE-02 – Refund calculated on the wrong cancellation slab

Field	Content
Module	Cancellation and refund
Steps / input	Cancel 11.9 h (or exactly 48 h) before departure
Expected	50 % deduction at 11.9 h; ₹60 per passenger at 48 h
Actual	25 % / lower-slab deduction
Severity	Major
Cause, fix and detecting test	Boundary coded wrongly (U2 / R1 type); caught by the specification-based boundary rows (Tasks 1–2).

RE-03 – Payment successful but no ticket generated

Field	Content
Module	Payment → Ticket
Steps / input	Ticket service down after a successful charge (F8); crash between payment and ticket (R1)
Expected	Refund, or ticket issued on restart; statuses agree
Actual	Money taken, no ticket
Severity	Critical
Cause, fix and detecting test	No compensation (H1) and no reconciliation (S3); found by system and recovery tests.

RE-04 – Site slow when 400 users search together

Field	Content
Module	Search
Steps / input	Load model at 400–1,000 users, trial build
Expected	p90 ≤ 3 s at 1,500 users
Actual	Slow on the un-indexed table (fails ≈ 500 users on 4 workers, ≈ 950 on 8)
Severity	Major
Cause, fix and detecting test	Full table scan on availability; index fixes it (fails only above ≈ 3,150 users).

RE-05 – Duplicate webhook after a compensating refund refunds twice (found by this answer)

Field	Content
Module	Payment

Field	Content
Steps / input	Ticket fails → refund → gateway's duplicate success webhook arrives
Expected	Webhook ignored
Actual	Second refund issued
Severity	Critical
Cause, fix and detecting test	Only PAID treated as final (H0); found only by the virtual gateway. Fixed: REFUNDED is final.

RE-06 – Berths leak when the gateway connection fails (found by this answer)

Field	Content
Module	Booking
Steps / input	Gateway raises a network error during charge (R2)
Expected	Berths released
Actual	Berths stay allotted
Severity	Major
Cause, fix and detecting test	Booking releases berths only on GatewayError, not on connection errors; found by the recovery test on both builds.

ANNEXURE I – Catalogue of Executed Unit and Harness Test Cases

All 89 pytest cases of Tasks 1–3 with their result on the correct build and the seeded faults each one exposed (from results_unit.jsonl and results_t3.jsonl).

#	Suite	Test case (inputs)	Correct	Faults exposed
1	t1_a_adhoc	test_fare_two_adults_3a	Pass	–
2	t1_a_adhoc	test_refund_day_before	Pass	–
3	t1_a_adhoc	test_refund_too_late	Pass	–
4	t1_ai	test_between_12_and_48	Pass	–
5	t1_ai	test_between_4_and_12	Pass	–
6	t1_ai	test_less_than_4	Pass	–
7	t1_ai	test_more_than_48_hours	Pass	–
8	t1_ai	test_multiple_passengers	Pass	U4
9	t1_ai	test_negative	Pass	R4
10	t1_ai	test_no_show	Pass	R3
11	t1_ai	test_small_fare_not_negative	Pass	–
12	t1_b_plan	test_fare[2A-1-30-True-2457.0]	Pass	–
13	t1_b_plan	test_fare[3A-1-30-False-1260.0]	Pass	–
14	t1_b_plan	test_fare[3A-1-60-True-1134.0]	Pass	U3
15	t1_b_plan	test_fare[SL-1-30-False-450.0]	Pass	U1
16	t1_b_plan	test_fare[SL-1-59-False-450.0]	Pass	U1
17	t1_b_plan	test_fare[SL-1-60-False-270.0]	Pass	U1, U3
18	t1_b_plan	test_fare[SL-2-age6-False-720.0]	Pass	U1
19	t1_b_plan	test_fare[SL-6-30-False-2700.0]	Pass	U1
20	t1_b_plan	test_fare_invalid[1A-1]	Pass	–
21	t1_b_plan	test_fare_invalid[SL-0]	Pass	–
22	t1_b_plan	test_fare_invalid[SL-7]	Pass	–
23	t1_b_plan	test_refund[11.9-1000-1-500.0]	Pass	U2
24	t1_b_plan	test_refund[12-1000-1-750.0]	Pass	–
25	t1_b_plan	test_refund[3.9-1000-1-0.0]	Pass	–
26	t1_b_plan	test_refund[4-1000-1-500.0]	Pass	–
27	t1_b_plan	test_refund[47.9-1000-1-750.0]	Pass	–
28	t1_b_plan	test_refund[48-1000-1-940.0]	Pass	–
29	t1_b_plan	test_refund[48-1350-3-1170.0]	Pass	U4
30	t1_b_plan	test_refund[None-1000-1-0.0]	Pass	–
31	t1_b_plan	test_refund_negative	Pass	–
32	t1_c_bdd	test_cancel_evening_before	Pass	–
33	t1_c_bdd	test_cancel_six_hours_before	Pass	–
34	t1_c_bdd	test_cancel_two_days_before_two_passengers	Pass	U4
35	t1_c_bdd	test_cancel_two_hours_before	Pass	–
36	t1_c_bdd	test_family_of_four_in_sleeper	Pass	U1
37	t1_c_bdd	test_no_show	Pass	–
38	t1_c_bdd	test_senior_in_ac3	Pass	–
39	t1_c_bdd	test_tatkal_ac2	Pass	–
40	t2_a_spec	test_25_percent_three_passengers	Pass	R2
41	t2_a_spec	test_after_departure_invalid	Pass	R4
42	t2_a_spec	test_boundary_12	Pass	–
43	t2_a_spec	test_boundary_4	Pass	–
44	t2_a_spec	test_boundary_48	Pass	R1
45	t2_a_spec	test_class_48_plus	Pass	–
46	t2_a_spec	test_departure_time	Pass	–
47	t2_a_spec	test_just_below_12	Pass	–
48	t2_a_spec	test_just_below_4	Pass	–
49	t2_a_spec	test_just_below_48	Pass	–
50	t2_a_spec	test_no_show	Pass	R3
51	t2_a_spec	test_per_passenger_deduction	Pass	R1
52	t2_b_struct	test_low_branch	Pass	–
53	t2_b_struct	test_mid_branch	Pass	–

#	Suite	Test case (inputs)	Correct	Faults exposed
54	t2_b_struct	test_negative_branch	Pass	R4
55	t2_b_struct	test_none_left_branch	Pass	–
56	t2_b_struct	test_noshow_branch	Pass	R3
57	t2_b_struct	test_top_branch	Pass	–
58	t2_b_struct	test_top_branch_floor_zero	Pass	–
59	t2_b_struct	test_top_branch_two_passengers	Pass	–
60	t2_c_data	test_refund_after_departure_rejected	Pass	R4
61	t2_c_data	test_refund_table[12 h boundary]	Pass	–
62	t2_c_data	test_refund_table[25 % of the whole fare]	Pass	R2
63	t2_c_data	test_refund_table[4 h boundary]	Pass	–
64	t2_c_data	test_refund_table[48 h boundary]	Pass	R1
65	t2_c_data	test_refund_table[48 h or more: \u20b960 per passenger]	Pass	–
66	t2_c_data	test_refund_table[\u20b960 \xd7 3 passengers]	Pass	R1
67	t2_c_data	test_refund_table[just under 12 h: 50 %]	Pass	–
68	t2_c_data	test_refund_table[just under 48 h: 25 %]	Pass	–
69	t2_c_data	test_refund_table[no-show]	Pass	R3
70	t2_c_data	test_refund_table[under 4 h: nothing]	Pass	–
71	t3_a_stubs	test_cancel_refunds_once	Pass	H6
72	t3_a_stubs	test_decline_releases_berths	Pass	H3
73	t3_a_stubs	test_duplicate_callback_one_ticket	Pass	H2
74	t3_a_stubs	test_paid_but_ticket_fails_is_refunded	Pass	H1
75	t3_a_stubs	test_success_gives_ticket	Pass	–
76	t3_a_stubs	test_timeout_not_confirmed	Pass	–
77	t3_b_mocks	test_cancel_refunds_once_with_right_amount	Pass	H6
78	t3_b_mocks	test_decline_releases_berths	Pass	H3
79	t3_b_mocks	test_duplicate_callback_one_ticket	Pass	H2
80	t3_b_mocks	test_paid_but_ticket_fails_is_refunded	Pass	H1
81	t3_b_mocks	test_success_charges_exact_paise	Pass	H4
82	t3_b_mocks	test_timeout_not_confirmed	Pass	–
83	t3_c_virtual	test_amount_contract_paise	Pass	H4
84	t3_c_virtual	test_cancel_refunds_once	Pass	H6
85	t3_c_virtual	test_decline_releases_berths	Pass	H3
86	t3_c_virtual	test_duplicate_callback_one_ticket	Pass	H2
87	t3_c_virtual	test_paid_but_ticket_fails_is_refunded	Pass	H0, H1, H2, H4
88	t3_c_virtual	test_success_gives_one_ticket_even_with_webhook	Pass	H2, H4
89	t3_c_virtual	test_timeout_then_late_success_is_settled	Pass	H4, H5

ANNEXURE J – The 60-Test Regression Suite (Task 7)

Traceability (current and outdated sheet), criticality, measured fault history (failures on the 18 earlier seeded faults), whether the test executes the changed refund code, the priority score and the regression faults it detects.

Test	Module	Sheet (current)	Sheet (outdated)	Crit.	History	Changed code	Score	Detects
S01	S	S	S	1	0	no	0.07	–
S02	S	S	S	1	0	no	0.07	–
S03	S	S	S	1	0	no	0.07	–
S04	S	S	S	1	0	no	0.07	–
S05	S	S	S	1	0	no	0.07	–
S06	S	S	S	1	0	no	0.07	–
S07	S	S	S	1	0	no	0.07	–
S08	S	S	S	1	0	no	0.07	–
B01	B	B,P	B,P	2	5	no	0.47	–
B02	B	B,P	B,P	2	6	no	0.53	–
B03	B	B,P	B,P	2	4	no	0.40	–
B04	B	B,P	B,P	2	4	no	0.40	–
B05	B	B,P	B,P	2	4	no	0.40	–
B06	B	B,P	B,P	2	5	no	0.47	–
B07	B	B,P	B,P	2	4	no	0.40	–
B08	B	B,P	B,P	2	4	no	0.40	–
B09	B	B	B	2	0	no	0.13	–
B10	B	B	B	2	2	no	0.27	–
B11	B	B,P	B,P	2	1	no	0.20	–
B12	B	B	B	3	0	no	0.20	–
P01	P	P,T	P,T	3	4	no	0.47	–
P02	P	P,T	P,T	3	4	no	0.47	–
P03	P	P,T	P,T	3	4	no	0.47	–
P04	P	P,T	P,T	3	4	no	0.47	–
P05	P	P,T	P,T	3	4	no	0.47	–
P06	P	P,T	P,T	3	2	no	0.33	–
P07	P	P,T	P,T	3	2	no	0.33	–
P08	P	P	P	3	0	no	0.20	–
P09	P	P,R	P	3	1	yes	0.67	G3
P10	P	P	P	3	1	no	0.27	–
T01	T	T,N	T,N	2	2	no	0.27	–
T02	T	T,N	T,N	2	2	no	0.27	–
T03	T	T,N	T,N	2	2	no	0.27	–
T04	T	T,N	T,N	2	2	no	0.27	–
T05	T	T,N	T,N	2	2	no	0.27	–
T06	T	T,N	T,N	2	2	no	0.27	–
T07	T	T,N	T,N	2	2	no	0.27	–
T08	T	T,N	T,N	2	2	no	0.27	–
R01	R	R,P	R,P	3	1	yes	0.67	–
R02	R	R,P	R,P	3	3	yes	0.80	–
R03	R	R,P	R,P	3	1	yes	0.67	–
R04	R	R,P	R,P	3	1	yes	0.67	–
R05	R	R,P	R,P	3	1	yes	0.67	–
R06	R	R,P	R,P	3	0	yes	0.60	–
R07	R	R,P	R,P	3	0	yes	0.60	G1
R08	R	R,P	R,P	3	1	yes	0.67	–
R09	R	R,P	R,P	3	1	yes	0.67	–
R10	R	R,P	R,P	3	1	yes	0.67	–
R11	R	R,P	R,P	3	1	yes	0.67	–

Test	Module	Sheet (current)	Sheet (outdated)	Crit.	History	Changed code	Score	Detects
R12	R	R,P	R,P	3	1	yes	0.67	–
R13	R	R	R	2	3	yes	0.73	–
R14	R	R	R	2	1	yes	0.60	–
N01	N	N	N	1	2	no	0.20	–
N02	N	N	N	1	2	no	0.20	–
N03	N	N	N	1	2	no	0.20	–
N04	N	N	N	1	2	no	0.20	–
N05	N	N	N	1	2	no	0.20	–
N06	N	N	N	1	2	no	0.20	–
N07	N	N,R	N	2	0	yes	0.53	G2
N08	N	N,R	N	2	1	yes	0.60	–

ANNEXURE K – Integration (B→P) and System Test Results by Build

B→P case	none	I1	I2	H1	H2	H4	H5
BP-01 amount in paise	Pass	Fail	Fail	Pass	Pass	Fail	Pass
BP-02 currency/int type	Pass	Fail	Fail	Pass	Pass	Fail	Pass
BP-03 timeout → pending	Pass	Fail	Pass	Pass	Pass	Pass	Pass
BP-04 duplicate payment callback	Pass	Pass	Pass	Pass	Fail	Pass	Pass
BP-05 partial failure (paid, ticket fails)	Pass	Fail	Pass	Fail	Pass	Pass	Pass
BP-06 retry after timeout	Pass	Fail	Pass	Pass	Pass	Pass	Pass
BP-07 passenger count	Pass	Fail	Pass	Pass	Pass	Pass	Pass
BP-08 callback after session expiry	Pass	Pass	Pass	Pass	Pass	Pass	Fail

System test	Focus	Correct build	Trial build
F1 family booking with seniors	A	Pass	Pass
F2 7 passengers rejected	A	Pass	Pass
F3 121 days ahead rejected	A	Pass	Pass
F4 cancel 30 h before: 25 % deduction	A	Pass	Pass
F5 e-ticket and SMS issued	A	Pass	Pass
F6 waitlist after the last berth	A	Pass	Pass
F7 no double allotment under 20 parallel bookings	A	Pass	Fail
F8 paid but no ticket → refunded	A	Pass	Fail
R1 crash after payment → reconciled on restart	C	Pass	Fail
R2 gateway down → berths kept	C	Fail	Fail
R3 duplicate webhook after restart	C	Pass	Pass
SEC1 SQL injection in station field	C	Pass	Fail
SEC2 session token dead after logout	C	Pass	Fail
SEC3 empty passenger list rejected	C	Pass	Pass

Load-model results at every user level

Users	Trial avg (s)	Trial p90 (s)	Fixed avg (s)	Fixed p90 (s)	Requests simulated (fixed)
100	0.067	0.185	0.020	0.047	4,172
200	0.067	0.186	0.020	0.047	8,284
300	0.069	0.189	0.020	0.047	12,585
400	0.073	0.193	0.020	0.046	16,550
600	0.201	0.420	0.020	0.047	24,927
800	1.626	2.001	0.020	0.047	33,143
1000	3.266	3.671	0.021	0.047	41,680
1500	7.451	7.964	0.024	0.054	62,715
2000	11.523	12.344	0.115	0.230	81,841
2500	15.823	17.977	1.297	1.458	83,264
3000	20.162	24.567	2.559	2.731	83,243

Tatkal burst capacity (Task 10)

Workers	Average (s)	90th percentile (s)	Requests simulated
8	32.92	33.64	35,758
16	13.91	14.06	71,454
32	4.44	4.54	143,068
48	1.29	1.36	214,820
64	0.02	0.05	269,263
96	0.02	0.05	268,577

ANNEXURE L – Execution Log of the Correct Build (pytest -v)

```
============================= test session starts ==============================
test_t1_a_adhoc.py::test_fare_two_adults_3a PASSED                  [  1%]
test_t1_a_adhoc.py::test_refund_too_late PASSED                     [  2%]
test_t1_a_adhoc.py::test_refund_day_before PASSED                   [  3%]
test_t1_ai.py::test_more_than_48_hours PASSED                       [  4%]
test_t1_ai.py::test_between_12_and_48 PASSED                        [  5%]
test_t1_ai.py::test_between_4_and_12 PASSED                         [  6%]
test_t1_ai.py::test_less_than_4 PASSED                              [  7%]
test_t1_ai.py::test_no_show PASSED                                  [  8%]
test_t1_ai.py::test_negative PASSED                                 [ 10%]
test_t1_ai.py::test_small_fare_not_negative PASSED                  [ 11%]
test_t1_ai.py::test_multiple_passengers PASSED                      [ 12%]
test_t1_b_plan.py::test_fare[SL-1-30-False-450.0] PASSED            [ 13%]
test_t1_b_plan.py::test_fare[3A-1-30-False-1260.0] PASSED           [ 14%]
test_t1_b_plan.py::test_fare[2A-1-30-True-2457.0] PASSED            [ 15%]
test_t1_b_plan.py::test_fare[SL-1-60-False-270.0] PASSED            [ 16%]
test_t1_b_plan.py::test_fare[SL-1-59-False-450.0] PASSED            [ 17%]
test_t1_b_plan.py::test_fare[3A-1-60-True-1134.0] PASSED            [ 19%]
test_t1_b_plan.py::test_fare[SL-2-age6-False-720.0] PASSED          [ 20%]
test_t1_b_plan.py::test_fare[SL-6-30-False-2700.0] PASSED           [ 21%]
test_t1_b_plan.py::test_fare_invalid[SL-0] PASSED                   [ 22%]
test_t1_b_plan.py::test_fare_invalid[SL-7] PASSED                   [ 23%]
test_t1_b_plan.py::test_fare_invalid[1A-1] PASSED                   [ 24%]
test_t1_b_plan.py::test_refund[48-1000-1-940.0] PASSED              [ 25%]
test_t1_b_plan.py::test_refund[47.9-1000-1-750.0] PASSED            [ 26%]
test_t1_b_plan.py::test_refund[12-1000-1-750.0] PASSED              [ 28%]
test_t1_b_plan.py::test_refund[11.9-1000-1-500.0] PASSED            [ 29%]
test_t1_b_plan.py::test_refund[4-1000-1-500.0] PASSED               [ 30%]
test_t1_b_plan.py::test_refund[3.9-1000-1-0.0] PASSED               [ 31%]
test_t1_b_plan.py::test_refund[None-1000-1-0.0] PASSED              [ 32%]
test_t1_b_plan.py::test_refund[48-1350-3-1170.0] PASSED             [ 33%]
test_t1_b_plan.py::test_refund_negative PASSED                      [ 34%]
test_t1_c_bdd.py::test_family_of_four_in_sleeper PASSED             [ 35%]
test_t1_c_bdd.py::test_senior_in_ac3 PASSED                         [ 37%]
test_t1_c_bdd.py::test_tatkal_ac2 PASSED                            [ 38%]
test_t1_c_bdd.py::test_cancel_two_days_before_two_passengers PASSED [ 39%]
test_t1_c_bdd.py::test_cancel_evening_before PASSED                 [ 40%]
test_t1_c_bdd.py::test_cancel_six_hours_before PASSED               [ 41%]
test_t1_c_bdd.py::test_no_show PASSED                               [ 42%]
test_t1_c_bdd.py::test_cancel_two_hours_before PASSED               [ 43%]
test_t2_a_spec.py::test_class_48_plus PASSED                        [ 44%]
test_t2_a_spec.py::test_boundary_48 PASSED                          [ 46%]
test_t2_a_spec.py::test_just_below_48 PASSED                        [ 47%]
test_t2_a_spec.py::test_boundary_12 PASSED                          [ 48%]
test_t2_a_spec.py::test_just_below_12 PASSED                        [ 49%]
test_t2_a_spec.py::test_boundary_4 PASSED                           [ 50%]
test_t2_a_spec.py::test_just_below_4 PASSED                         [ 51%]
test_t2_a_spec.py::test_departure_time PASSED                       [ 52%]
test_t2_a_spec.py::test_no_show PASSED                              [ 53%]
test_t2_a_spec.py::test_after_departure_invalid PASSED              [ 55%]
test_t2_a_spec.py::test_per_passenger_deduction PASSED              [ 56%]
test_t2_a_spec.py::test_25_percent_three_passengers PASSED          [ 57%]
test_t2_b_struct.py::test_noshow_branch PASSED                      [ 58%]
test_t2_b_struct.py::test_negative_branch PASSED                    [ 59%]
test_t2_b_struct.py::test_top_branch PASSED                         [ 60%]
test_t2_b_struct.py::test_top_branch_floor_zero PASSED              [ 61%]
test_t2_b_struct.py::test_mid_branch PASSED                         [ 62%]
test_t2_b_struct.py::test_low_branch PASSED                         [ 64%]
test_t2_b_struct.py::test_none_left_branch PASSED                   [ 65%]
test_t2_b_struct.py::test_top_branch_two_passengers PASSED          [ 66%]
test_t2_c_data.py::test_refund_table[48 h or more: \u20b960 per passenger] PASSED [ 67%]
test_t2_c_data.py::test_refund_table[48 h boundary] PASSED          [ 68%]
test_t2_c_data.py::test_refund_table[just under 48 h: 25 %] PASSED [ 69%]
test_t2_c_data.py::test_refund_table[\u20b960 \xd7 3 passengers] PASSED [ 70%]
test_t2_c_data.py::test_refund_table[25 % of the whole fare] PASSED [ 71%]
test_t2_c_data.py::test_refund_table[12 h boundary] PASSED          [ 73%]
test_t2_c_data.py::test_refund_table[just under 12 h: 50 %] PASSED [ 74%]
test_t2_c_data.py::test_refund_table[4 h boundary] PASSED           [ 75%]
test_t2_c_data.py::test_refund_table[under 4 h: nothing] PASSED     [ 76%]
test_t2_c_data.py::test_refund_table[no-show] PASSED                [ 77%]
test_t2_c_data.py::test_refund_after_departure_rejected PASSED      [ 78%]
test_t3_a_stubs.py::test_success_gives_ticket PASSED                [ 79%]
test_t3_a_stubs.py::test_decline_releases_berths PASSED             [ 80%]
```

```
test_t3_a_stubs.py::test_timeout_not_confirmed PASSED              [ 82%]
test_t3_a_stubs.py::test_duplicate_callback_one_ticket PASSED      [ 83%]
test_t3_a_stubs.py::test_paid_but_ticket_fails_is_refunded PASSED  [ 84%]
test_t3_a_stubs.py::test_cancel_refunds_once PASSED                [ 85%]
test_t3_b_mocks.py::test_success_charges_exact_paise PASSED        [ 86%]
test_t3_b_mocks.py::test_decline_releases_berths PASSED            [ 87%]
test_t3_b_mocks.py::test_timeout_not_confirmed PASSED              [ 88%]
test_t3_b_mocks.py::test_duplicate_callback_one_ticket PASSED      [ 89%]
test_t3_b_mocks.py::test_paid_but_ticket_fails_is_refunded PASSED  [ 91%]
test_t3_b_mocks.py::test_cancel_refunds_once_with_right_amount PASSED [ 92%]
test_t3_c_virtual.py::test_success_gives_one_ticket_even_with_webhook PASSED [ 93%]
test_t3_c_virtual.py::test_decline_releases_berths PASSED          [ 94%]
test_t3_c_virtual.py::test_timeout_then_late_success_is_settled PASSED [ 95%]
test_t3_c_virtual.py::test_duplicate_callback_one_ticket PASSED    [ 96%]
test_t3_c_virtual.py::test_paid_but_ticket_fails_is_refunded PASSED [ 97%]
test_t3_c_virtual.py::test_cancel_refunds_once PASSED              [ 98%]
test_t3_c_virtual.py::test_amount_contract_paise PASSED            [100%]
============================== 89 passed in 4.86s ==============================
```

ANNEXURE M – Acceptance Test Case Specifications

AT-01 – Booking completes in under 3 minutes

Field	Content
Precondition	Clerk logged in; passenger details available
Steps / data	Book 3 passengers SL incl. a senior, pay by card
Acceptance criterion	≥ 95 % of 20 timed bookings < 3 min
Result	
Signed	

AT-02 – Fare correct for every class, Tatkal and senior

Field	Content
Precondition	Fare table loaded
Steps / data	20 sample tickets (3 classes × Tatkal × senior × 1–6 passengers)
Acceptance criterion	100 % equal to the fare table to the paisa
Result	
Signed	

AT-03 – Refund correct on every slab

Field	Content
Precondition	Booked tickets
Steps / data	Cancel at 72, 48, 47.9, 12, 11.9, 4, 3.9 h and no-show
Acceptance criterion	100 % equal to the refund table
Result	
Signed	

AT-04 – No double allotment

Field	Content
Precondition	8 berths left
Steps / data	20 parallel bookings
Acceptance criterion	Each berth owned by exactly one booking
Result	
Signed	

AT-05 – Payment and ticket status agree

Field	Content
Precondition	Ticket service can be stopped
Steps / data	50 bookings, 5 with ticket service down
Acceptance criterion	Every booking ends PAID + ticket, or REFUNDED
Result	
Signed	

AT-06 – Performance at 1,500 users

Field	Content
Precondition	Production-like staging
Steps / data	JMeter plan F0 at 1,500 users for 10 min
Acceptance criterion	p90 ≤ 3 s, errors < 1 %
Result	
Signed	

AT-07 – E-ticket SMS within 60 s

Field	Content
Precondition	SMS sandbox
Steps / data	50 bookings
Acceptance criterion	≥ 95 % SMS within 60 s
Result	
Signed	

AT-08 – Booking limits enforced

Field	Content
Precondition	–
Steps / data	7 passengers; journey 121 days ahead
Acceptance criterion	Both refused with a clear message
Result	
Signed	

AT-09 – GST shown separately on AC tickets

Field	Content
Precondition	AC booking
Steps / data	Open e-ticket
Acceptance criterion	GST line present on AC, absent on Sleeper
Result	
Signed	

AT-10 – Daily refund report

Field	Content
Precondition	Commercial officer login
Steps / data	Export report for one day
Acceptance criterion	Totals equal the sum of refunds in the database
Result	
Signed	

ANNEXURE N – Master Test Plan for RailEase Go-Live (IEEE 829 structure)

1. Identifier and introduction

RE-MTP-1.0 for the go-live before 5 January; the plan combines the choices of Tasks 1–9 (balanced pyramid).

2. Test items

Modules S, B, P, T, R, N; virtual payment gateway; SMS sandbox; staging copy of 18 months of anonymised data.

3. Features to be tested

Fare and refund rules; booking limits; berth uniqueness; payment–ticket agreement; notifications; search performance; recovery and security.

4. Features not to be tested

Agent commission settlement (not in this release); legacy counter terminals.

5. Approach

Plan-driven unit tests with BDD examples; data-driven refund tests; mocks and virtual gateway; thread-based integration with a build-schedule plan; system tests (functional 45 %, performance 30 %, recovery/security/compatibility 25 %); prioritised regression with a 12-test smoke suite; alpha, beta and formal acceptance.

6. Pass/fail criteria

Results equal the fare and refund tables; statuses agree; no double allotment; p90 $\leq$ 3 s at 1,500 users.

7. Suspension and resumption

Suspend if the smoke suite fails or more than 3 critical defects are open; resume when fixed and smoke passes.

8. Deliverables

Plans, test cases, CI reports, load report, defect reports, acceptance record signed by the commercial officer.

9. Environment

Test, staging (production worker count), UAT; JMeter injectors; virtual gateway; SMS sandbox.

10. Responsibilities

Developers: unit tests. Testers: integration, system, regression. Clerk representative: BDD examples and alpha. Commercial officer: acceptance.

11. Schedule

Week 1 unit; week 2 integration thread 1 and baseline load test; week 3 thread 2, system tests, alpha; weeks 4–5 beta, regression, full load test, acceptance.

12. Risks

Gateway sandbox behaviour differs from production (mitigation: recorded virtual gateway); festival concentration (mitigation: concentrated-booking concurrency tests); late Notification module (mitigation: stub).

13. Approvals

Test lead, development lead, commercial officer.

ANNEXURE O – Formula Sheet with Worked Examples

Measure	Formula	Worked example
Fault-detection %	faults found ÷ faults seeded	Task 2 data-driven: 4 ÷ 4 = 100 %
Branch coverage	executed branch outcomes ÷ all outcomes	Task 1 B: 23 ÷ 24 = 96 %
Lines changed	difflib count between old and new test file	Task 2 C: 2
Locating time (model)	20 min × suspect interfaces	Big-bang: 5 faults × 6 × 20 = 600 min
DRE per level	found at level ÷ faults entering the level	Pyramid system level: 3 ÷ 3 = 100 %
Time saved	1 − selected time ÷ retest-all time	Prioritised: 1 − 2.2 ÷ 8.9 = 75 %
APFD	$1 - \Sigma TF_i \div (n \times m) + 1 \div 2n$	Prioritised: 1 − 35 ÷ 180 + 1 ÷ 120 = 0.81
Chance a beta user meets a problem	$1 - (1 - p)^n$	p = 1 %, n = 50 → 39.5 %
Margin of error (proportion)	$1.96 \times \sqrt{(0.25 \div n)}$	n = 50 → ±13.9 points
Tester-hours per release	testers × days × 8	2 × 10 × 8 = 160 h
Manual minutes per test	96 tester-hours × 60 ÷ 650	8.86 min

ANNEXURE P – Product Risk Register

ID	Risk	P	I	Exposure	Addressed by
RR-10	Tatkal burst overload	4	5	20	Automated 15,000-user load test
RR-04	Search slow at peak	4	4	16	Load model, JMeter plan
RR-01	Berth allotted twice at peak	3	5	15	F7, concentrated-booking test
RR-02	Payment taken without ticket	3	5	15	F8, R1, BP-05, BP-08, virtual gateway
RR-03	Refund on the wrong slab	3	4	12	Data-driven refund rows
RR-05	Gateway behaves unlike the stub	3	4	12	Virtual gateway from recorded traffic
RR-06	SQL injection in search	2	5	10	SEC1
RR-09	Seat service contract change	3	3	9	Contract tests (Task 10)
RR-07	Session reuse after logout	2	4	8	SEC2
RR-08	E-ticket unreadable on phones	3	2	6	Playwright viewport checks

ANNEXURE Q – Procedure Followed for Each Task

Task 1

- Step 1: Wrote ticket_fare and refund_amount in rail.py with switches U1–U4.
- Step 2: Wrote the plan, then three suites (ad hoc 3, plan-driven 20, BDD 8).
- Step 3: Ran exp12.py: each suite on the correct build and on each fault; coverage.py with --branch.

Task 2

- Step 1: Wrote three refund suites; seeded R1–R4; ran exp12.py.
- Step 2: Edited the suites for 12 h → 10 h in tests_v2/ and counted changed lines with difflib.
- Step 3: Captured pytest messages for R2.

Task 3

- Step 1: Built StubGateway, Mock-based tests and the HTTP virtual gateway (virtual_gateway.py).
- Step 2: Ran the three harness suites on H0–H6; measured run time with pytest.

Task 4

- Step 1: Wired the six modules with interface faults I1–I5; ran end-to-end checks per fault and all at once.
- Step 2: Computed orders, stubs and drivers from the call graph and the step at which each fault appears.

Task 5

- Step 1: Wrote the three plans; implemented the eight B→P cases; ran them on the correct build and on I1, I2, H1, H2, H4, H5.

Task 6

- Step 1: Ran 14 system tests on the correct and trial builds; Playwright checks at three viewports.
- Step 2: Measured search and booking time on a 20-lakh-row table; ran the load model for 100–3,000 users and the worker sensitivity.

Task 7

- Step 1: Built 60 regression tests with a traceability sheet; changed MID_HOURS to 10 with faults G1–G3.
- Step 2: Measured fault history on 18 earlier faults and changed-code coverage by tracing; computed selections, APFD and minimisation.

Task 8

- Step 1: Wrote the alpha plan, feedback form and ten acceptance criteria; checked the measurable ones on the prototype; computed beta representativeness.

Task 9

- Step 1: Ran unit, integration, system and acceptance levels of each strategy on ten faults (exp9.py); replayed spread and concentrated bookings with C1.

Task 10

- Step 1: Added UPI, wallet and the seat microservice (ext10.py) with N1–N6; ran strategies A, B and C (exp10.py) incl. contract test, canary and 15,000-user load model.

Commands

```
pip install pytest coverage playwright
python exp12.py   # Tasks 1–2
python exp36.py   # Tasks 3–6
python exp6b.py   # compatibility + load sensitivity
python exp7.py    # Task 7
python exp9.py    # Task 9
python exp10.py   # Task 10
```

ANNEXURE R – Integration Step Log per Strategy (Task 4)

For each strategy: the modules added at each step, the interfaces that become real at that step and the seeded faults that first show there (computed from the call graph and the end-to-end checks).

Top-down

Step	Modules added	Interfaces now real	Faults first visible	Stubs still needed	Drivers still needed
1	S	–	–	B	–
2	B	S→B	–	P	–
3	P	B→P	I1, I2	T	R
4	T	P→T	I3	N	R
5	N	T→N	I4	–	R
6	R	R→P, R→N	I5	–	–

Bottom-up

Step	Modules added	Interfaces now real	Faults first visible	Stubs still needed	Drivers still needed
1	N	–	–	–	R, T
2	T	T→N	I4	–	P, R
3	P	P→T	I3	–	B, R
4	B	B→P	I1, I2	–	R, S
5	R	R→P, R→N	I5	–	S
6	S	S→B	–	–	–

Thread-based

Step	Modules added	Interfaces now real	Faults first visible	Stubs still needed	Drivers still needed
1	S, B, P, T, N	S→B, B→P, P→T, T→N	I1, I2, I3, I4	–	R
2	R	R→P, R→N	I5	–	–

Big-bang

Step	Modules added	Interfaces now real	Faults first visible	Stubs still needed	Drivers still needed
1	S, B, P, T, N, R	S→B, B→P, P→T, T→N, R→P, R→N	I1, I2, I3, I4, I5	–	–

ANNEXURE S – Test Inventory per Level for the Three Strategies (Task 9)

Level	A Bottom-heavy	B Top-heavy	C Balanced pyramid
Unit	Plan-driven (20) + spec-based refund (12)	Ad hoc (3)	Plan-driven (20) + spec-based refund (12)
Integration	Mock harness (6) + B→P cases (8) + end-to-end interface checks (5)	none	Mock harness (6) + B→P cases (8) + end-to-end checks (5)
System	F1, F4, F5 (3)	F1–F8, R1, R3, SEC1–SEC3 (13) + end-to-end checks	F4, F7, F8, R1, R3, SEC1, SEC2 (7) + load test
Acceptance	BDD examples (8)	BDD examples (8)	BDD examples (8), then alpha / beta / formal acceptance
Faults found / leaked	7 / 3 (C1, S1, S3)	9 / 1 (U3)	10 / 0

ANNEXURE T – Alpha Session Guide and Beta Usability Questions (Task 8)

T1. Moderator guide for one 90-minute alpha session

- Welcome (5 min): purpose, "we test the system, not you", consent to observe and time.
- Warm-up (5 min): search trains MAS → CBE for next Friday.
- Tasks T1–T6 (60 min): read each task card aloud; time from first click to confirmation; note errors, hesitations and questions; do not help unless the user is stuck for 2 minutes.
- After each task: ease rating 1–5 and one sentence on what was hard.
- Debrief (15 min): three things to change, anything that would slow the counter queue, satisfaction 1–5.
- Record: issue, type (defect / usability / change request), severity 1–4, in the issue sheet.

T2. Usability questions added to the beta form (1 = strongly disagree … 5 = strongly agree)

#	Statement
1	I found a train for my journey quickly.
2	The fare shown matched what I expected to pay.
3	Paying was straightforward and I knew when it had succeeded.
4	I received my e-ticket and SMS promptly.
5	The refund shown when cancelling was clear.
6	The site worked well on my phone.
7	I would use RailEase again instead of the counter.

ANNEXURE U – Smoke Suite and Test Summary Report

U1. The 12-test smoke suite (run on every build)

#	Test	Module	Why it is in the smoke suite
1	B01 Sleeper adult booking, amount in paise	B, P	Most common booking, no GST
2	B03 AC 3-tier booking with GST	B, P	GST on AC
3	B07 AC 3-tier Tatkal senior	B	Tatkal and senior together
4	R02 cancel 48 h, 2 passengers	R	₹60 per passenger edge
5	R07 cancel 10 h (new edge)	R	Changed slab boundary
6	R11 cancel 3.9 h	R	No-refund edge
7	P06 paid but ticket fails → refund	P, T	Payment–ticket agreement
8	P07 duplicate webhook	P	Idempotency
9	P09 refund paise exact	R, P	Money to the paisa
10	B10 waitlist after last berth	B	Inventory limit
11	T01 e-ticket PNR	T	Ticket generation
12	N07 refund SMS amount	N, R	Customer-visible refund

U2. Test summary report (prototype)

Item	Result
Unit level	Plan-driven + data-driven suites: all 8 seeded unit faults found; 96–100 % branch coverage
Harness / integration	Virtual gateway 7 of 7 harness defects; B→P cases catch all interface faults; thread-based order
System	Trial build: 6 failing system tests + compatibility failure; correct build: one real defect (R2) open
Performance	Fixed build meets 3 s up to ≈ 3,150 users (model); Tatkal needs ≈ 48 workers
Regression	Prioritised order APFD 0.81; impact selection needs a current traceability sheet
Acceptance	Measurable criteria AT-02 to AT-06 and AT-08 pass on the prototype; volunteer sessions pending
Open defects	RE-06 berth leak on connection failure (fix before go-live)
Recommendation	Go-live after RE-06 is fixed, the JMeter plan passes on staging and the commercial officer signs AT-01 to AT-10

ANNEXURE V – Release Evidence Record for the Railway Board (Task 10)

Every critical check of the redesigned strategy, where it runs and the evidence item it leaves for each fortnightly release. The board's rule – traceable proof that every critical test ran – is met when every row has a stored artefact for the release.

Check	Runs in	Frequency	Evidence item stored	Measured run time (prototype)
Unit tests (fare, refund, UPI status, wallet, refund source)	CI	every commit	JUnit XML + coverage report	34 ms incl. integration
Integration with virtual gateway (B→P, R→P, P→T)	CI	every commit	JUnit XML, recorded request log	≈ 5 s
Prioritised regression + 12-test smoke suite	CI	every build	Selection log (inputs, model version), results	< 1 min automated
Consumer contract: seat service	CI of provider and consumer	every provider build	Contract verification result	1.0 s (incl. slow-provider check)
Wallet parallel-session test	CI	every build	Result + negative-balance count	< 1 s
Tatkal load test, 15,000 users	Load environment	every release candidate	Load report: p90, errors, workers	0.9 s (model)
Canary 5 % → 25 % → 100 %	Production	every release	Metrics dashboard snapshot, rollback log	2 days
Alpha with clerks (workflow changes only)	UAT	when counter flow changes	Session sheets	½ day
Release decision	Test lead + commercial officer	every release	Signed record linking all items above	–

Release evidence sign-off (template)

Release	Build	All critical checks green?	Open S1/S2 defects	Test lead	Commercial officer	Date